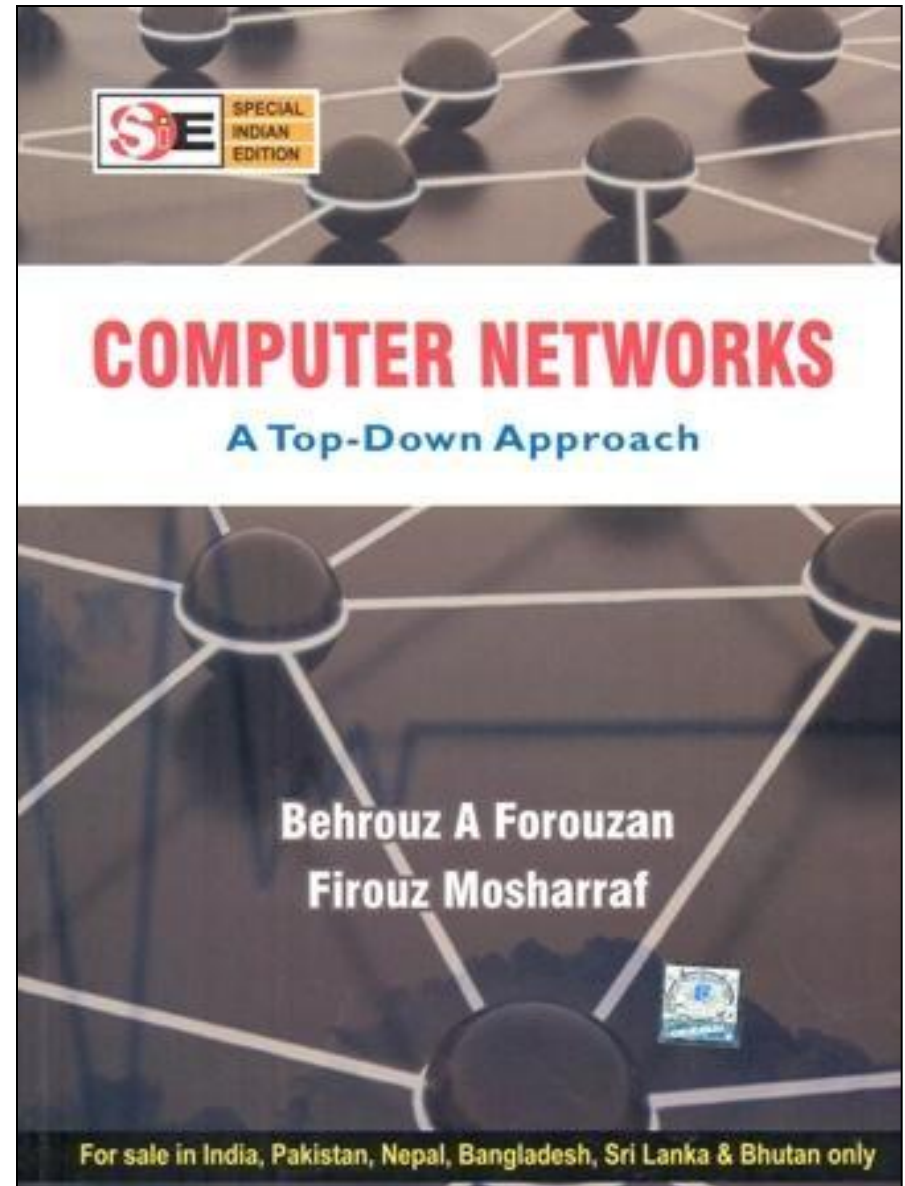


Chapter 3

Transport Layer





Chapter 3: Outline

3.1. Introduction to Transport Layer

3.2. Transport Layer Services

3.3. Multiplexing and de-multiplexing

3.4. Flow Control in Transport Layer

-Stop-and-wait

-Go-back-N

-Selective-Repeat

3.5.UDP: Services and Applications, Segment format

3.6. TCP: Services

-Segment format

-TCP Connection management

-State Transition Diagram

-Windows in TCP

-Flow Control

-Congestion Control (Slow start, congestion avoidance, and fast recovery)



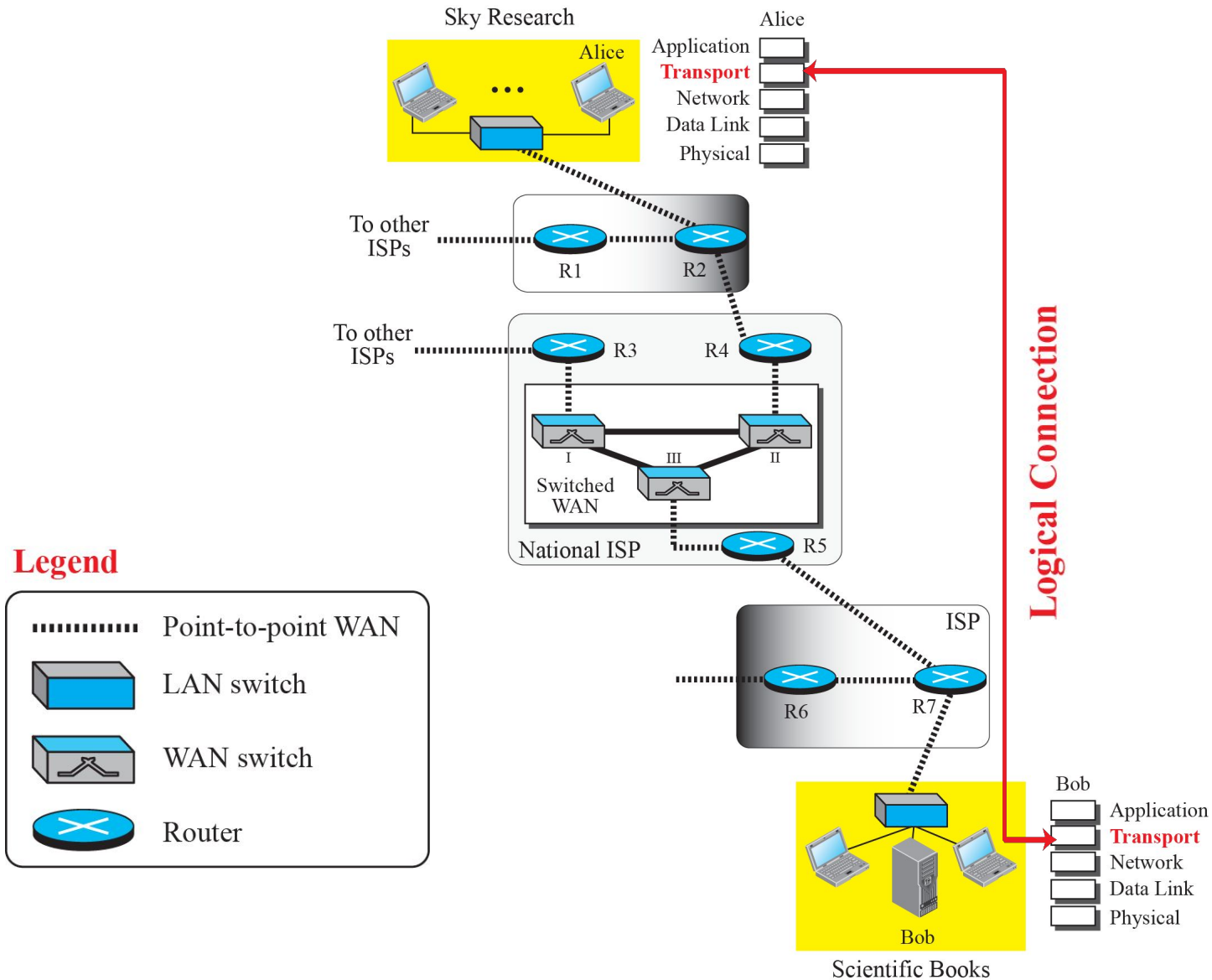
Chapter 3: Objective

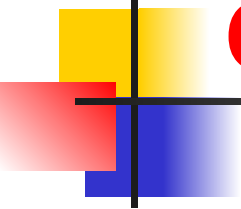
- ☐ *We introduce general services we normally require from the transport layer, such as process-to-process communication, addressing, multiplexing, error, flow, and congestion control.*
- ☐ *We discuss general transport-layer protocols such as Stop-and-Wait, Go-Back-N, and Selective-Repeat.*
- ☐ *We discuss UDP, which is the simpler of the two protocols we discuss in this chapter.*
- ☐ *We discuss TCP services and features. We then show how TCP provides a connection-oriented service using a transition diagram. Finally, we discuss flow and error control, and congestion control in TCP.*

INTRODUCTION

- The transport layer provides a process-to-process communication between two application layers.
- Communication is provided using a logical connection, which means that the two application layers assume that there is an imaginary direct connection through which they can send and receive messages.

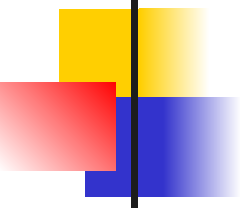
INTRODUCTION





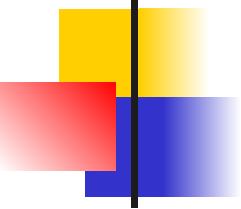
Chapter 3: Outline

3.2. Transport Layer Services



Transport-Layer Services

- As we discussed in Chapter 1, the transport layer is located between the network layer and the application layer.
- The transport layer is responsible for providing services to the application layer; it receives services from the network layer.
- In this section, we discuss the services that can be provided by the transport layer; in the next section, we discuss several transport-layer protocols.



Transport-Layer Services

☐ *Process-to-Process Communication*

☐ *Addressing: Port Numbers*

☐ *ICANN Ranges*

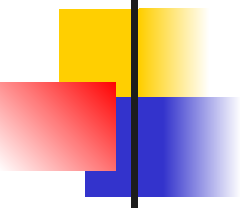
◆ *Well-known ports*

◆ *Registered ports*

◆ *Dynamic ports*

☐ *Encapsulation and Decapsulation*

☐ *Multiplexing and Demultiplexing*



Transport-Layer Services

Flow Control

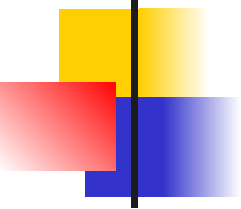
- ❖ *Pushing or Pulling*
- ❖ *Flow Control at Transport Layer*
- ❖ *Buffers*

Error Control

- ❖ *Sequence Numbers*
- ❖ *Acknowledgment*

Combination of Flow and Error Control

- ❖ *Sliding Window*



Transport-Layer Services

- *Congestion Control*

- *Connectionless and Connection-Oriented*

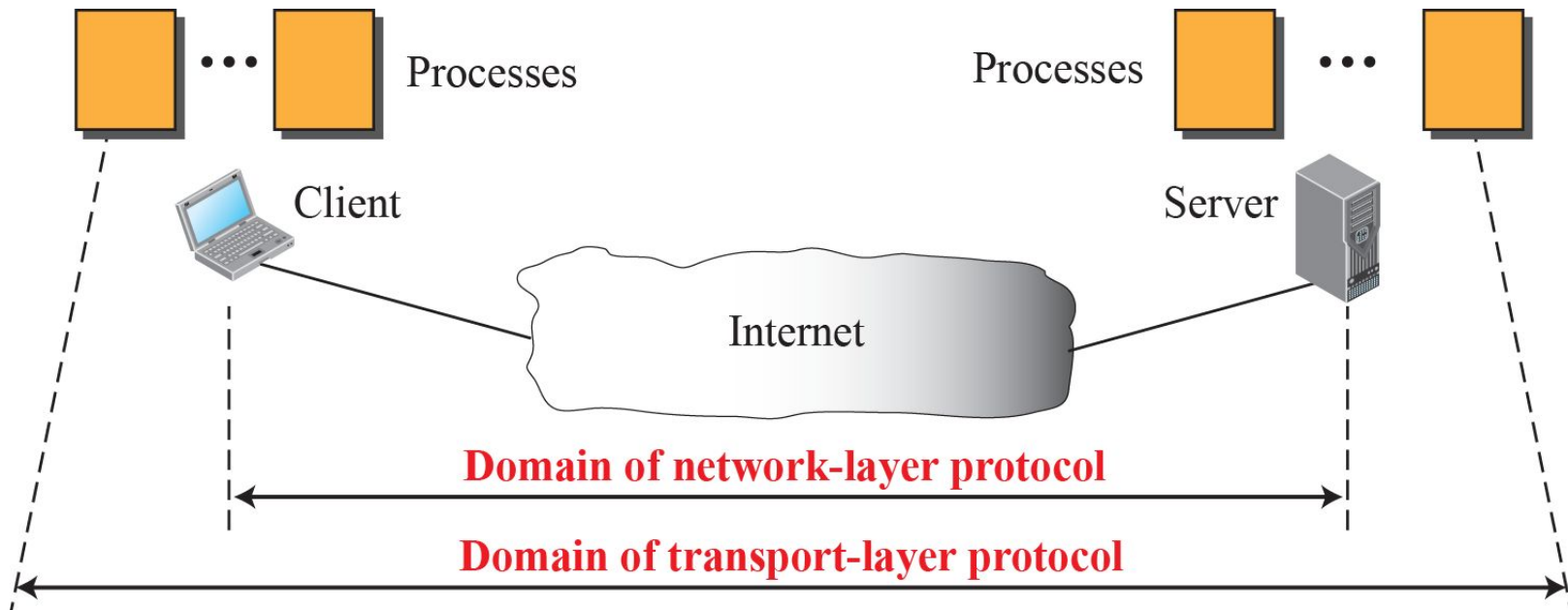
- ◆ *Connectionless Service*
- ◆ *Connection-Oriented Service*
- ◆ *Finite State Machine*

Transport-Layer Services

- Consumers communicate with the producers on two occasions: when the buffer is full and when there are vacancies.
- Assume that each transport layer uses one single memory location to hold a packet.
- When this single slot in the sending transport layer is empty, the sending transport layer sends a note to the application layer to send its next chunk;
- When this single slot in the receiving transport layer is empty, it sends an acknowledgment to the sending transport layer to send its next packet.

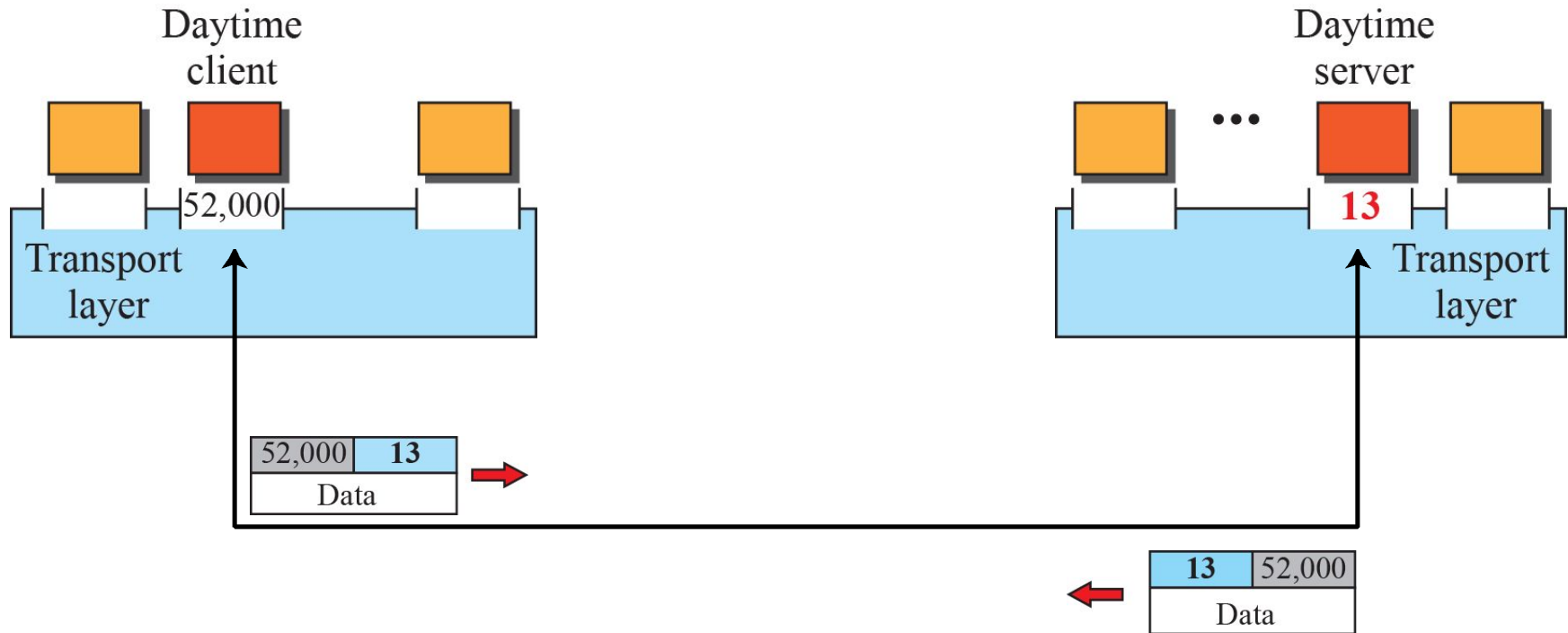
Transport-Layer Services

1. Process-to-Process Communication



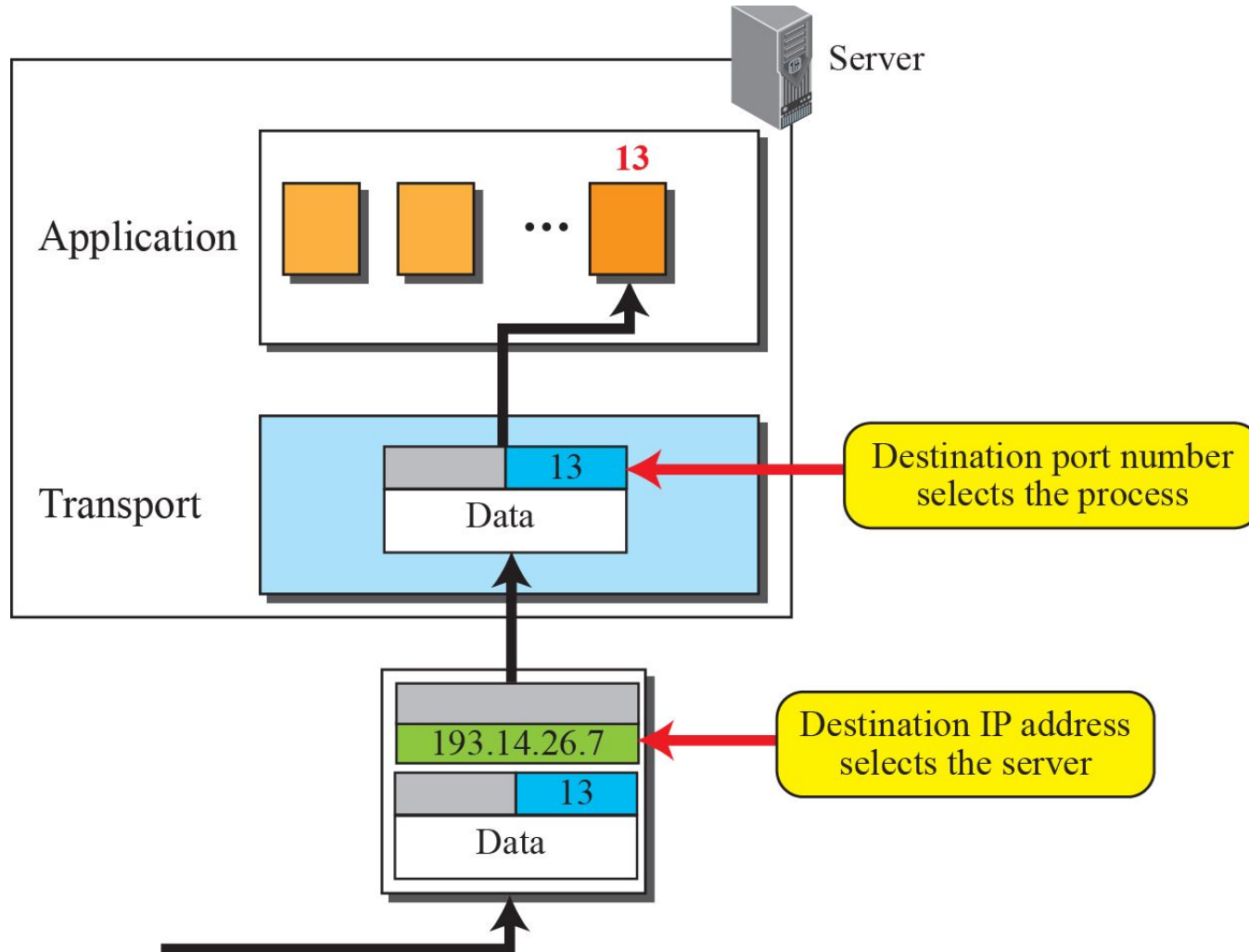
Transport-Layer Services

1. Process-to-Process Communication



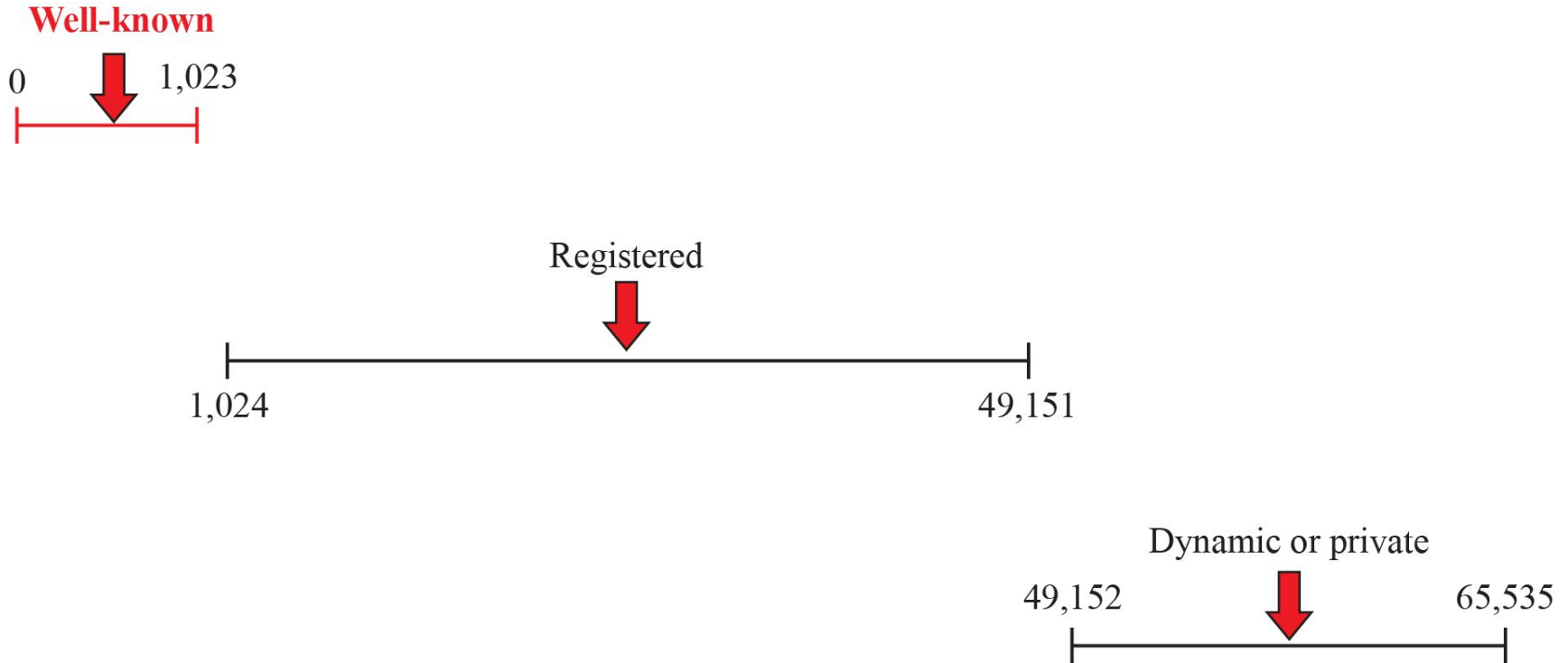
Transport-Layer Services

1. Process-to-Process Communication



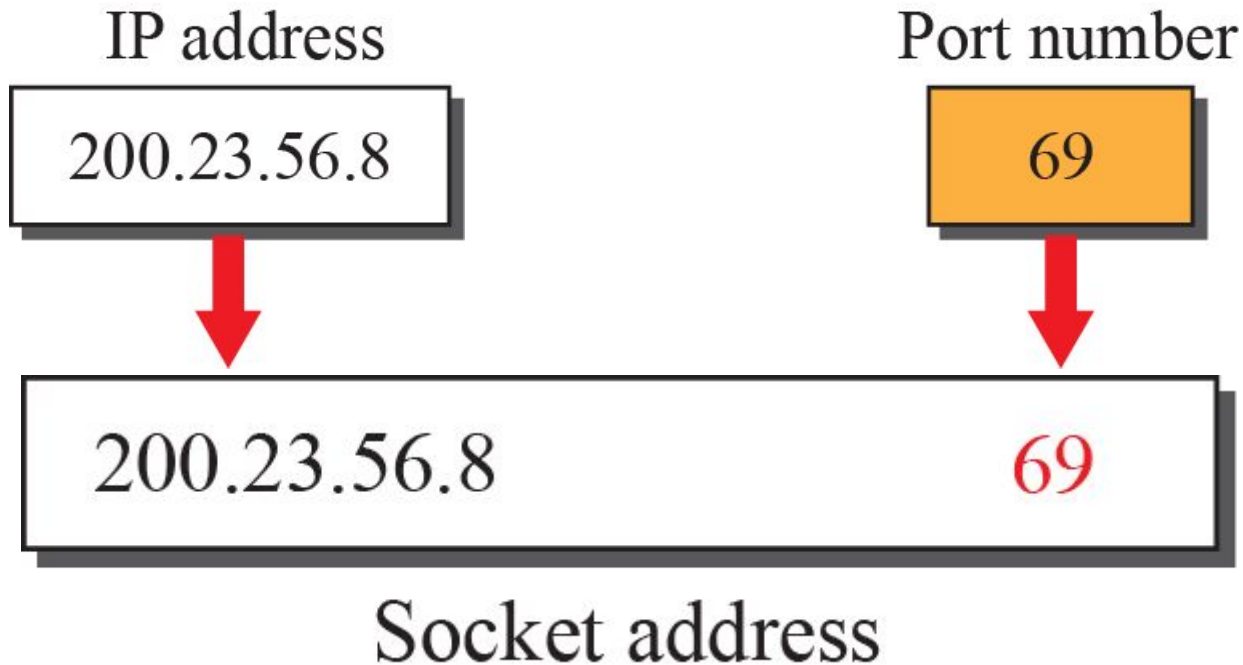
Transport-Layer Services

□ *Addressing: Port Numbers*



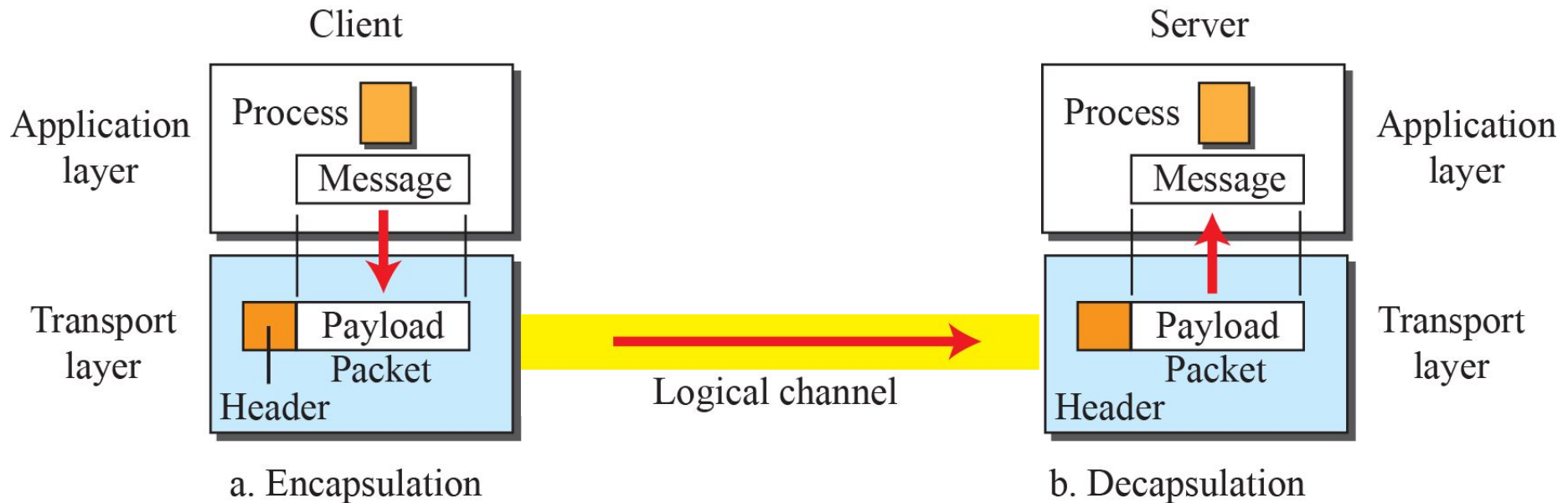
Transport-Layer Services

□ *Addressing: Port Numbers*



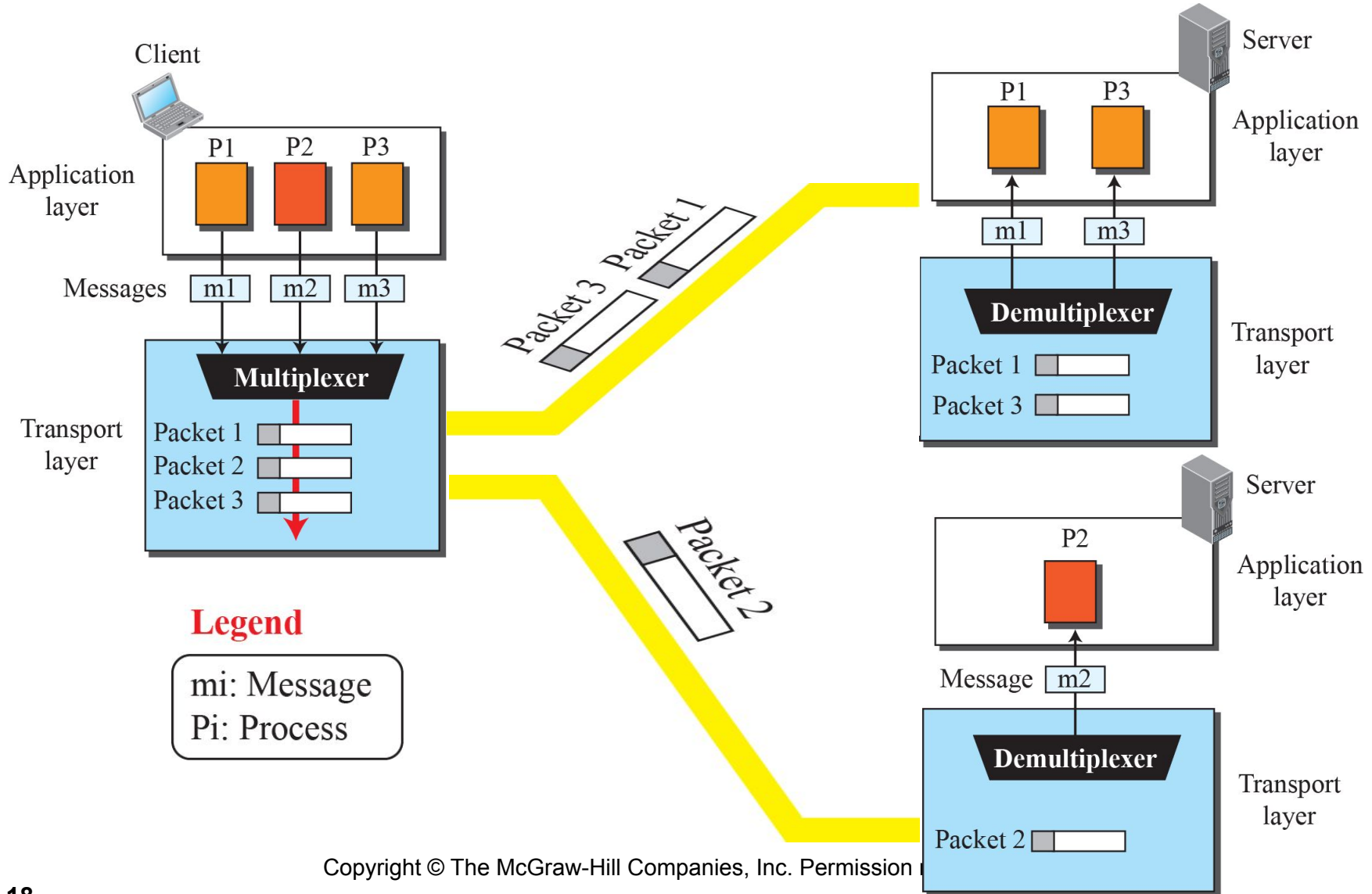
Transport-Layer Services

□ *Encapsulation and Decapsulation*



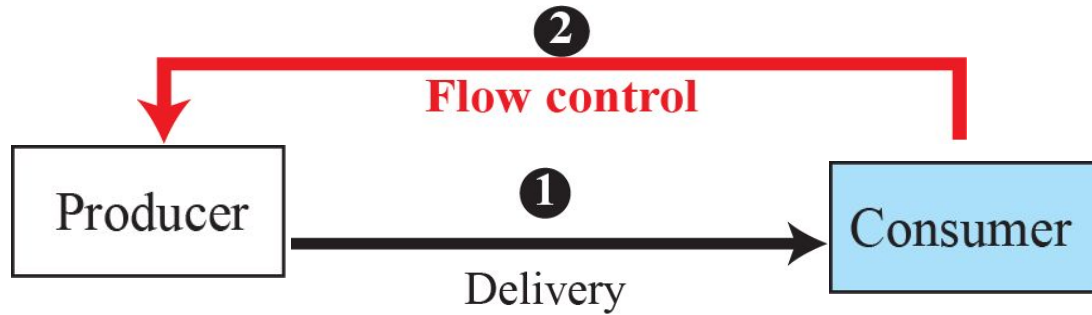
Transport-Layer Services

□ *Multiplexing and Demultiplexing*

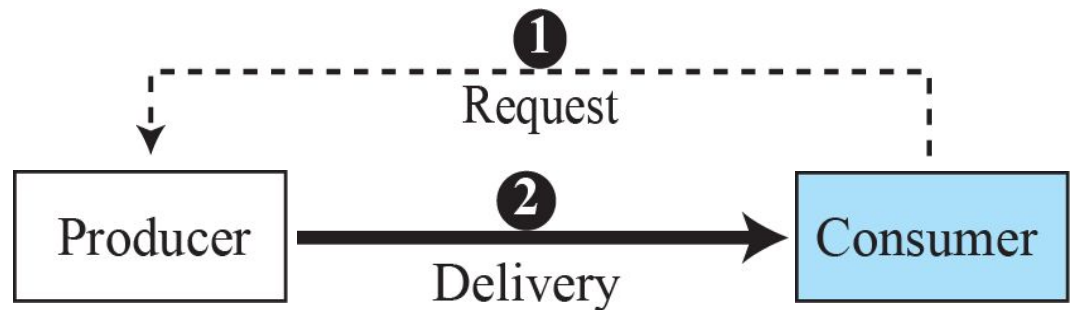


Transport-Layer Services

□ *Flow Control*



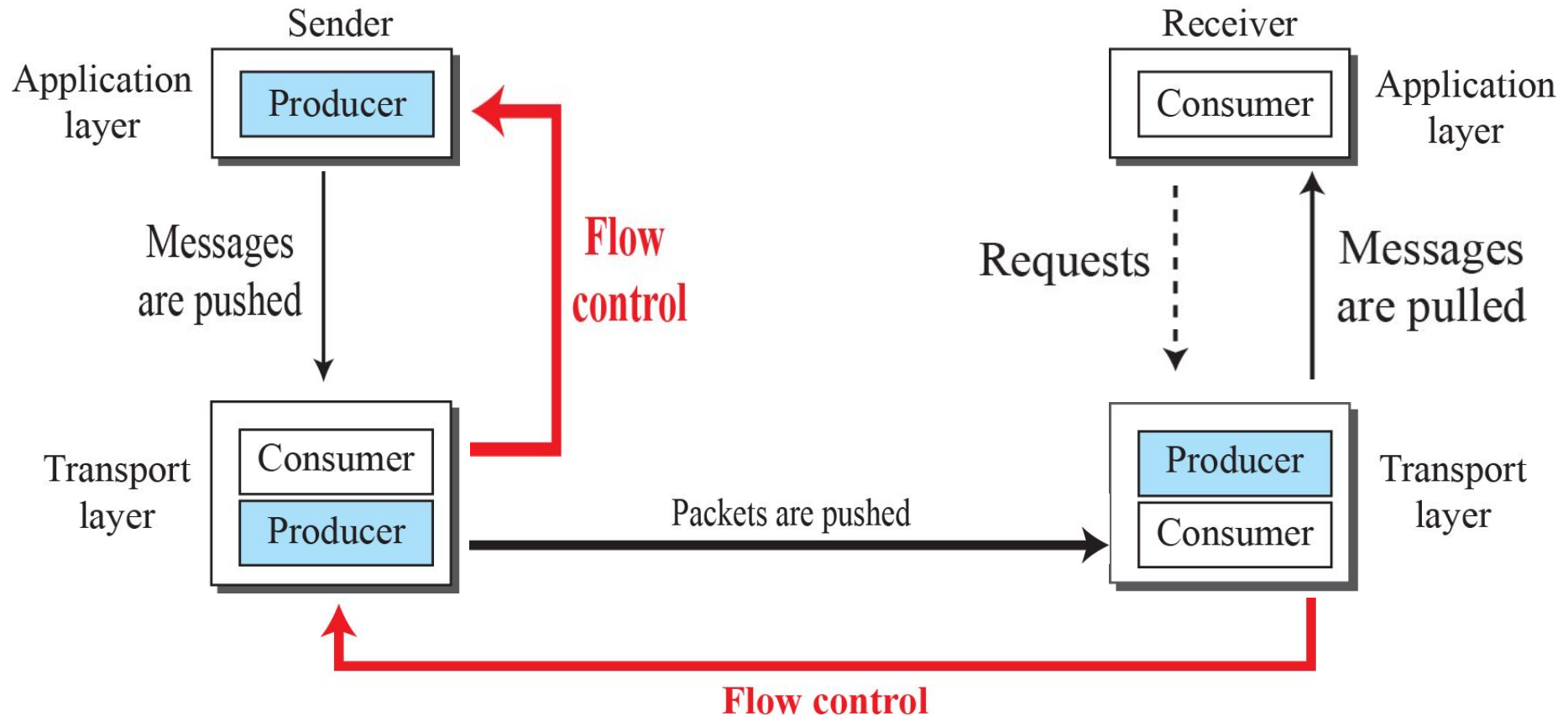
a. Pushing



b. Pulling

Transport-Layer Services

□ *Flow Control*

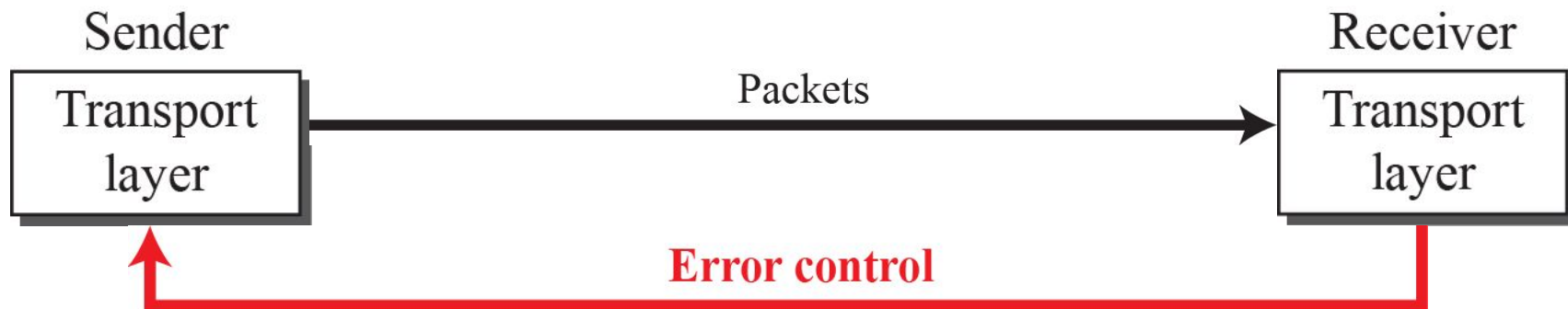


Transport-Layer Services

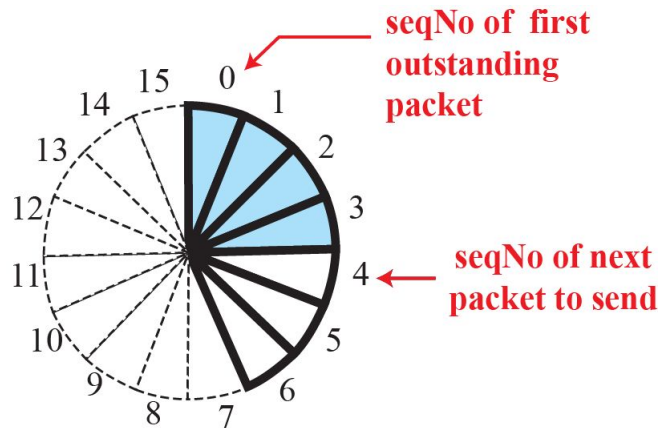
- Consumers communicate with the producers on two occasions: when the buffer is full and when there are vacancies.
- Assume that each transport layer uses one single memory location to hold a packet.
- When this single slot in the sending transport layer is empty, the sending transport layer sends a note to the application layer to send its next chunk;
- When this single slot in the receiving transport layer is empty, it sends an acknowledgment to the sending transport layer to send its next packet.

Transport-Layer Services

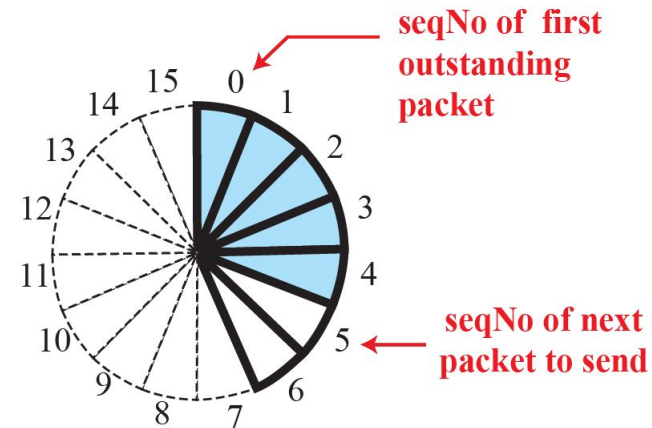
□ *Error Control*



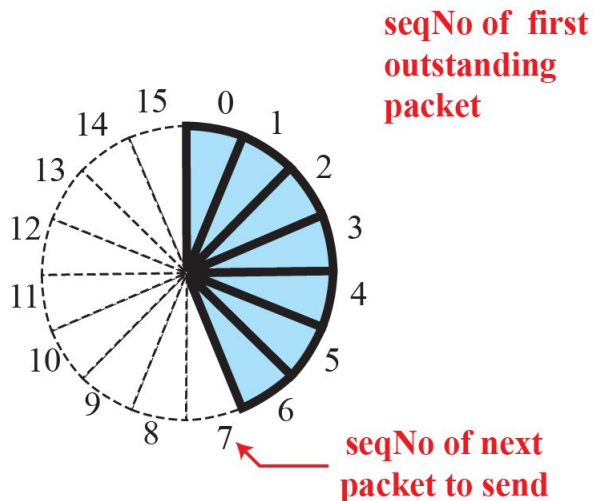
Transport-Layer Services



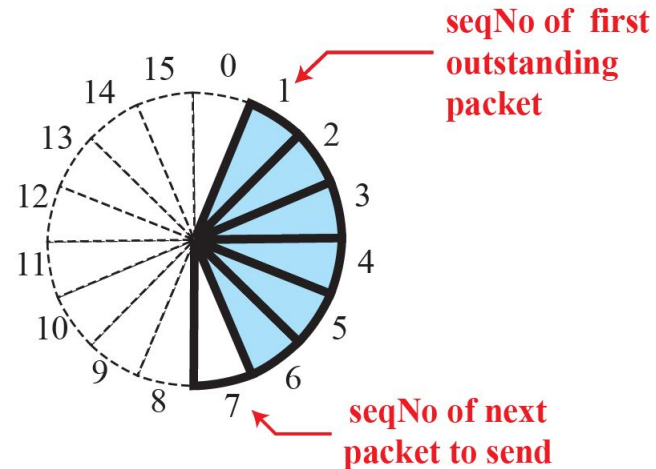
a. Four packets have been sent.



b. Five packets have been sent.



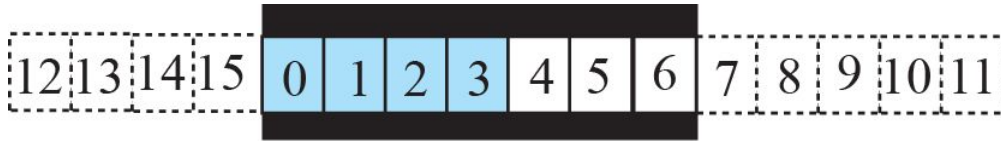
c. Seven packets have been sent;
window is full.



d. Packet 0 has been acknowledged;
window slides.

Transport-Layer Services

❑ *Error Control*



a. Four packets have been sent.



b. Five packets have been sent.



c. Seven packets have been sent;
window is full.



d. Packet 0 has been acknowledged;
window slides.

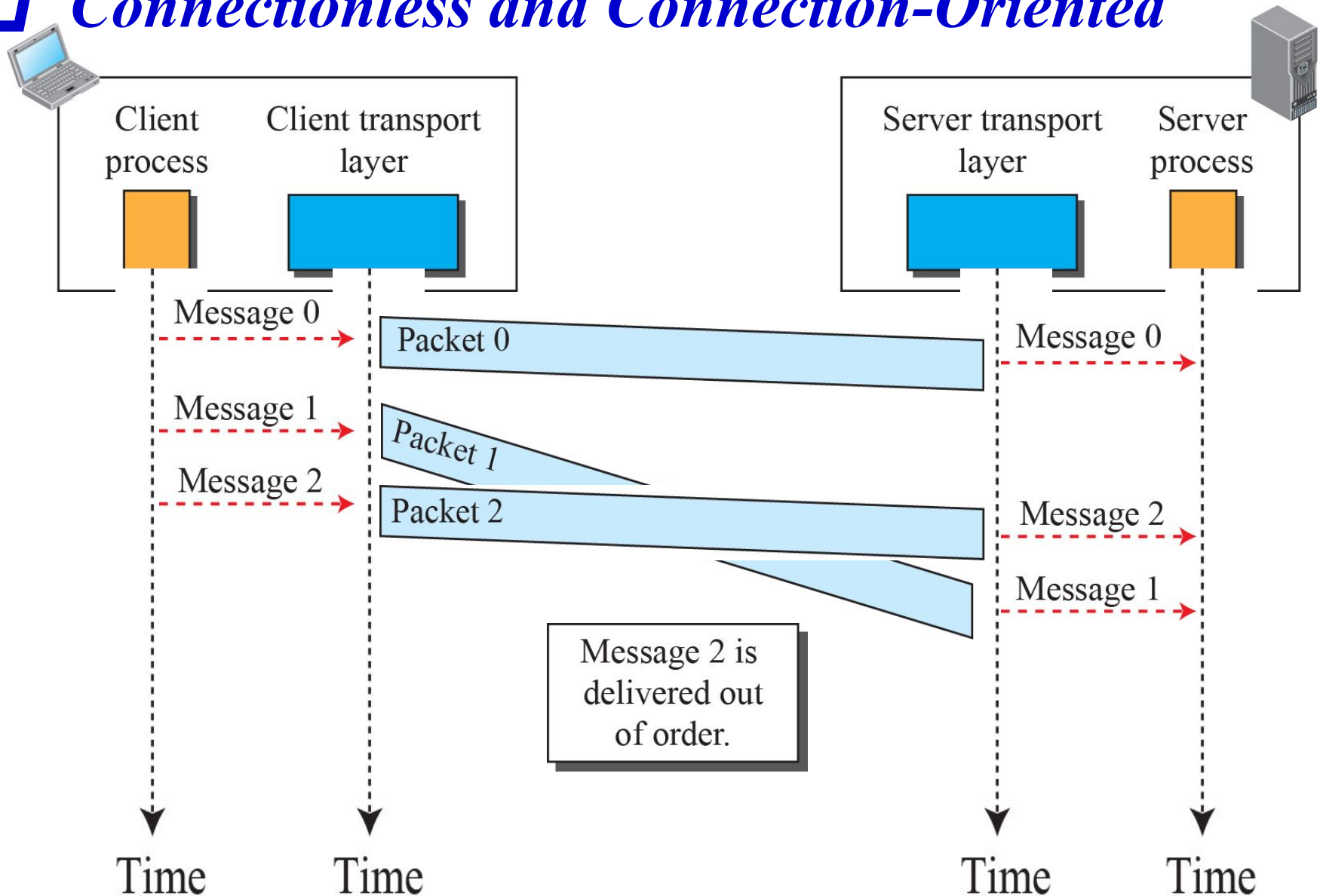
Transport-Layer Services

❑ *Congestion Control*

❑ *Connectionless and Connection-Oriented*

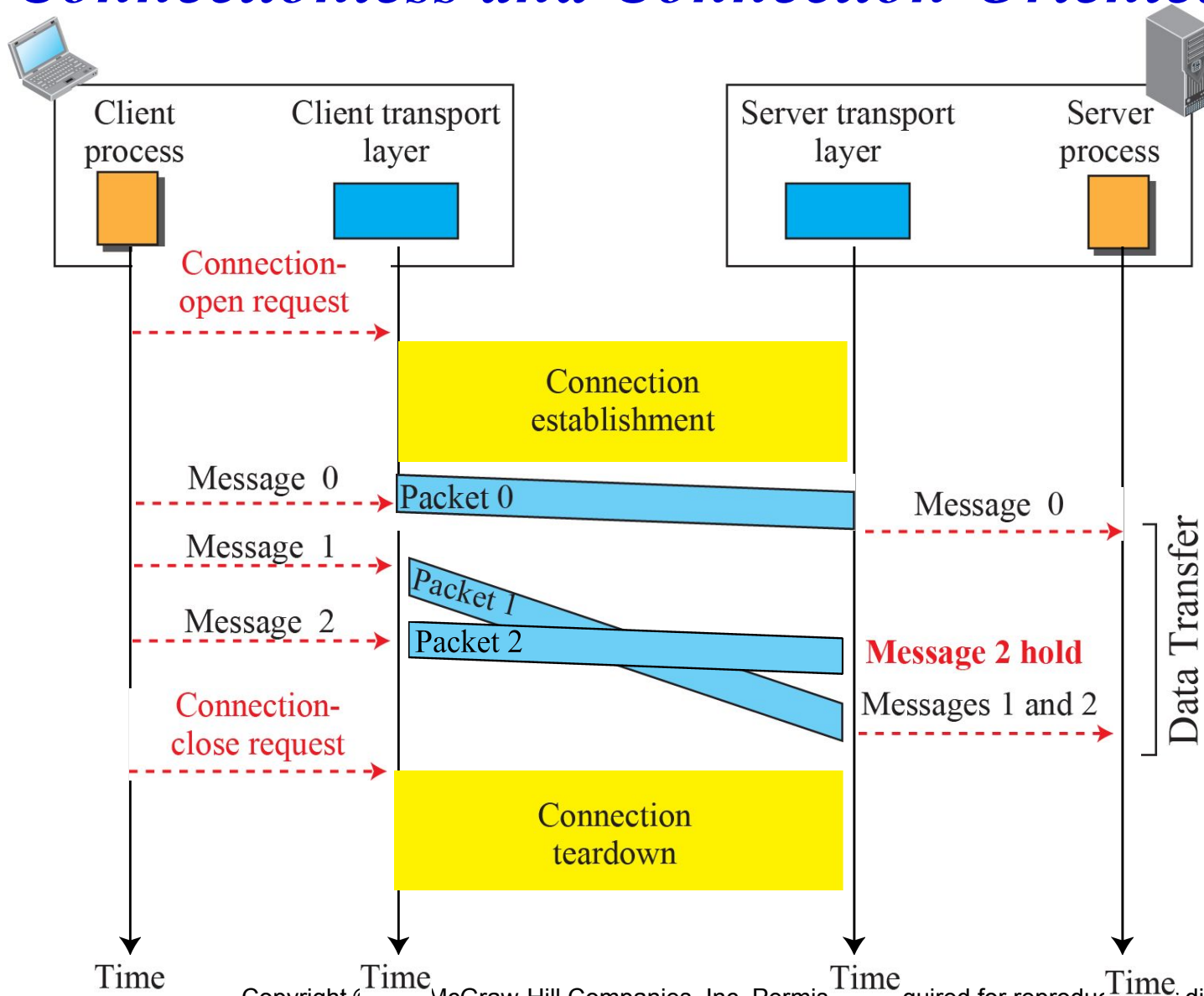
Transport-Layer Services

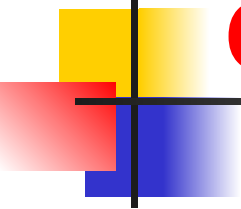
□ *Connectionless and Connection-Oriented*



Transport-Layer Services

□ *Connectionless and Connection-Oriented*





Chapter 3: Outline

3.4. Flow Control in Transport Layer

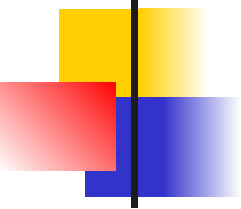
-Stop-and-wait

-Go-back-N

-Selective-Repeat

TRANSPORT-LAYER PROTOCOLS

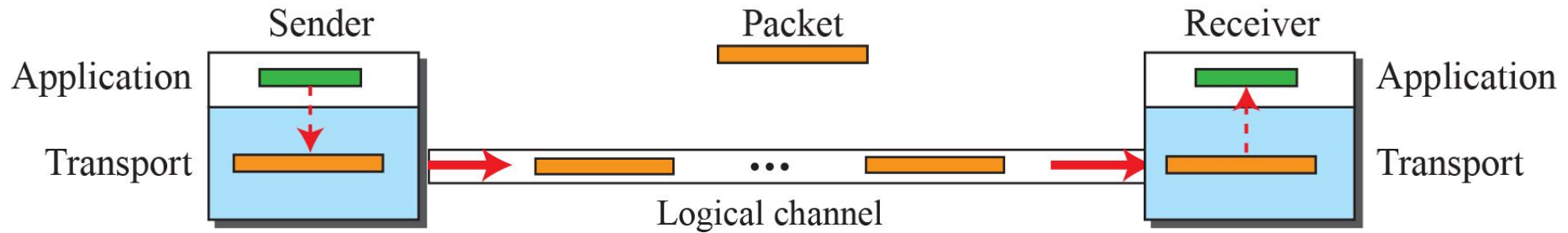
- We can create a transport-layer protocol by combining a set of services described in the previous sections.
- To better understand the behavior of these protocols, we start with the simplest one and gradually add more complexity.
- Four types of protocol:
 - Simple*
 - Stop-and-wait*
 - Go-back-N*
 - Selective-Repeat*



Simple Protocol

- Our first protocol is a simple connectionless protocol with neither flow nor error control.
- We assume that the receiver can immediately handle any packet it receives.
- In other words, the receiver can never be overwhelmed with incoming packets.
- Figure 3.17 shows the layout for this protocol.

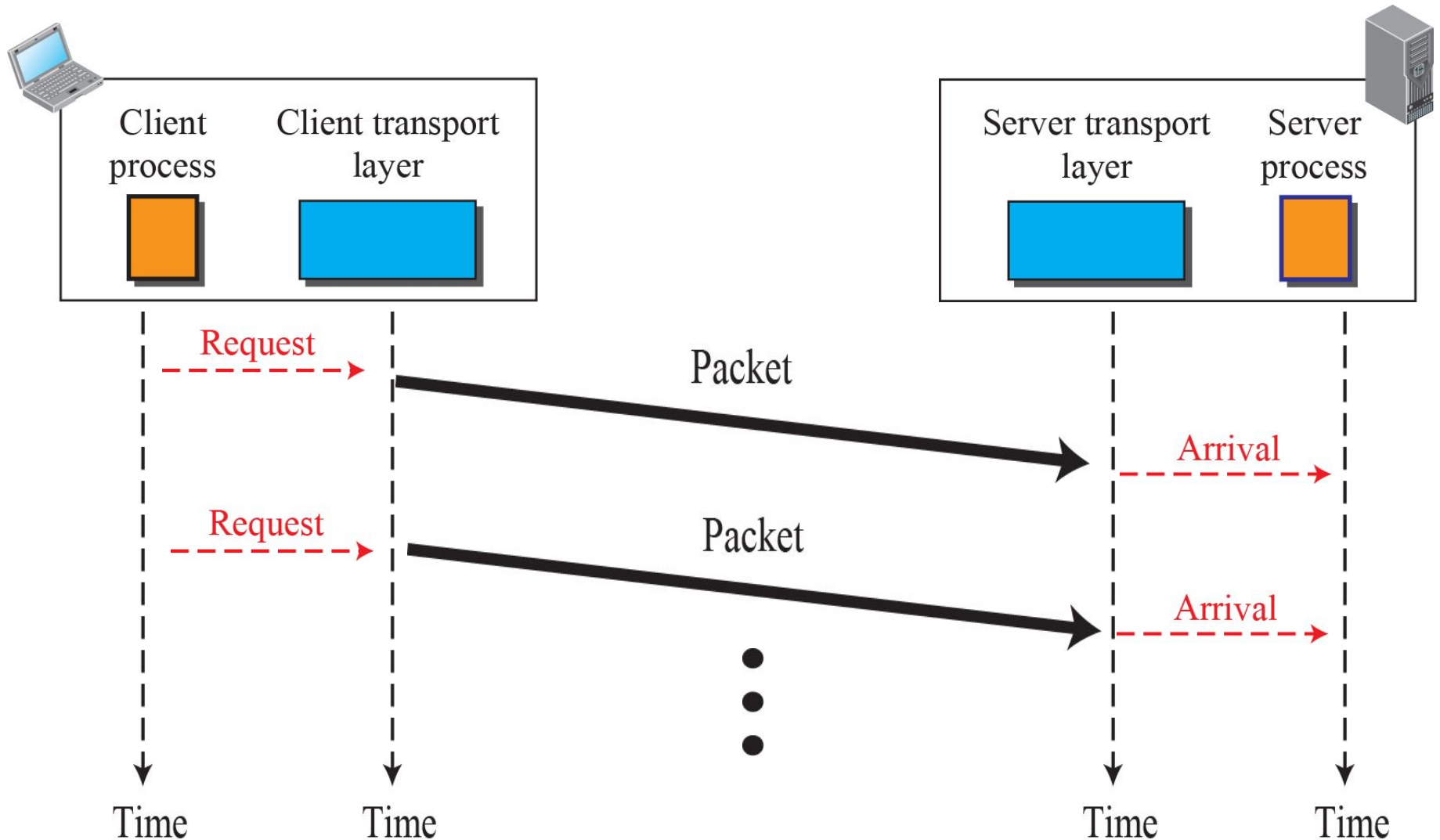
Simple Protocol

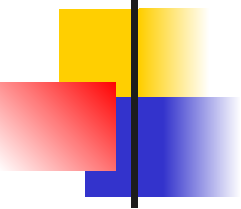


Simple Protocol

- Figure 3.19 shows an example of communication using this protocol.
- It is very simple.
- The sender sends packets one after another without even thinking about the receiver.

Simple Protocol

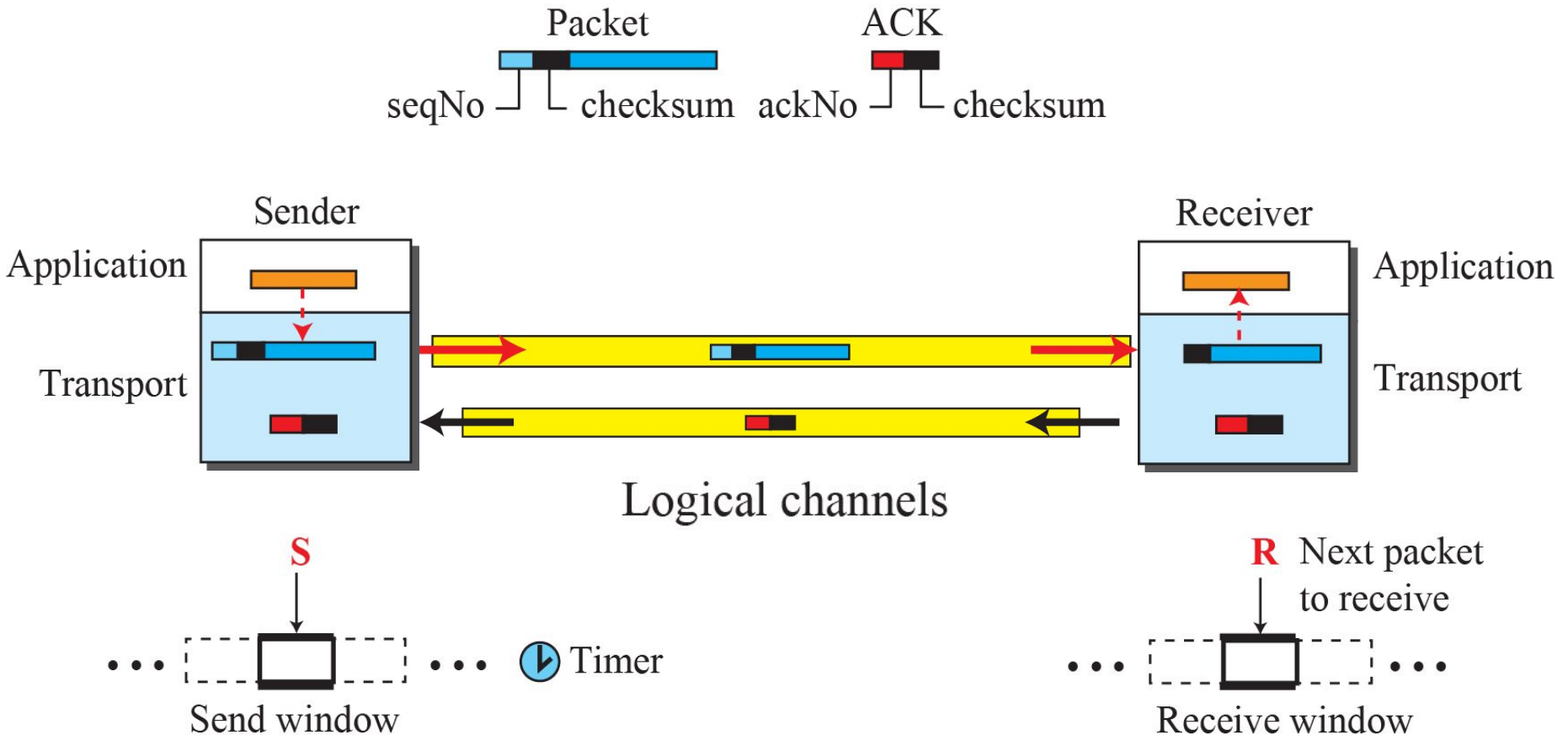




Stop-and-Wait Protocol

- Our second protocol is a connection-oriented protocol called the Stop-and-Wait protocol, which uses both flow and error control.
- Both the sender and the receiver use a sliding window of size 1.
- The sender sends one packet at a time and waits for an acknowledgment before sending the next one.
- To detect corrupted packets, we need to add a checksum to each data packet.
- When a packet arrives at the receiver site,

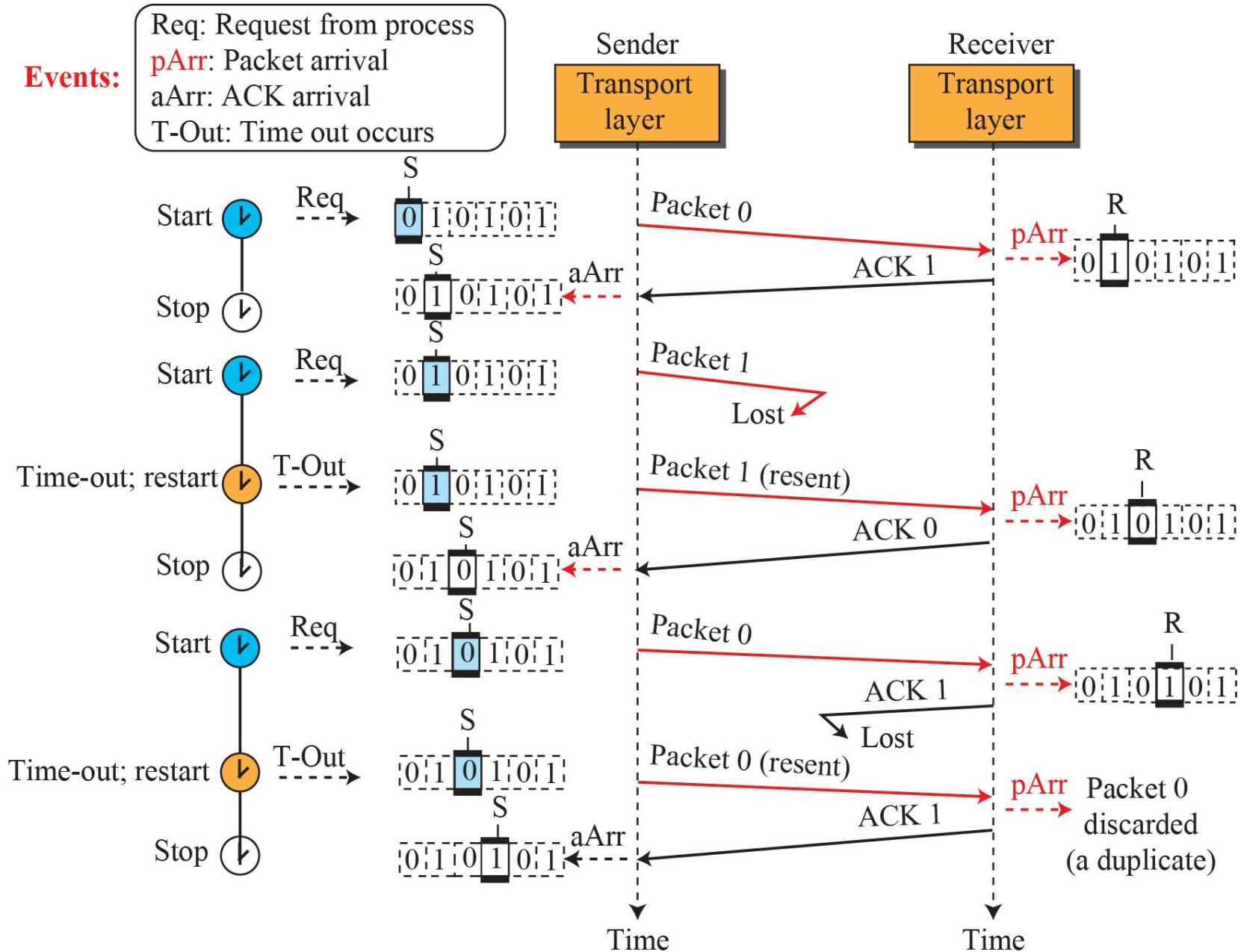
Stop-and-Wait Protocol



Stop-and-Wait Protocol

- Figure 3.22 shows an example of the Stop-and-Wait protocol.
- Packet 0 is sent and acknowledged.
- Packet 1 is lost and resent after the time-out.
- The resent packet 1 is acknowledged and the timer stops.
- Packet 0 is sent and acknowledged, but the acknowledgment is lost.
- The sender has no idea if the packet or the acknowledgment is lost, so after the time-out, it resends packet 0, which is acknowledged.

Stop-and-Wait Protocol



Stop-and-Wait Protocol

Link Utilization:

$$\begin{aligned} U &= \text{Time taken to transmit the packet to link} / \text{Total time} \\ &= D_{\text{trans}} / (D_{\text{trans}} + 2 * D_{\text{prop}} + D_{\text{proc}} + D_{\text{queue}}) \\ &= D_{\text{trans}} / (D_{\text{trans}} + 2 * D_{\text{prop}}) \end{aligned}$$

$$U = 1 / (1 + 2a)$$

$$a = D_{\text{prop}} / D_{\text{trans}}$$

If P = probability of frame error then

$$U = (1 - P) / (1 + 2a)$$

Stop-and-Wait Protocol

Assume that, in a Stop-and-Wait system, the bandwidth of the line is 1 Mbps, and 1 bit takes 20 milliseconds to make a round trip. What is the bandwidth-delay product? If the system data packets are 1,000 bits in length, what is the utilization percentage of the link?

Solution

The bandwidth-delay product is $(1 \times 10^6) \times (20 \times 10^{-3}) = 20,000$ bits. The system can send 20,000 bits during the time it takes for the data to go from the sender to the receiver and the acknowledgment to come back. However, the system sends only 1,000 bits. The link utilization is only $1,000/20,000$, or 5 percent.

Stop-and-Wait Protocol

What is the utilization percentage of the link in Example 3.5 if we have a protocol that can send up to 15 packets before stopping and worrying about the acknowledgments?

Solution

The bandwidth-delay product is still 20,000 bits. The system can send up to 15 packets or 15,000 bits during a round trip. This means the utilization is $15,000/20,000$, or 75 percent. Of course, if there are damaged packets, the utilization percentage is much less because packets have to be resent.

Stop-and-Wait Protocol

Q: The distance from earth to planet is $9 \times 10^9 \text{m}$. What is the channel utilization if stop-and-wait protocol is used for frame transmission on a 64Mbps point to point link? Assume frame size 32KB and speed of light is $3 \times 10^8 \text{m/s}$.

Ans: $1/(1+(15 \times 10^3))$

Q: Consider the use of 10kilobit size frame on a 10Mbps satellite channel with 270ms delay. What is the link utilization for SAW ARQ technique assuming $P=10^{-3}$ where P is the probability of single bit error.

Stop-and-Wait Protocol

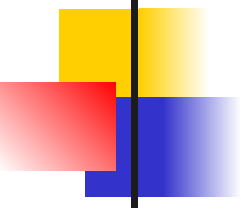
Q: Consider a computer network which sends the packet with size of 1000bits of data using SAW ARQ protocol. If the distance between source and destination is 400km and propagation speed is $2 \times 10^8 \text{m/s}$. Find the time required to send 10000bits.

Q: Assume a 100Mbps link of 10000m in length with 5ns per meter propagation delay. Assume a constant length of 400byte data frame, 64 byte ACK frame, 10microsec of processing delay for data frame, 5microsec processing delay for ACK. Calculate the link utilization assuming SAW protocol.

Stop-and-Wait Protocol

Q-1: Assume that, in a Stop-and-Wait system, the bandwidth of the line is 1Mbps, and 1 bit takes 20 milliseconds to make a round trip. If the system data packets are 1,000 bits in length, what is the utilization percentage of the link?

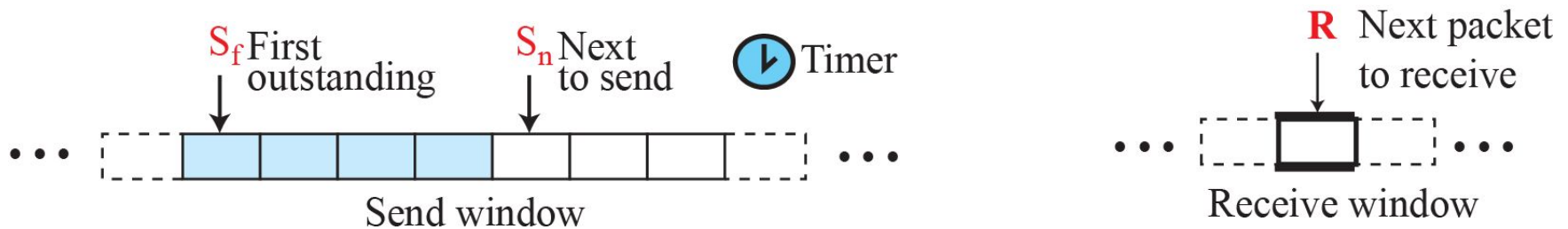
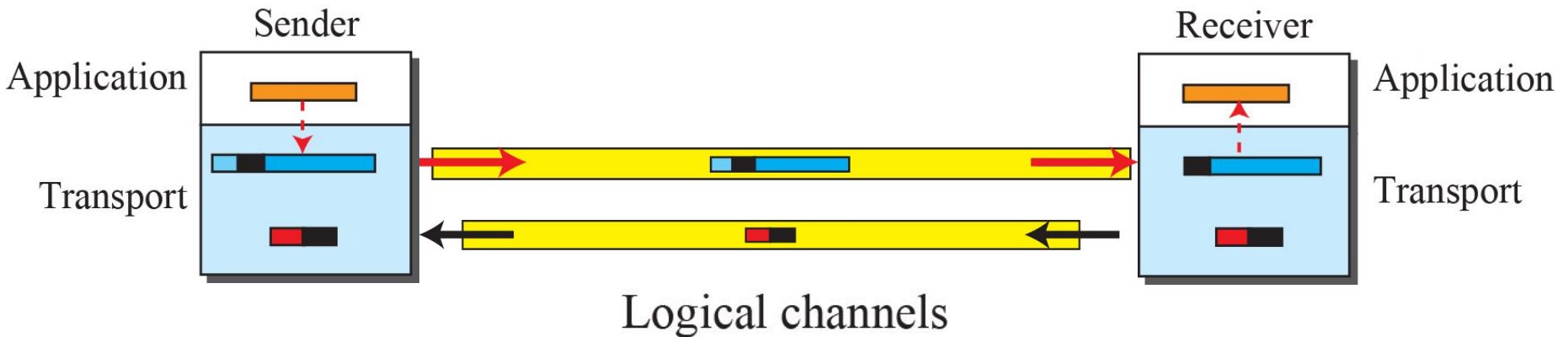
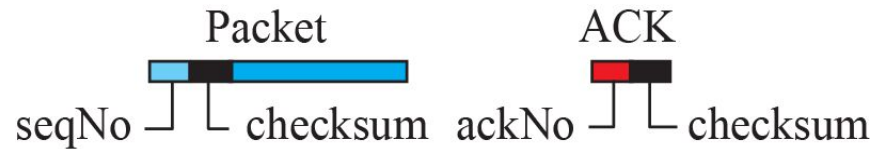
Q-2:



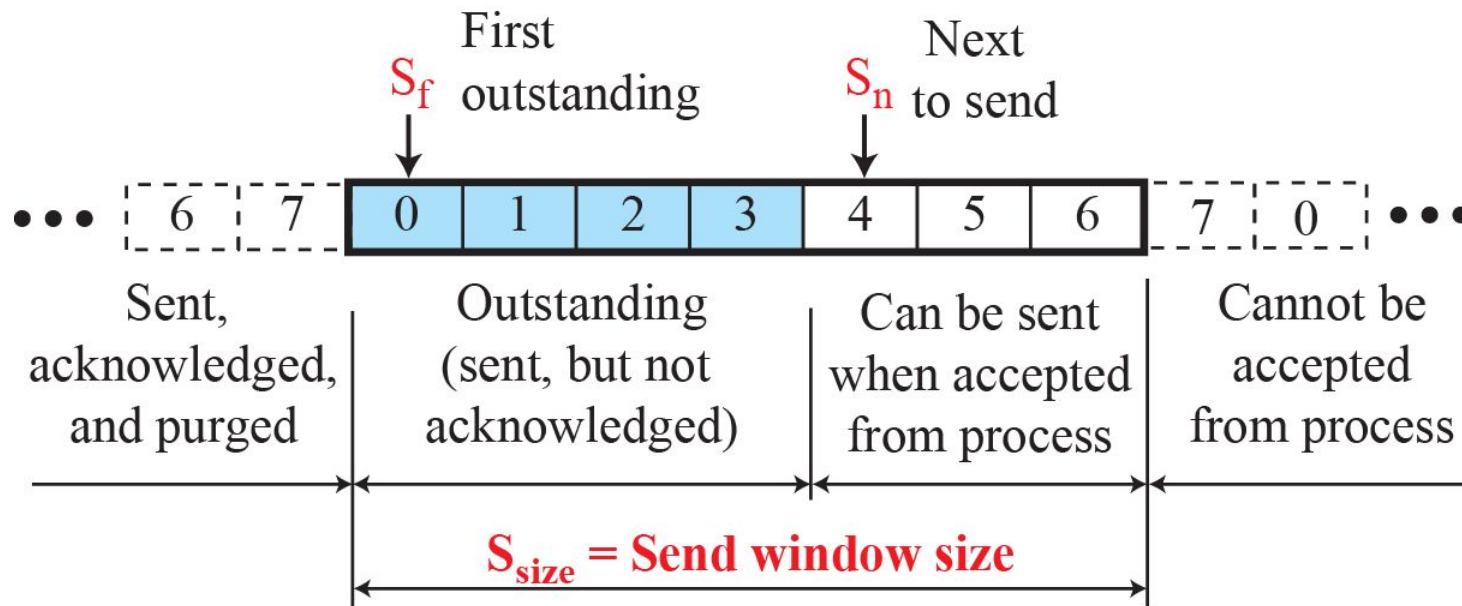
Go-Back-N Protocol

- To improve the efficiency of transmission multiple packets must be in transition while the sender is waiting for acknowledgment.
- In this section, we discuss one protocol that can achieve this goal; in the next section, we discuss a second.
- The first is called Go-Back-N (GBN) (the rationale for the name will become clear later).

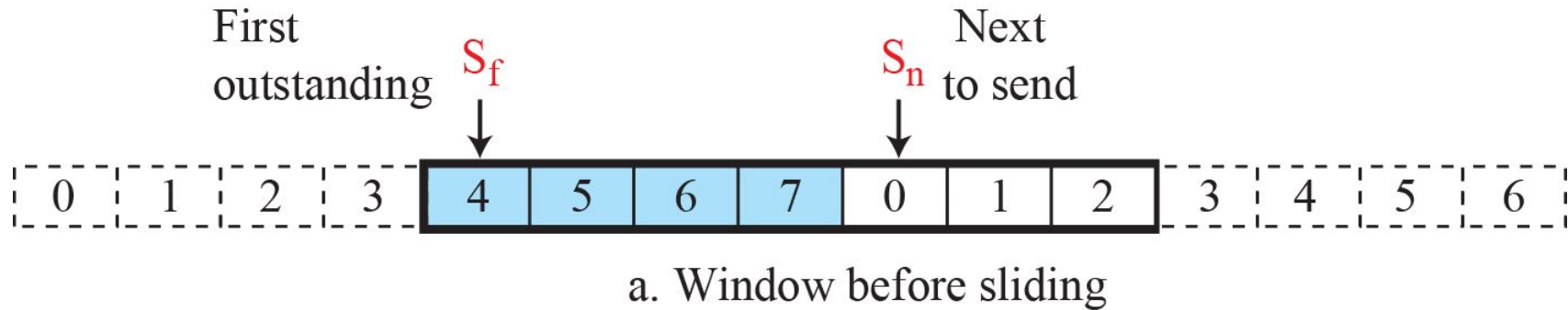
Go-Back-N Protocol



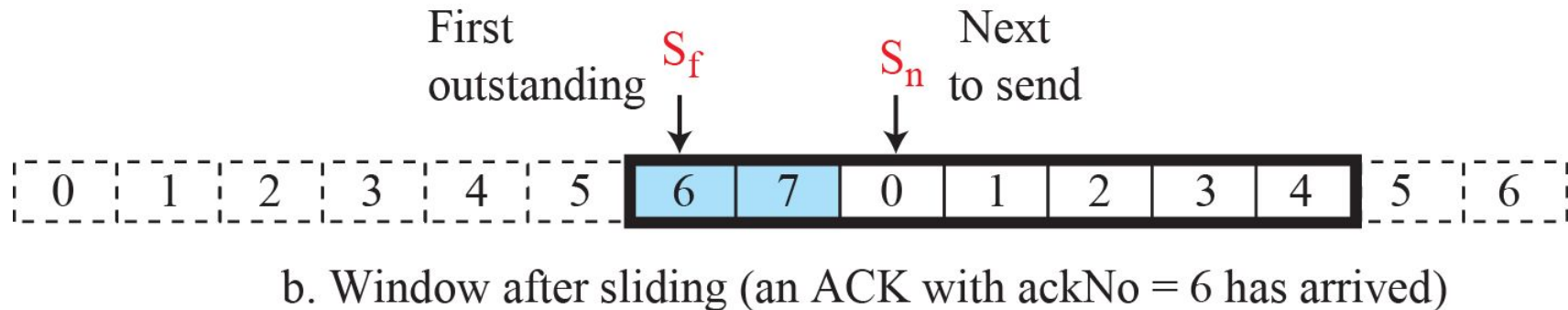
Go-Back-N Protocol



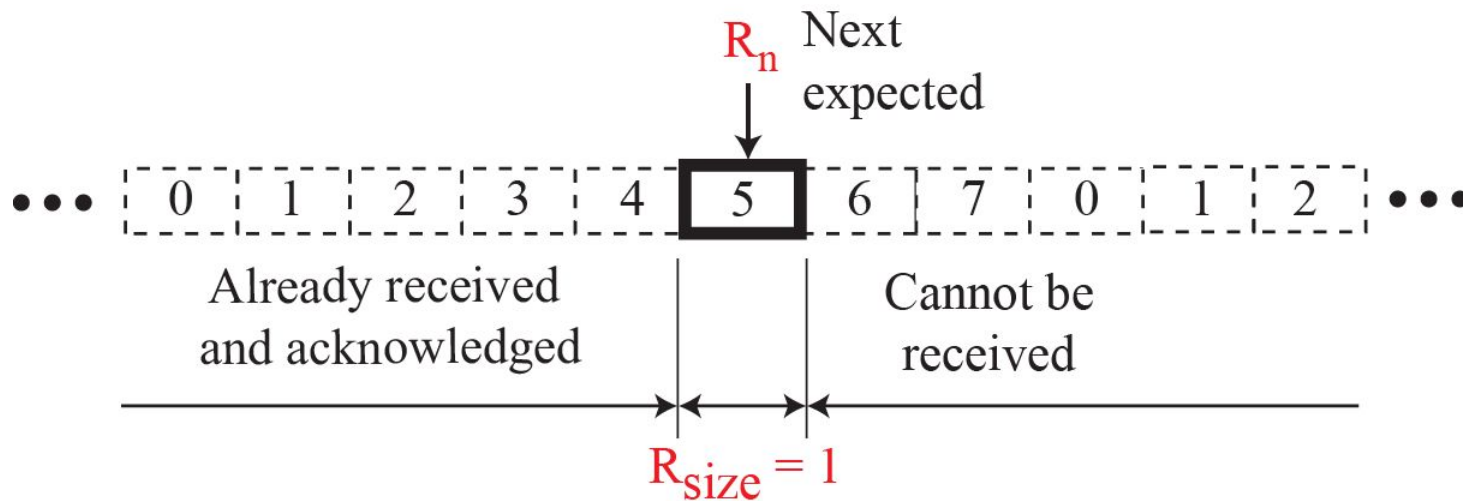
Go-Back-N Protocol



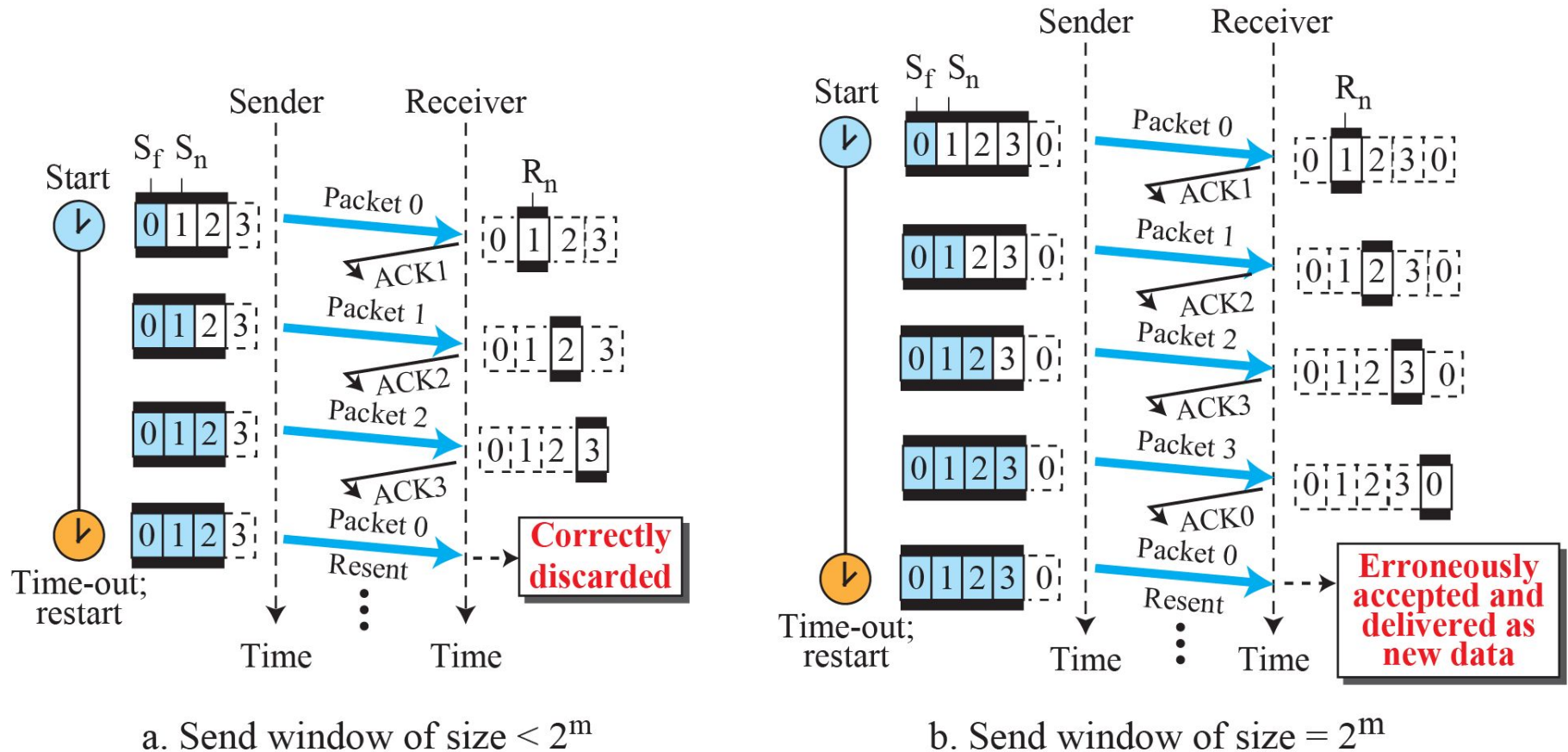
 *Sliding direction*



Go-Back-N Protocol



Go-Back-N Protocol

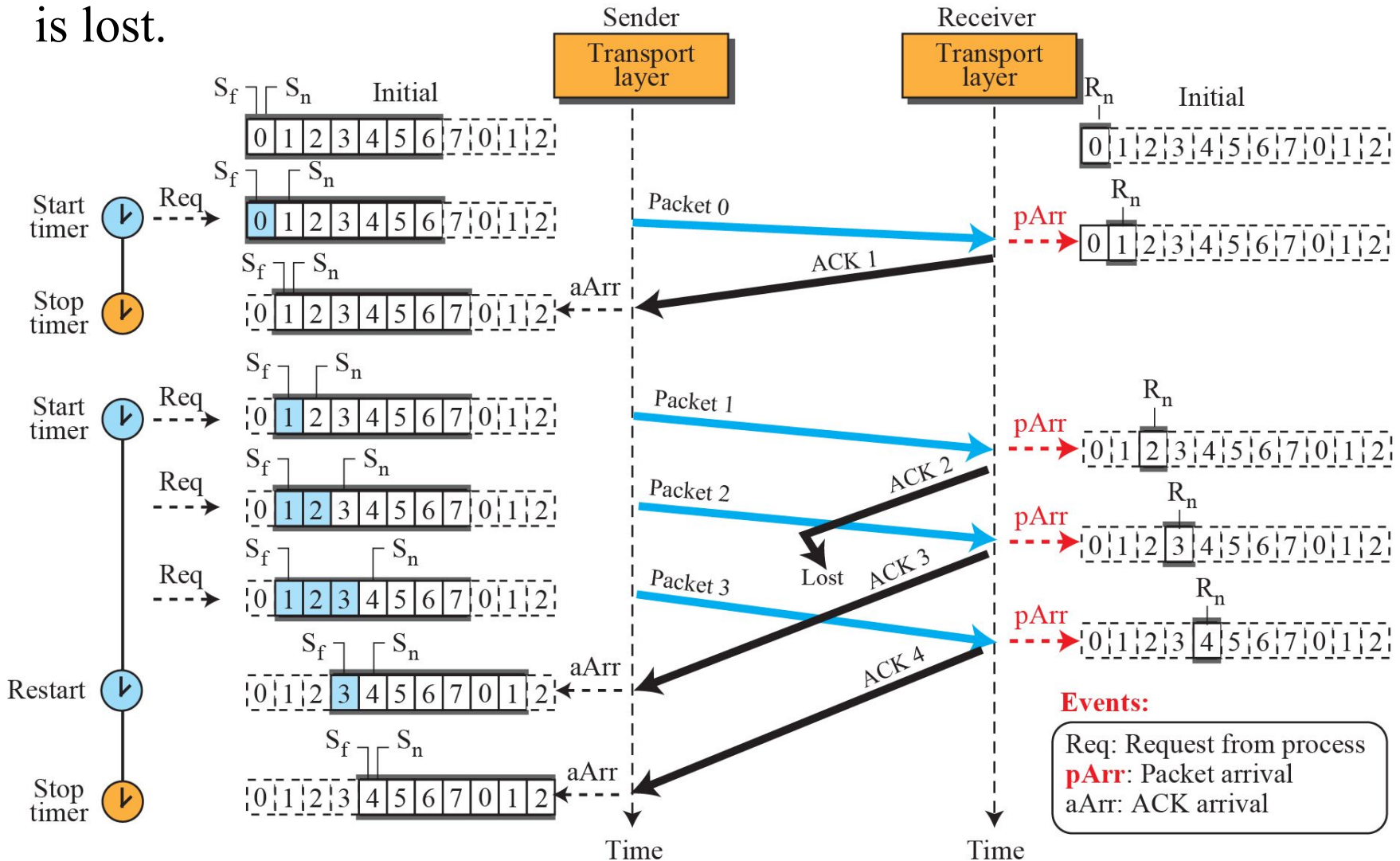


Go-Back-N Protocol

- Figure 3.29 shows an example of Go-Back-*N*.
- This is an example of a case where the forward channel is reliable, but the reverse is not.
- No data packets are lost, but some ACKs are delayed and one is lost.
- The example also shows how cumulative acknowledgments can help if acknowledgments are delayed or lost.

Go-Back-N

No data packets are lost, but some ACKs are delayed and one is lost.

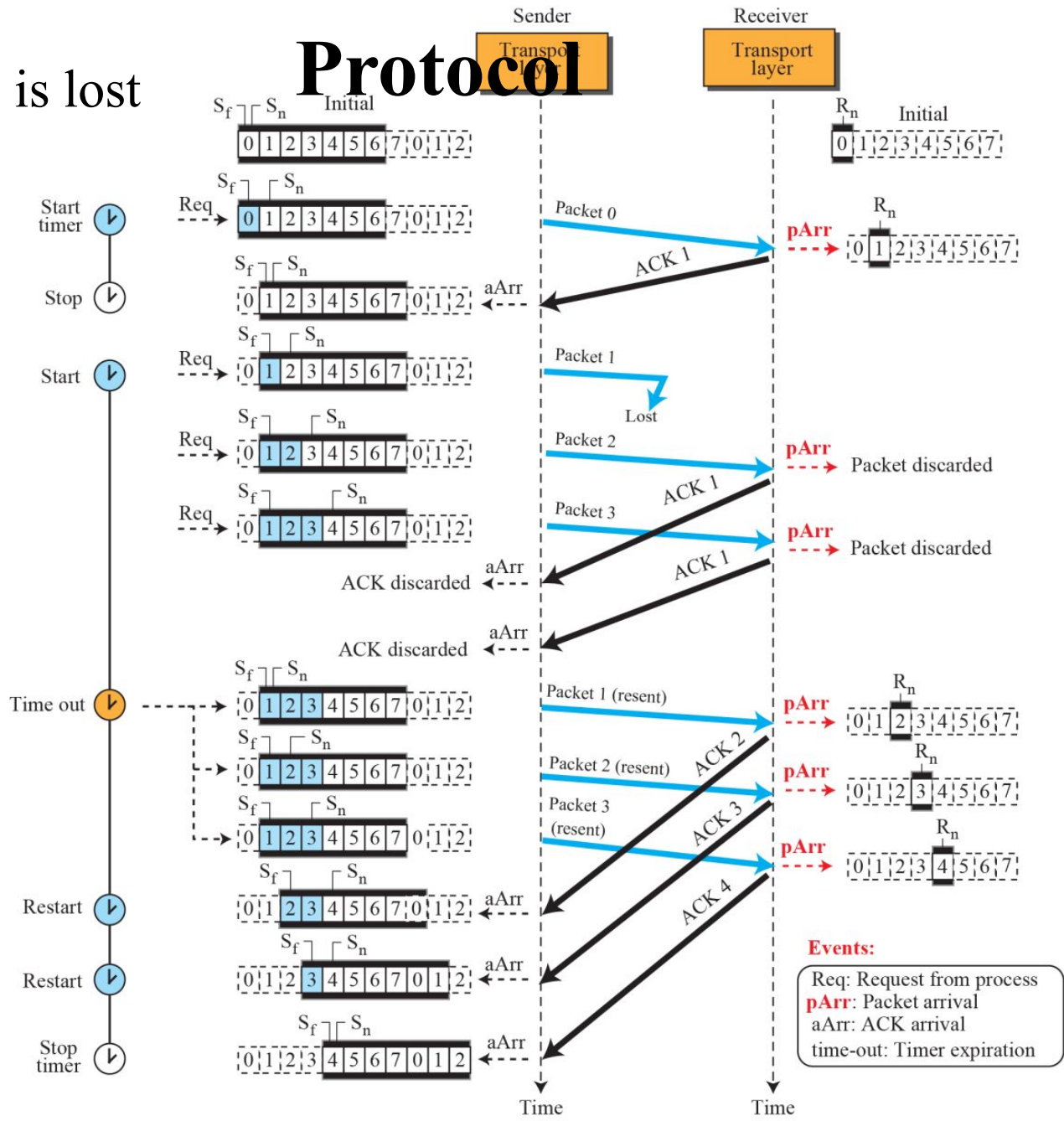


Go-Back-N

- Figure 3.30 shows ~~What happens~~ **Protocol** when a packet is lost. Packets 0, 1, 2, and 3 are sent.
- However, packet 1 is lost.
- The receiver receives packets 2 and 3, but they are discarded because they are received out of order (packet 1 is expected).
- When the receiver receives packets 2 and 3, it sends ACK1 to show that it expects to receive packet 1.
- However, these ACKs are not useful for the sender because the ackNo is equal to S_f , not greater than S_f .
- So the sender discards them.
- When the time-out occurs, the sender resends packets 1, 2, and 3, which are acknowledged.

Go-Back-N Protocol

Data packets is lost



Go-Back-N Protocol

Link Utilization:

$$\begin{aligned} U &= \text{Time taken to transmit the packet to link} / \text{Total time} \\ &= D_{\text{trans}} / (D_{\text{trans}} + 2 * D_{\text{prop}} + D_{\text{proc}} + D_{\text{queue}}) \\ &= D_{\text{trans}} / (D_{\text{trans}} + 2 * D_{\text{prop}}) \end{aligned}$$

$$U = N / (1 + 2a)$$

$$a = D_{\text{prop}} / D_{\text{trans}}$$

N = Number of packets

If P = probability of frame error then

$$U = N(1-P) / \{(1+2a)(1-P+NP)\}$$

Go-Back-N Protocol

Q-1: Consider the use of 10kilobit size frame on a 10Mbps satellite channel with 270ms delay. What is the link utilization for GBN protocol assuming $P=10^{-3}$ where P is the probability of single bit error and window size is 7.

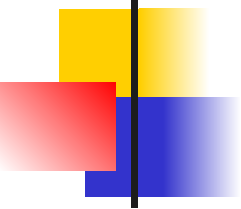
Q-2: Consider designing a GBN protocol with bandwidth of 100Kbps and one way delay of 4sec. Assume each packet carries 1KB Data. What is the minimum number of bits required for sequence number.

Answer:7

Go-Back-N Protocol

Q: In a network using the Go-Back-N protocol with $m = 3$, the values of variables are $Sf = 62$, $Sn = 66$, and $Rn = 64$. Assume that the network does not duplicate or reorder the packets. Draw the flow diagram of the above scenario and find out the below answers with the help of the diagram.

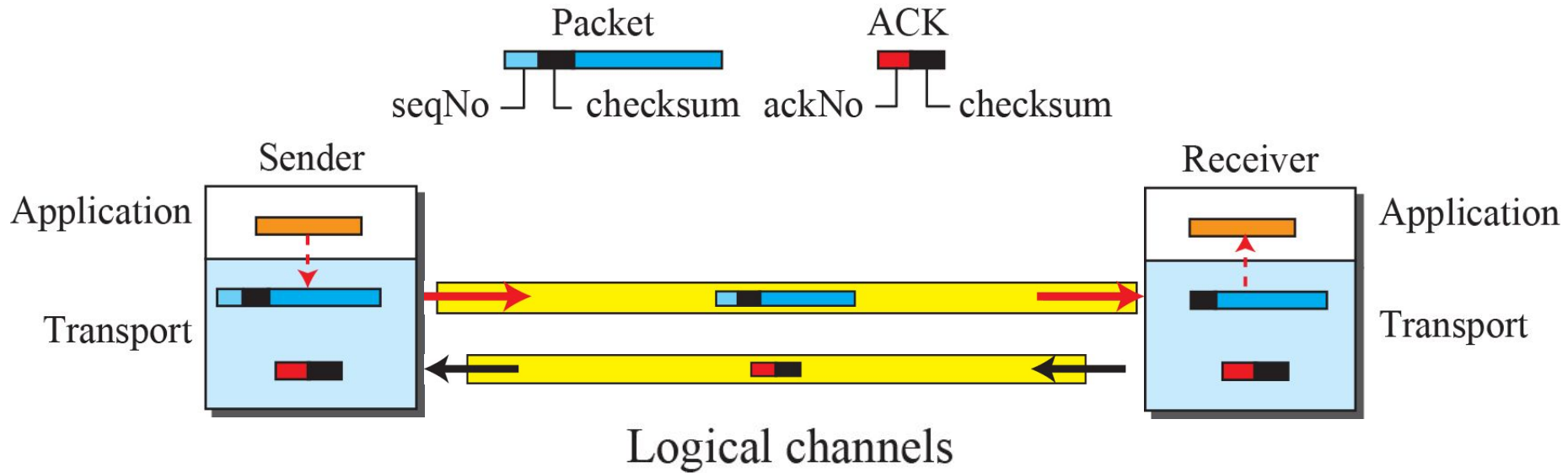
- a. What are the sequence numbers of data packets in transit?
- b. What are the acknowledgment numbers of ACK packets in transit?



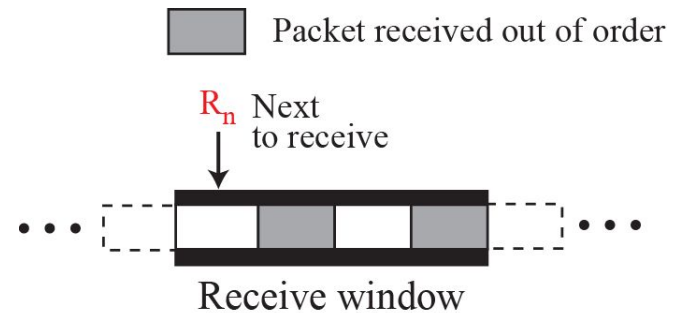
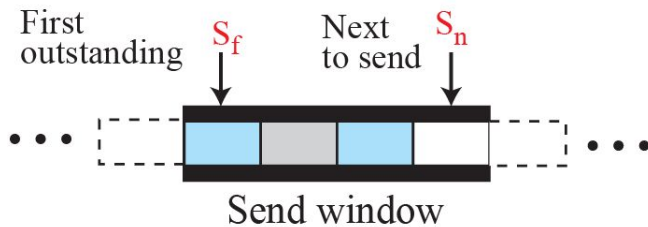
Selective-Repeat Protocol

- The Go-Back-N protocol simplifies the process at the receiver.
- The receiver keeps track of only one variable, and there is no need to buffer out-of-order packets; they are simply discarded.
- Another protocol, called the Selective-Repeat (SR) protocol, has been devised, which, as the name implies, resends only selective packets, those that are actually lost.
- The outline of this protocol is shown in Figure 3.31.

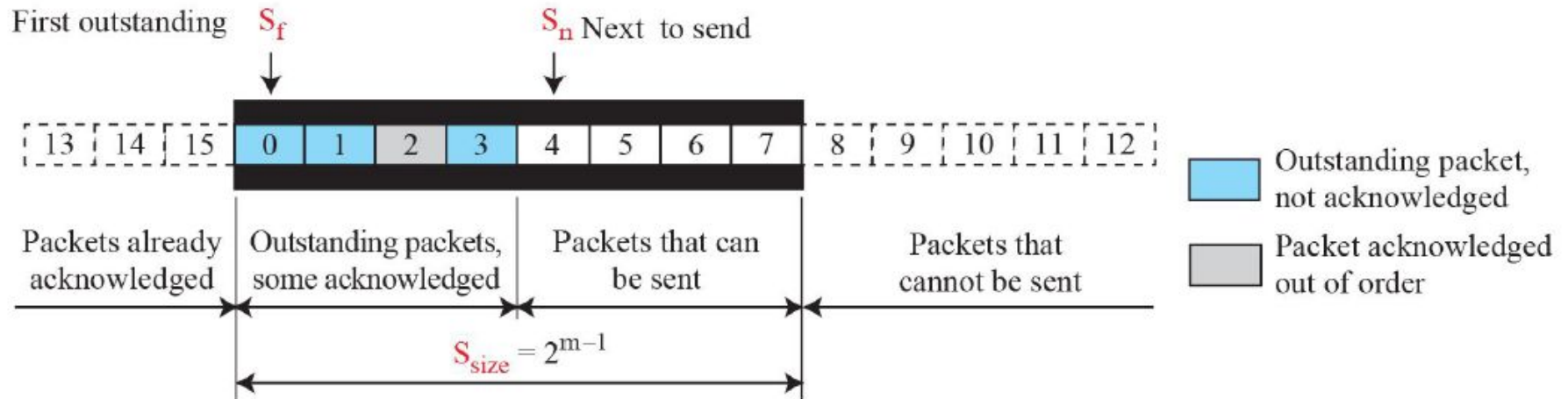
Selective-Repeat Protocol



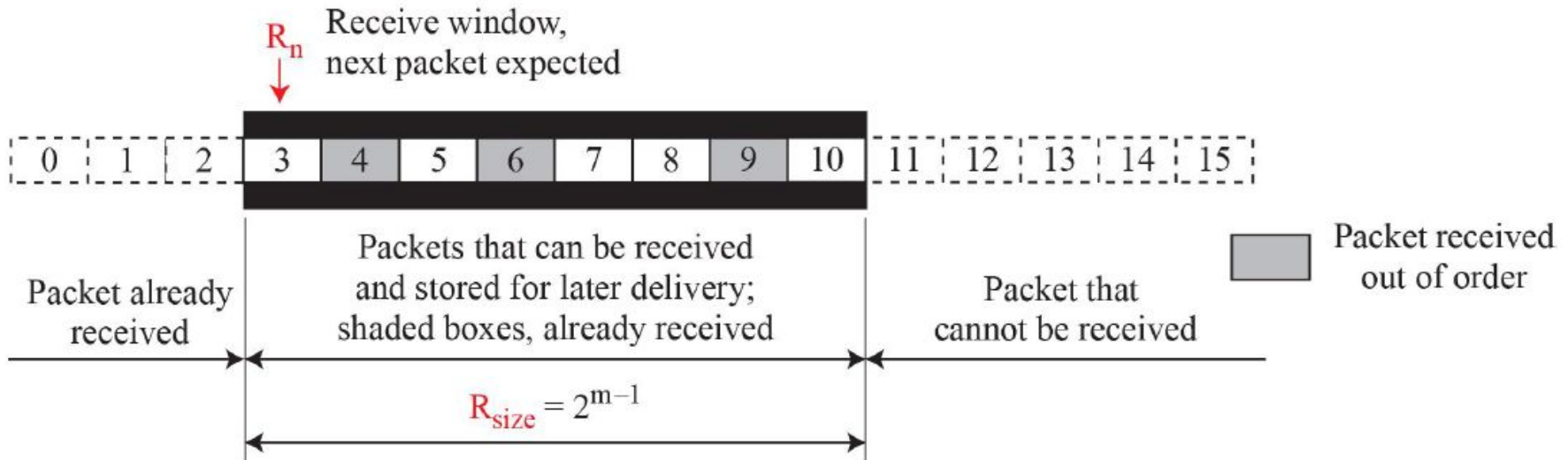
Sent, but not acknowledged
 Acknowledged out of order



Selective-Repeat Protocol



Selective-Repeat Protocol



Selective-Repeat

Protocol

Assume a sender sends 6 packets: packets 0, 1, 2, 3, 4, and 5. The sender receives an ACK with $\text{ackNo} = 3$. What is the interpretation if the system is using GBN or SR?

Solution

If the system is using GBN, it means that packets 0, 1, and 2 have been received uncorrupted and the receiver is expecting packet 3. If the system is using SR, it means that packet 3 has been received uncorrupted; the ACK does not say anything about other packets.

Selective-Repeat Protocol

This example is similar to Example 3.8 (Figure 3.30) in which packet 1 is lost. We show how Selective-Repeat behaves in this case. Figure 3.35 shows the situation.

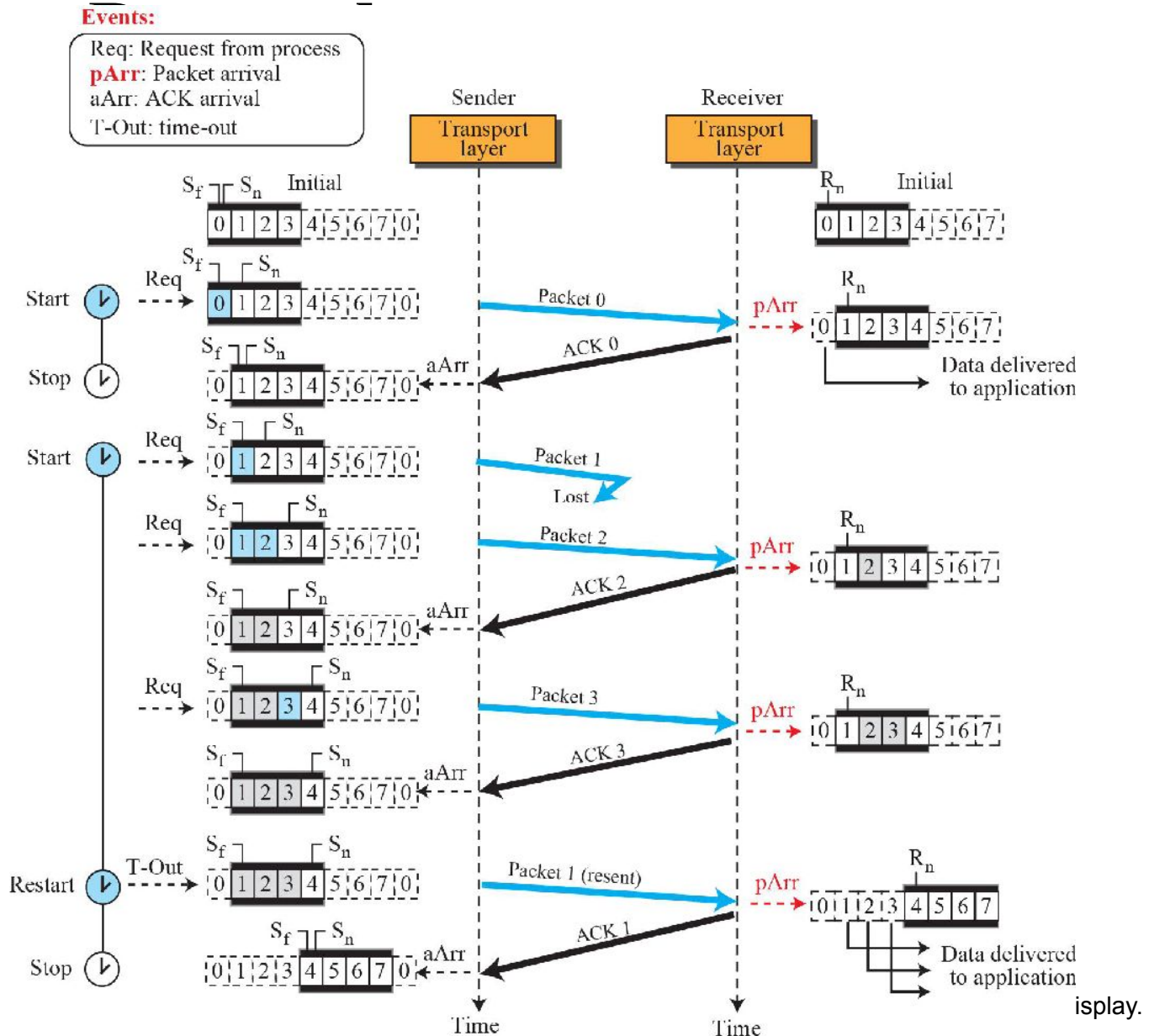
At the sender, packet 0 is transmitted and acknowledged. Packet 1 is lost. Packets 2 and 3 arrive out of order and are acknowledged. When the timer times out, packet 1 (the only unacknowledged packet) is resent and is acknowledged. The send window then slides.

Selective-Repeat

Protocol

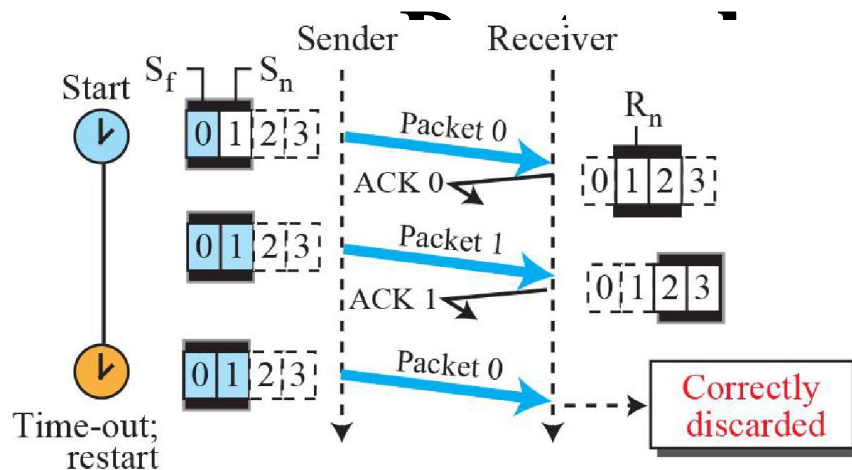
- At the receiver site, acceptance of a packet and its delivery to the application layer.
- At the second arrival, packet 2 arrives and is stored and marked (shaded slot), but it cannot be delivered because packet 1 is missing.
- At the next arrival, packet 3 arrives and is marked and stored, but still none of the packets can be delivered.
- Only at the last arrival, when finally a copy of packet 1 arrives, can packets 1, 2, and 3 be delivered to the application layer.
- There are two conditions for the delivery of packets to the application layer: First, a set of consecutive packets must have arrived. Second, the set starts from the beginning of the window.

Selective-Repeat

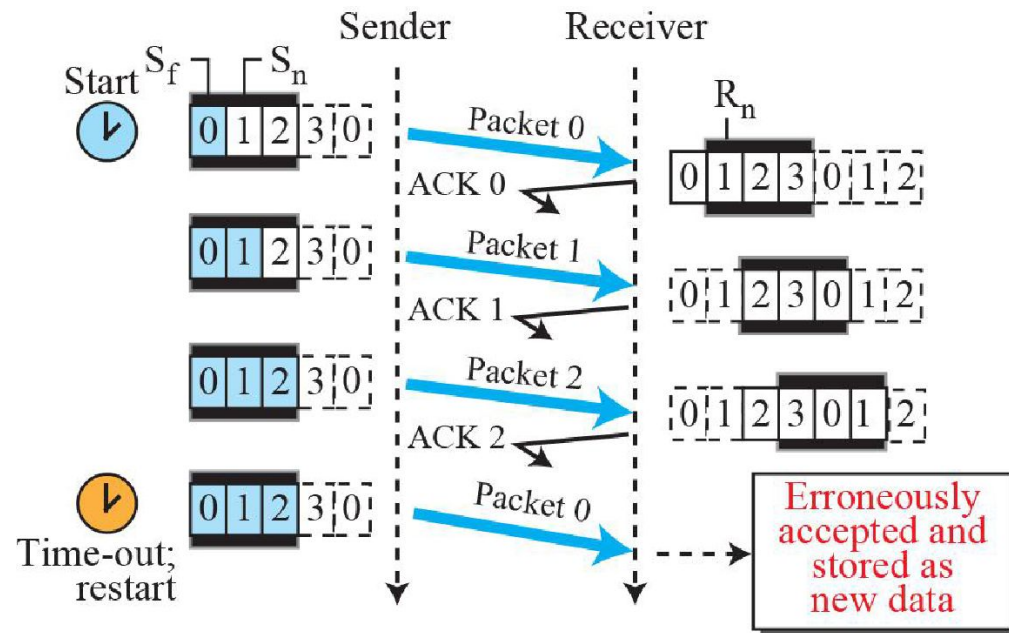


isplay.

Selective-Repeat



a. Send and receive windows of size $= 2^m - 1$



b. Send and receive windows of size $> 2^m - 1$

Selective-Repeat

Protocol

Link Utilization:

$$\begin{aligned} U &= \text{Time taken to transmit the packet to link} / \text{Total time} \\ &= D_{\text{trans}} / (D_{\text{trans}} + 2 * D_{\text{prop}} + D_{\text{proc}} + D_{\text{queue}}) \\ &= D_{\text{trans}} / (D_{\text{trans}} + 2 * D_{\text{prop}}) \end{aligned}$$

$$U = N / (1 + 2a)$$

$$a = D_{\text{prop}} / D_{\text{trans}}$$

N = Number of packets

If P = probability of frame error then

$$U = N(1 - P) / (1 + 2a)$$

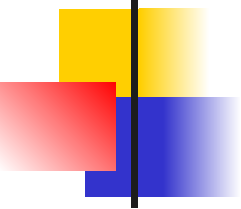
Selective-Repeat

Protocol

Q-1: Consider the use of 10kilobit size frame on a 10Mbps satellite channel with 270ms delay. What is the link utilization for SR protocol assuming $P=10^{-3}$ where P is the probability of single bit error and window size is 7.

Q-2: Consider designing a SR protocol with bandwidth of 100Kbps and one way delay of 4sec. Assume each packet carries 1KB Data. What is the minimum number of bits required for sequence number.

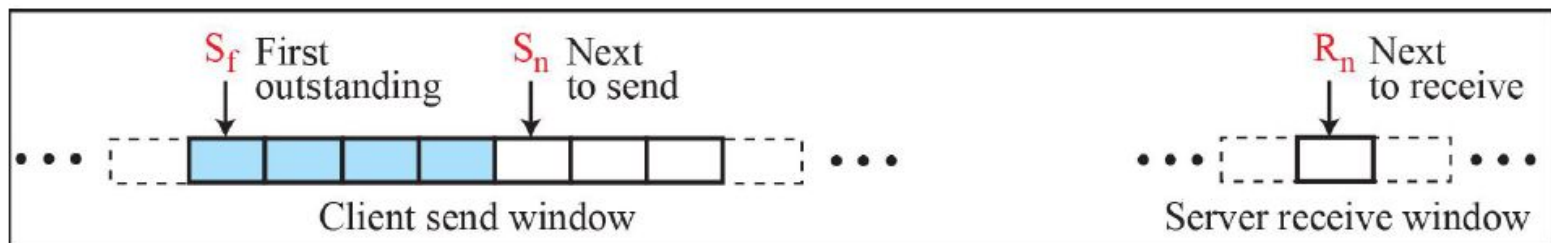
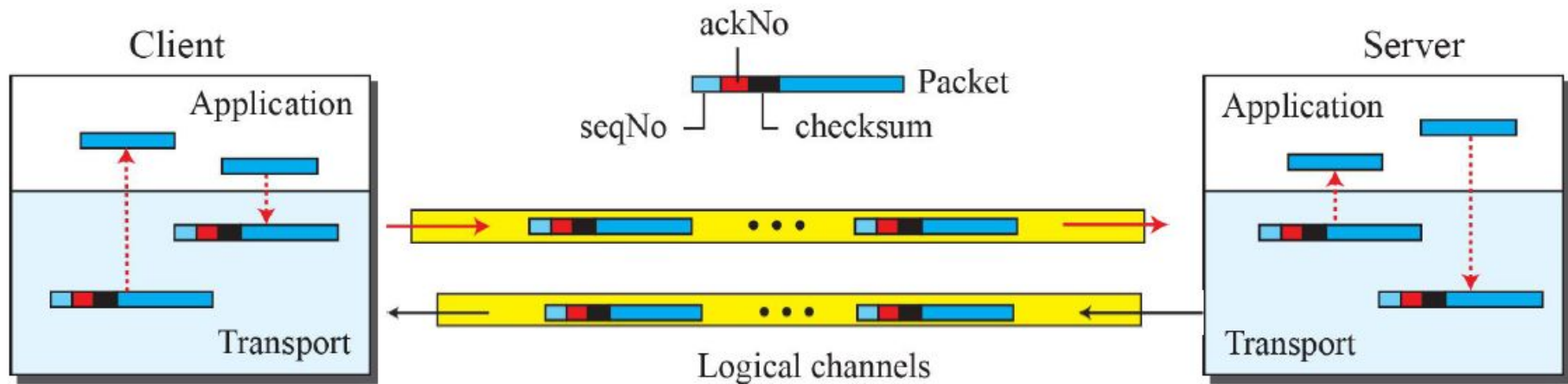
Answer: 8



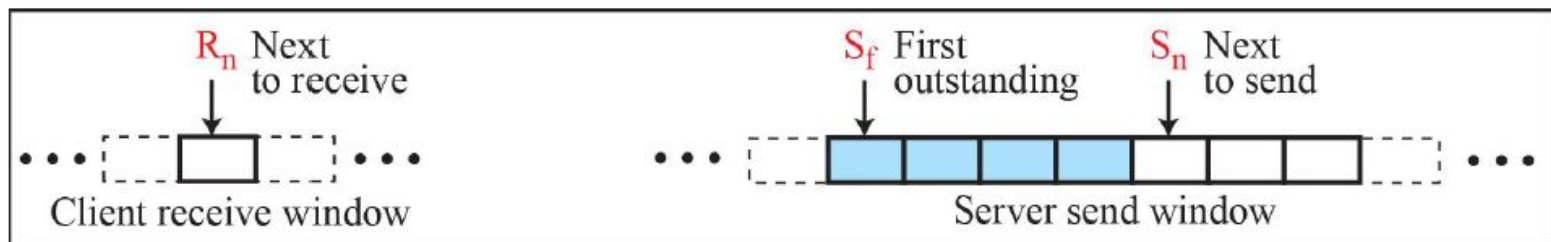
Bidirectional Protocols

- The four protocols we discussed earlier in this section are all unidirectional: data packets flow in only one direction and acknowledgments travel in the other direction.
- In real life, data packets are normally flowing in both directions: from client to server and from server to client.
- This means that acknowledgments also need to flow in both directions.
- A technique called piggybacking is used to improve the efficiency of the bidirectional protocols.

Bidirectional



Windows for communication from client to server



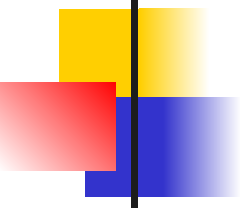
Windows for communication from server to client

Bidirectional Protocols

Consider the sliding window flow-control protocol operating between a sender and a receiver over a full-duplex error-free link. Assume the following:

1. The time taken for processing the data frame by the receiver is negligible.
2. The time taken for processing the acknowledgement frame by the sender is negligible.
3. The sender has infinite number of frames available for transmission.
4. The size of the data frame is 2,000 bits and the size of the acknowledgment frame is 10 bits.
5. The link data rate in each direction is 1 Mbps ($= 10^6$ bits per second).
6. One way propagation delay of the link is 100 milliseconds.

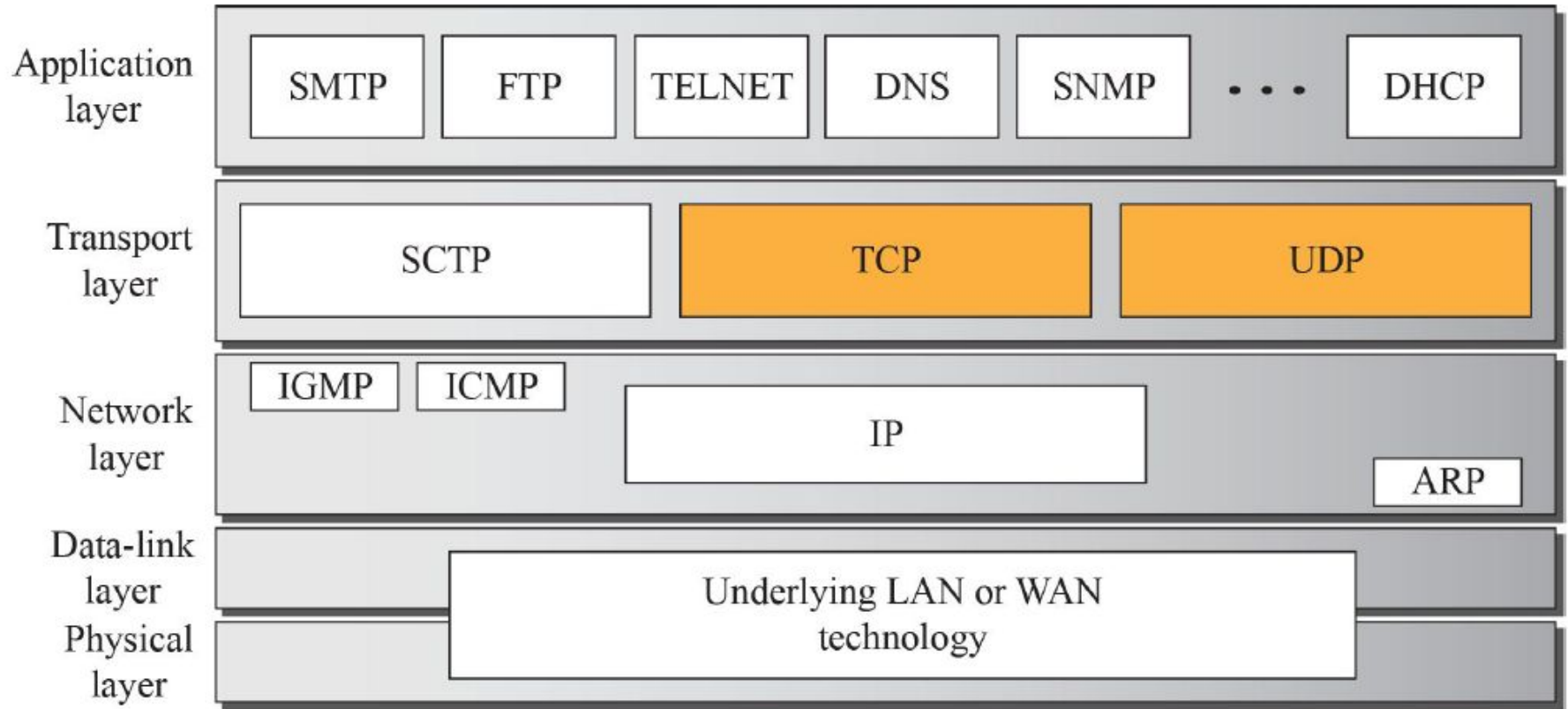
The minimum value of the sender's window size in terms of the number of frames, (rounded to the nearest integer) needed to achieve a link utilization of 50% is _____

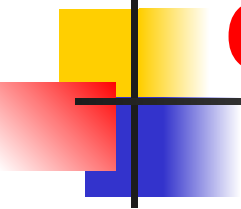


Internet Transport-Layer Protocols

- A network is the interconnection of a set of devices capable of communication.
- In this definition, a device can be a host such as a large computer, desktop, laptop, workstation, cellular phone, or security system.
- A device in this definition can also be a connecting device such as a router a switch, a modem that changes the form of data, and so on.

Internet Transport-Layer



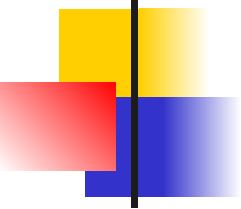


Chapter 3: Outline

3.5.UDP: Services and Applications, Segment format

USER DATAGRAM PROTOCOL (UDP)

- The User Datagram Protocol (UDP) is a connectionless, unreliable transport protocol.
- It does not add anything to the services of IP except for providing process-to-process instead of host-to-host communication.
- UDP is a very simple protocol using a minimum of overhead.



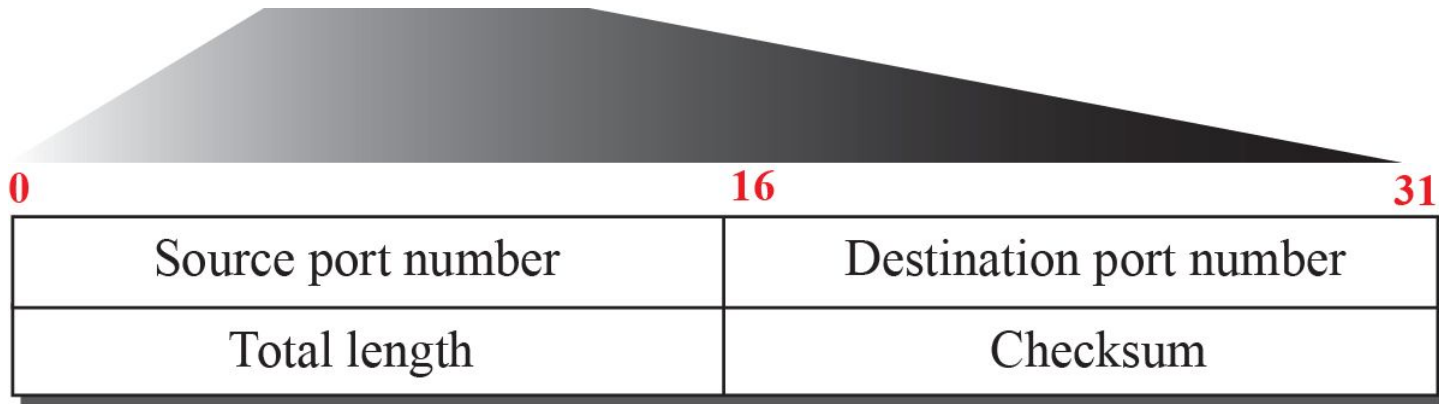
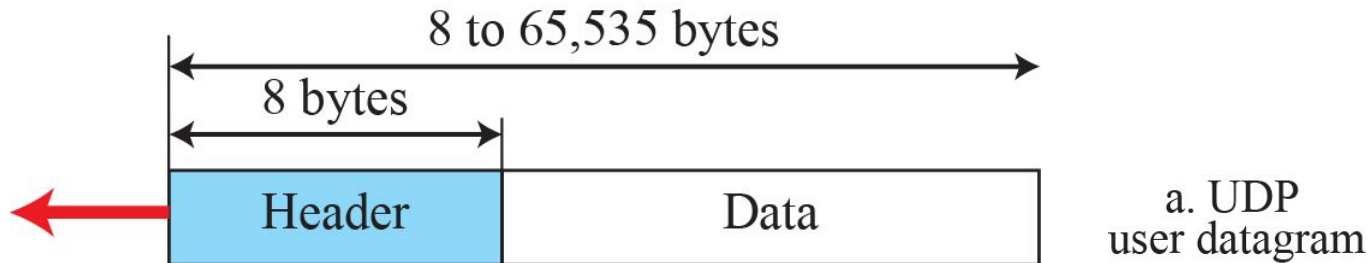
USER DATAGRAM PROTOCOL

User

Datagram

- UDP packets, called user datagrams, have a fixed size header of 8 bytes made of four fields, each of 2 bytes (16 bits).
- Figure 3.39 shows the format of a user datagram.
- The first two fields define the source and destination port numbers.
- The third field defines the total length of the user datagram, header plus data.
- The 16 bits can define a total length of 0 to 65,535 bytes.

USER DATAGRAM PROTOCOL



b. Header format

USER DATAGRAM PROTOCOL

The following is the contents of a UDP header in hexadecimal format.

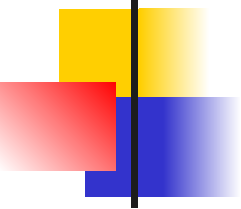
CB84000D001C001C

- a.** What is the source port number?
- b.** What is the destination port number?
- c.** What is the total length of the user datagram?
- d.** What is the length of the data?
- e.** Is the packet directed from a client to a server or vice versa?
- f.** What is the client process?

USER DATAGRAM PROTOCOL

Solution

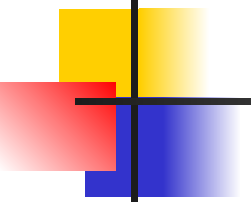
- a.** The source port number is the first four hexadecimal digits $(CB84)_{16}$ or 52100
- b.** The destination port number is the second four hexadecimal digits $(000D)_{16}$ or 13.
- c.** The third four hexadecimal digits $(001C)_{16}$ define the length of the whole UDP packet as 28 bytes.
- d.** The length of the data is the length of the whole packet minus the length of the header, or $28 - 8 = 20$ bytes.
- e.** Since the destination port number is 13 (well-known port), the packet is from the client to the server.
- f.** The client process is the Daytime (see Table 3.1).



USER DATAGRAM PROTOCOL

Services are provided by UDP:

- ☐ *Process-to-Process Communication*
- ☐ *Connectionless Services*
- ☐ *No Flow Control*
- ☐ *No Error Control*



USER DATAGRAM PROTOCOL

☐ *Checksum*

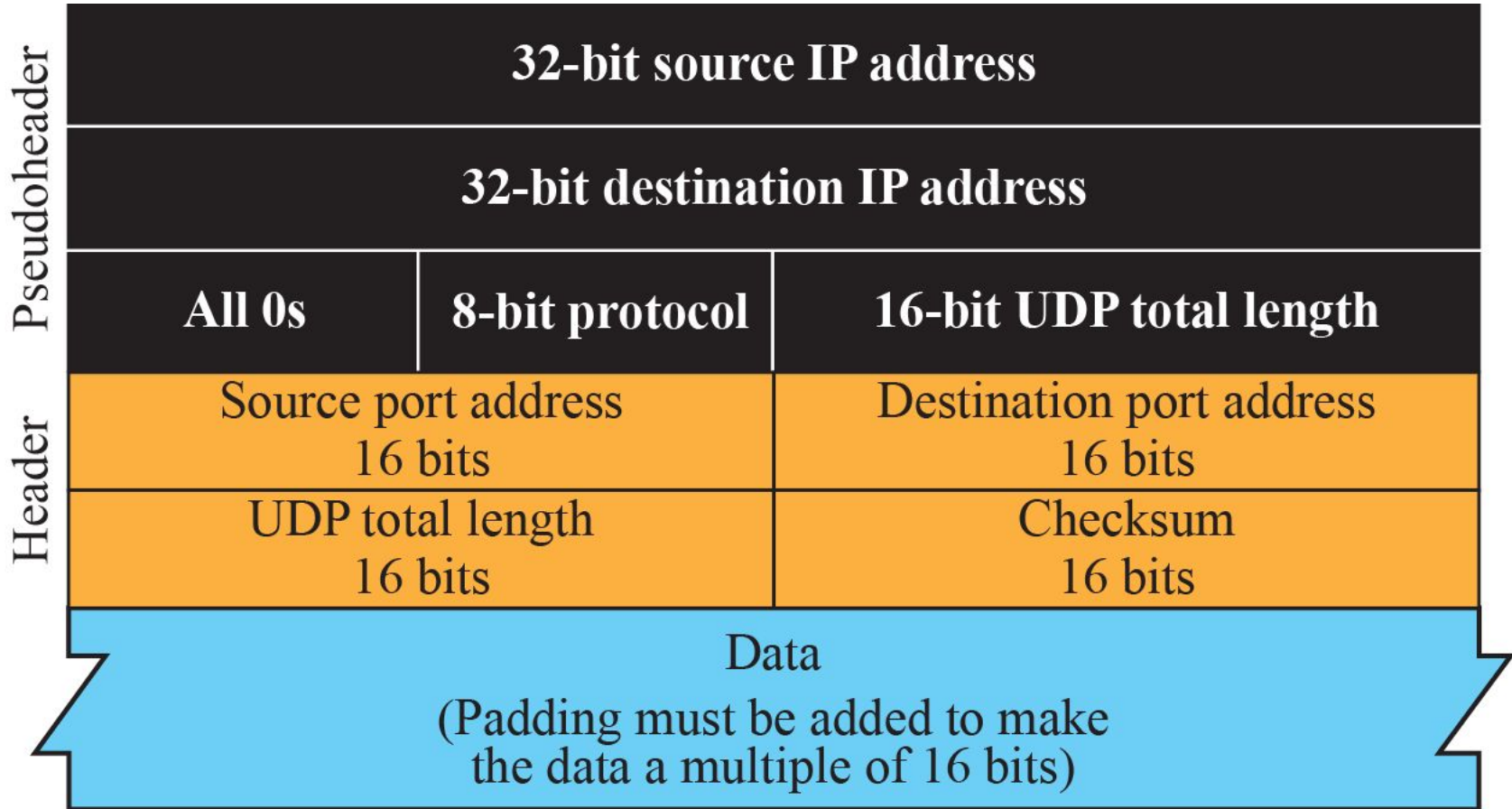
- ❖ *Optional Inclusion of Checksum*

☐ *No Congestion Control*

☐ *Encapsulation and Decapsulation*

☐ *Multiplexing and Demultiplexing*

USER DATAGRAM PROTOCOL



USER DATAGRAM PROTOCOL

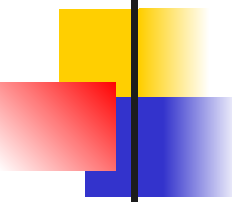
What value is sent for the checksum in one of the following hypothetical situations?

- a.** The sender decides not to include the checksum.
- b.** The sender decides to include the checksum, but the value of the sum is all 1s.
- c.** The sender decides to include the checksum, but the value of the sum is all 0s.

USER DATAGRAM PROTOCOL

Solution

- a.** The value sent for the checksum field is all 0s to show that the checksum is not calculated.
- b.** When the sender complements the sum, the result is all 0s; the sender complements the result again before sending. The value sent for the checksum is all 1s. The second complement operation is needed to avoid confusion with the case in part a.
- c.** This situation never happens because it implies that the value of every term included in the calculation of the sum is all 0s, which is impossible; some fields in the pseudoheader have nonzero values.



USER DATAGRAM PROTOCOL

UDP

Applications

- Although UDP meets almost none of the criteria we mentioned earlier for a reliable transport-layer protocol, UDP is preferable for some applications.
- The reason is that some services may have some side effects that are either unacceptable or not preferable.
- An application designer sometimes needs to compromise to get the optimum.

UDP Features

- ◆ *Connectionless Service*
- ◆ *Lack of Error Control*
- ◆ *Lack of Congestion Control*

USER DATAGRAM PROTOCOL

- A client-server application such as DNS uses the services of UDP because a client needs to send a short request to a server and to receive a quick response from it.
- The request and response can each fit in one user datagram. Since only one message is exchanged in each direction, the connectionless feature is not an issue; the client or server does not worry that messages are delivered out of order.

USER DATAGRAM PROTOCOL

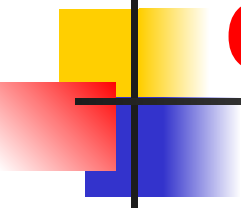
- A client-server application such as SMTP which is used in electronic mail, cannot use the services of UDP because a user can send a long e-mail message, which may include multimedia (images, audio, or video).
- If the application uses UDP and the message does not fit in one single user datagram, the message must be split by the application into different user datagrams. Here the connectionless service may create problems.
- The user datagrams may arrive and be delivered to the receiver application out of order.
- The receiver application may not be able to reorder the pieces.
- This means the connectionless service has a disadvantage for an application program that sends long messages.

USER DATAGRAM PROTOCOL

- Assume we are downloading a very large text file from the Internet.
- We definitely need to use a transport layer that provides reliable service.
- We don't want part of the file to be missing or corrupted when we open the file.
- The delay created between the deliveries of the parts is not an overriding concern for us; we wait until the whole file is composed before looking at it.
- In this case, UDP is not a suitable transport layer.
- Assume we are using a real-time interactive application, such as Skype. Audio and video are divided into frames and sent one after another.

USER DATAGRAM PROTOCOL

- If the transport layer is supposed to resend a corrupted or lost frame, the synchronizing of the whole transmission may be lost.
- The viewer suddenly sees a blank screen and needs to wait until the second transmission arrives. This is not tolerable.
- However, if each small part of the screen is sent using one single user datagram, the receiving UDP can easily ignore the corrupted or lost packet and deliver the rest to the application program.
- That part of the screen is blank for a very short period of time, which most viewers do not even notice.



Chapter 3: Outline

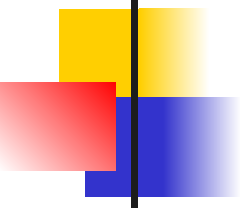
3.6. TCP: Services

- Segment format*
- TCP Connection management*
- State Transition Diagram*
- Windows in TCP*
- Flow Control*
- Congestion Control (Slow start, congestion avoidance, and fast recovery)*

TRANSMISSION CONTROL

PROTOCOL

- Transmission Control Protocol (TCP) is a connection-oriented, reliable protocol.
- TCP explicitly defines connection establishment, data transfer, and connection teardown phases to provide a connection-oriented service.
- TCP uses a combination of GBN and SR protocols to provide reliability.



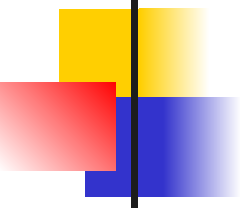
TCP Services

A Before discussing TCP in detail, let us explain the services offered by TCP to the processes at the application layer.

□ *Process-to-Process Communication*

□ *Stream Delivery Service*

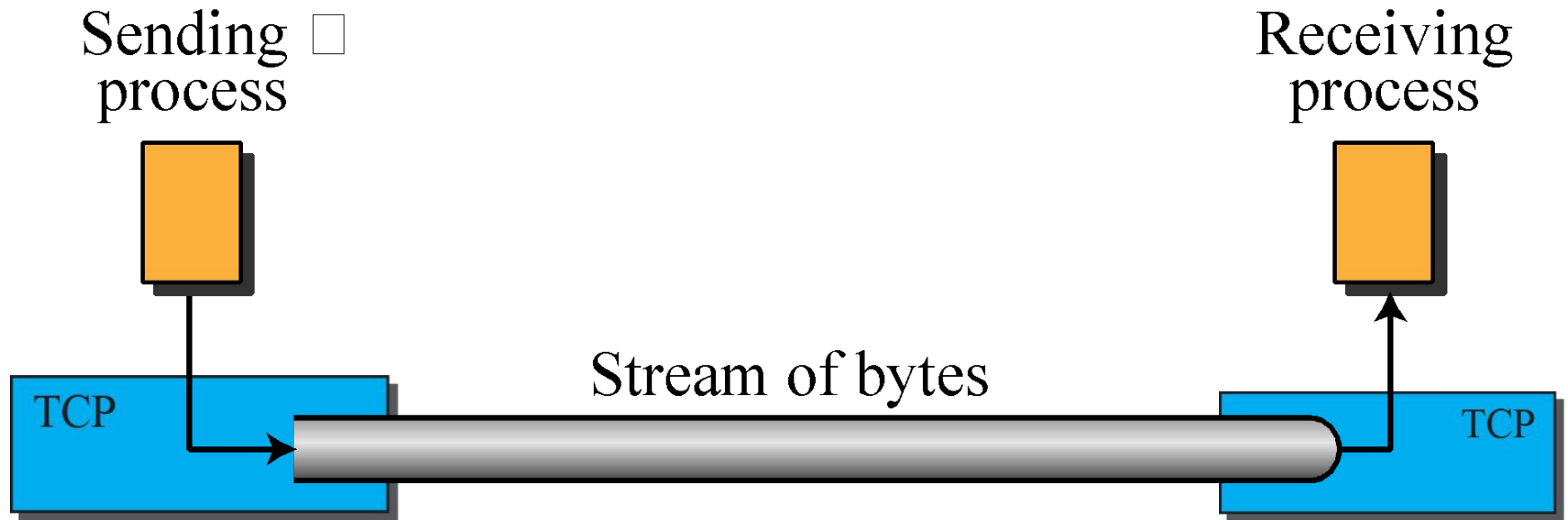
- ◆ *Sending and Receiving Buffers*
- ◆ *Segments*



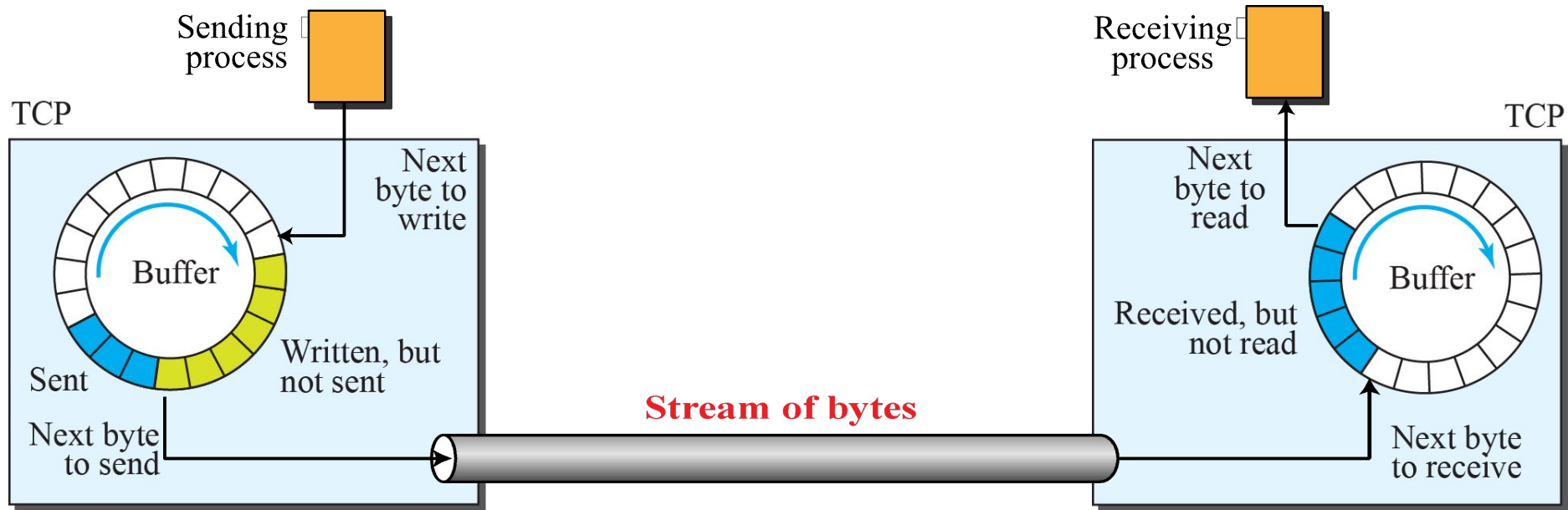
TCP Services

- ☐ *Full-Duplex Communication*
- ☐ *Multiplexing and Demultiplexing*
- ☐ *Connection-Oriented Service*
- ☐ *Reliable Service*

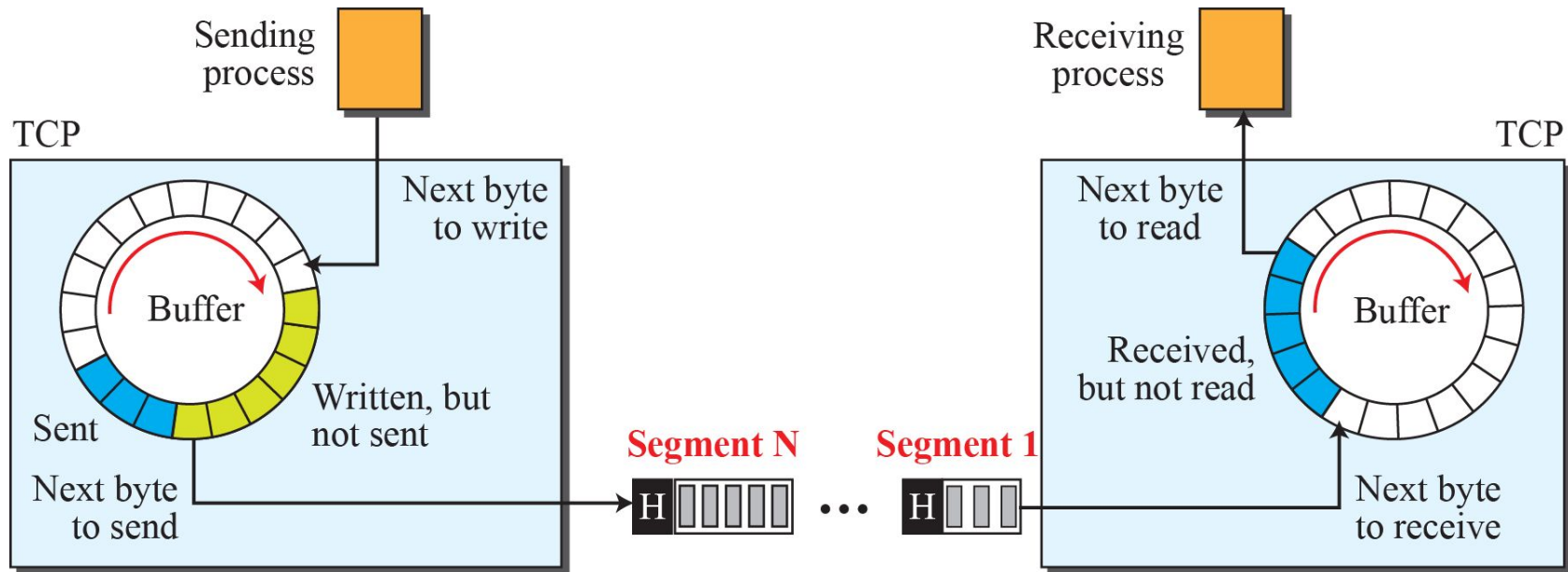
TCP Services

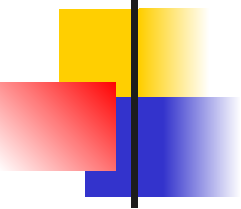


TCP Services



TCP Services





TCP

Features

To provide the services mentioned in the previous section, TCP has several features that are briefly summarized in this section and discussed later in detail.

Numbering System

- ❖ Byte Number
- ❖ Sequence Number
- ❖ Acknowledgment Number

TCP

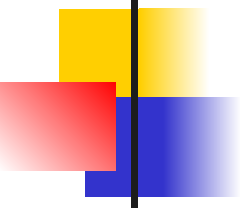
Features

Suppose a TCP connection is transferring a file of 5,000 bytes. The first byte is numbered 10,001. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1,000 bytes?

Solution

The following shows the sequence number for each segment:

Segment 1	→	Sequence Number:	10,001	Range:	10,001	to	11,000
Segment 2	→	Sequence Number:	11,001	Range:	11,001	to	12,000
Segment 3	→	Sequence Number:	12,001	Range:	12,001	to	13,000
Segment 4	→	Sequence Number:	13,001	Range:	13,001	to	14,000
Segment 5	→	Sequence Number:	14,001	Range:	14,001	to	15,000



TCP Segment

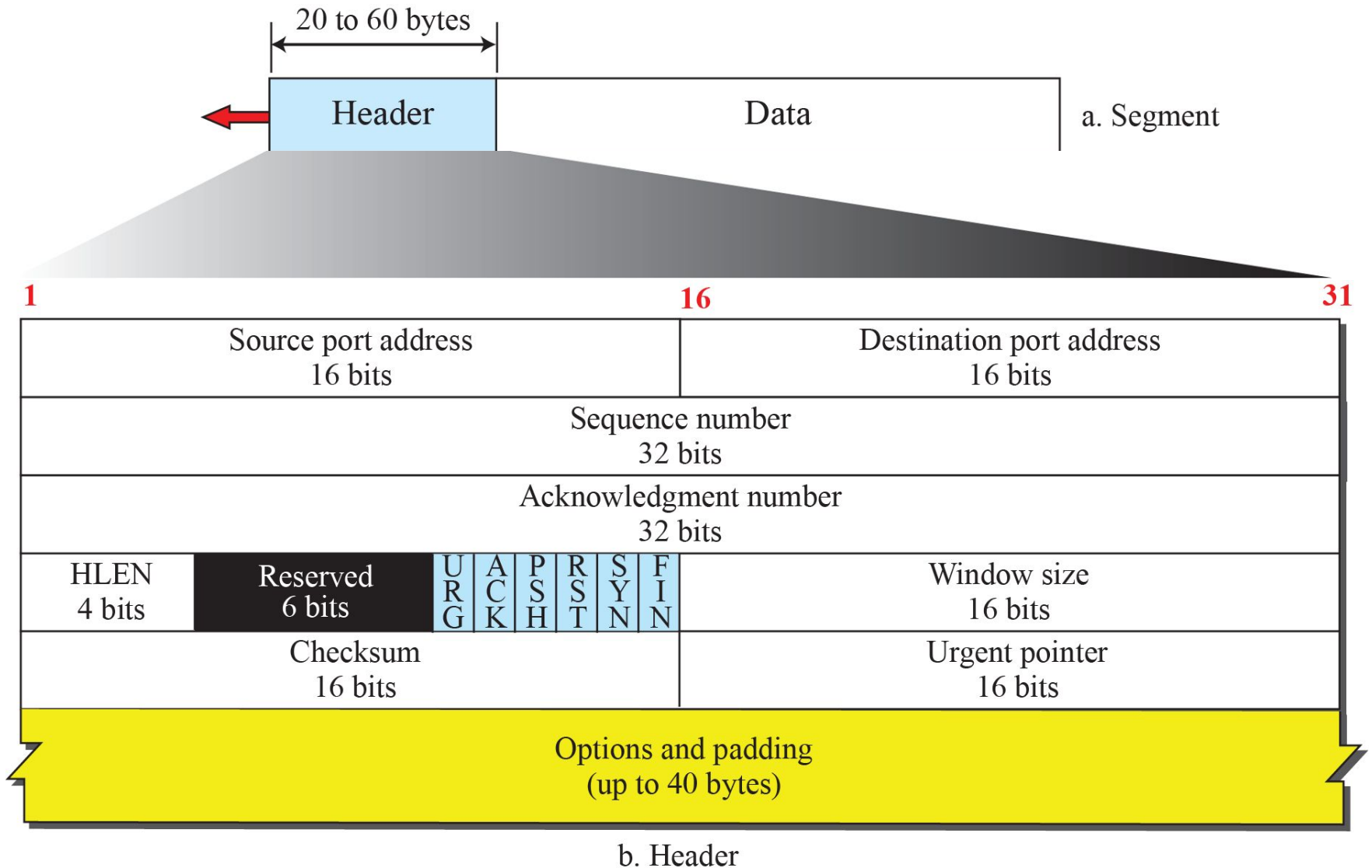
Before discussing TCP in more detail, let us discuss the TCP packets themselves.

A packet in TCP is called a **segment**.

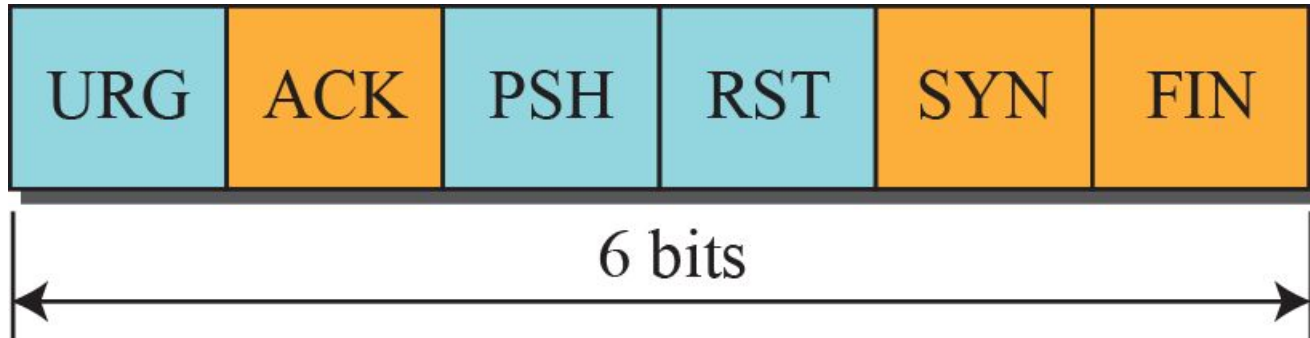
- *Format*

- *Encapsulation*

TCP Segment

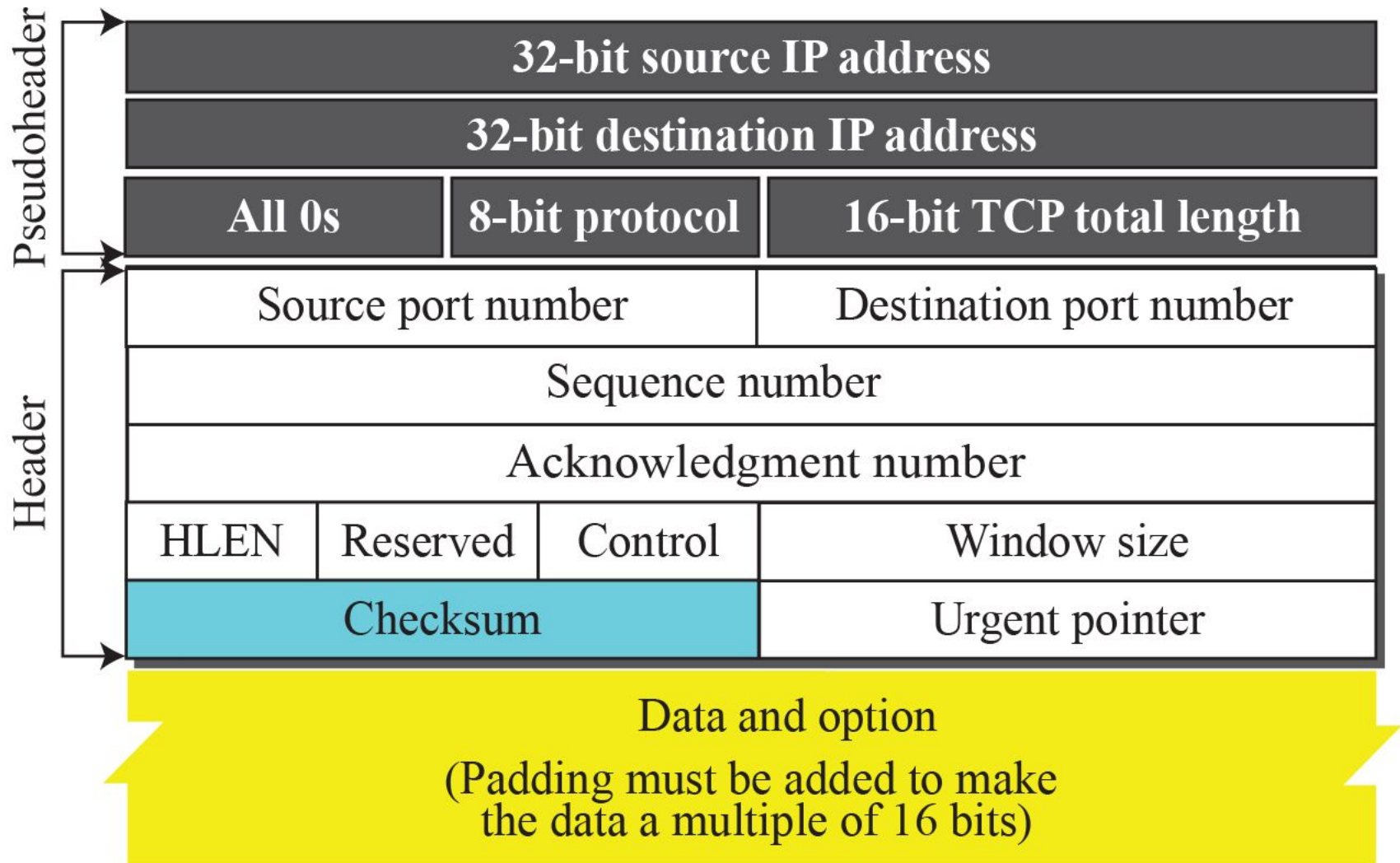


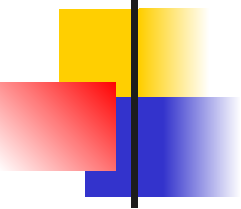
TCP Segment



URG: Urgent pointer is valid
ACK: Acknowledgment is valid
PSH: Request for push
RST: Reset the connection
SYN: Synchronize sequence numbers
FIN: Terminate the connection

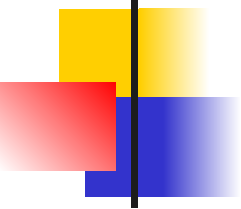
TCP Segment





TCP Connection

- TCP is connection-oriented.
- A connection-oriented transport protocol establishes a logical path between the source and destination.
- All of the segments belonging to a message are then sent over this logical path.
- Using a single logical pathway for the entire message facilitates the acknowledgment process as well as retransmission of damaged or lost frames.



TCP

Connection

Connection Establishment

- ◆ *Three-Way Handshaking*
- ◆ *SYN Flooding Attack*

Data Transfer

- ◆ *Pushing Data*
- ◆ *Urgent Data*

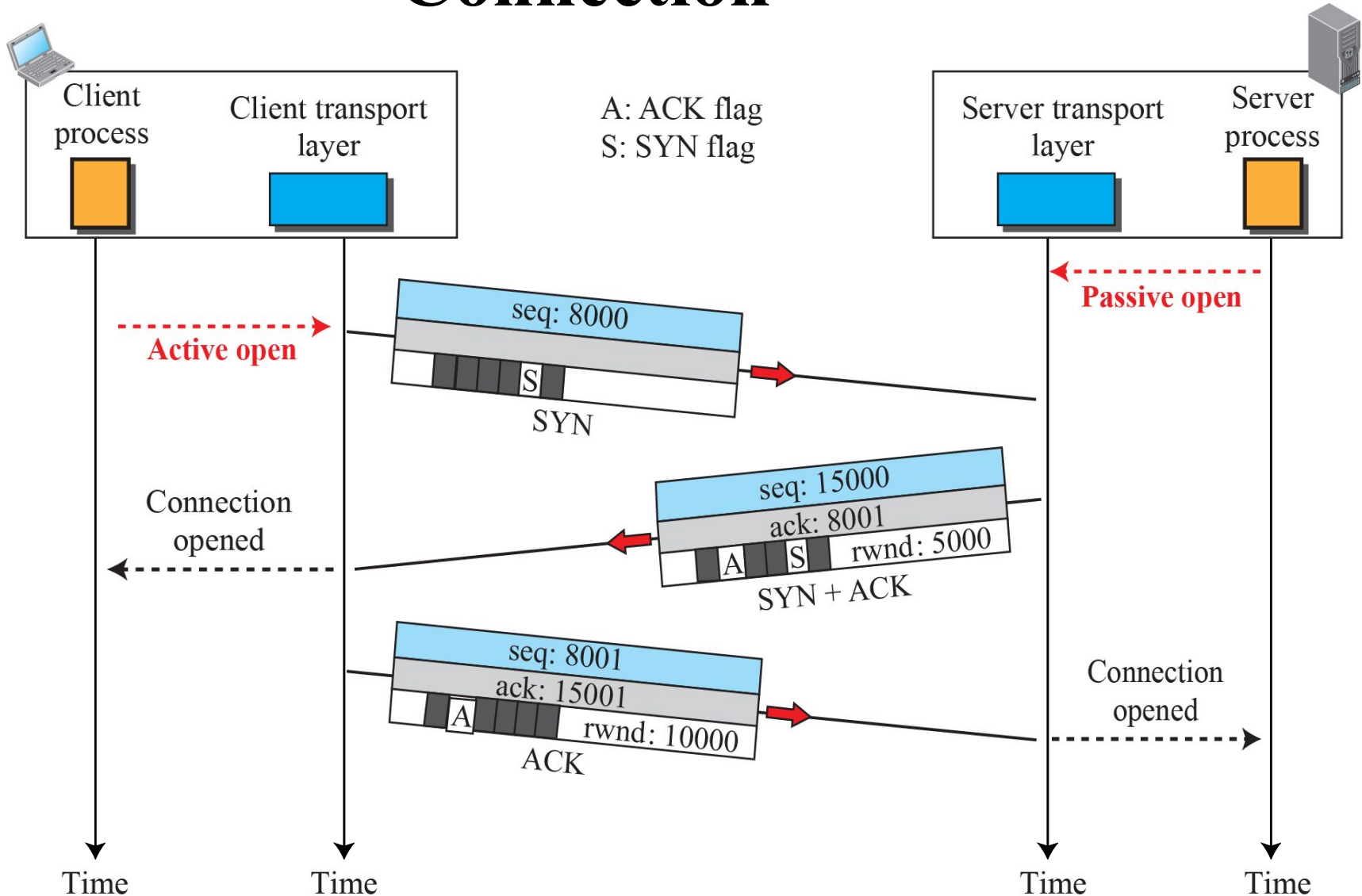
Connection Termination

- ◆ *Three-Way Handshaking*
- ◆ *Half-Close*

Connection Reset

TCP

❏ Connection Establishment

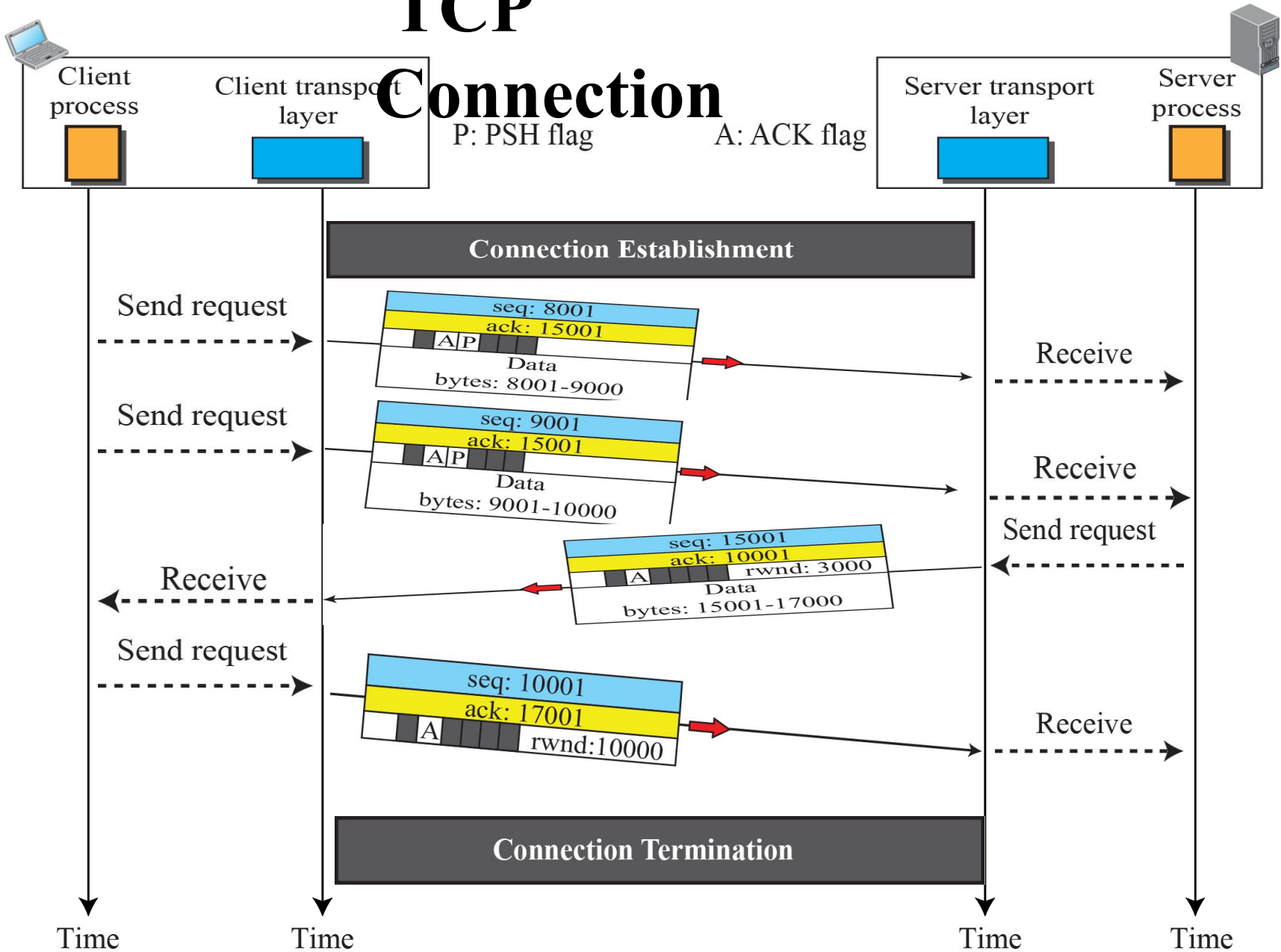


TCP

Connection

P: PSH flag

A: ACK flag



TCP

Connection Termination



Client process

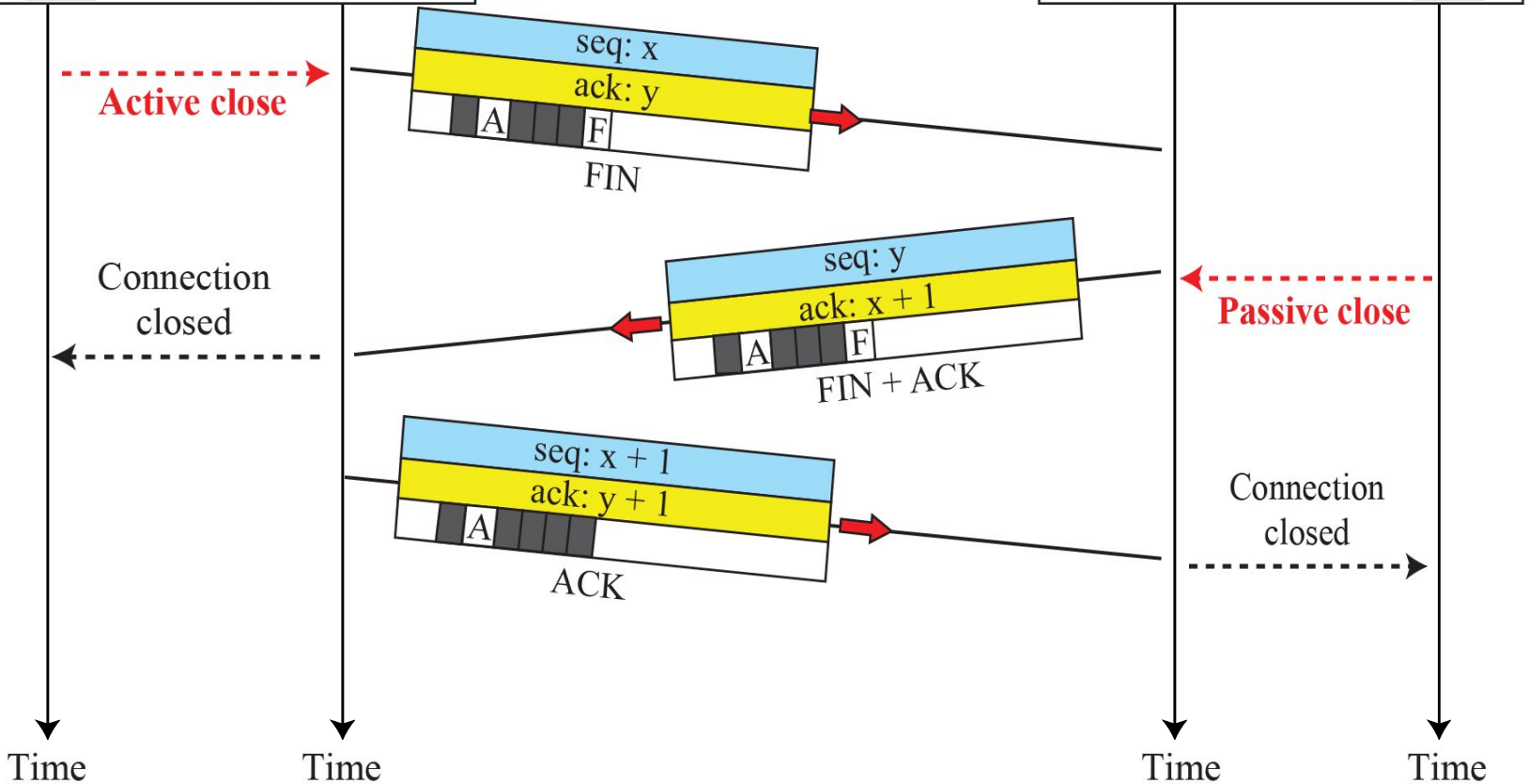
Client transport layer

A: ACK flag

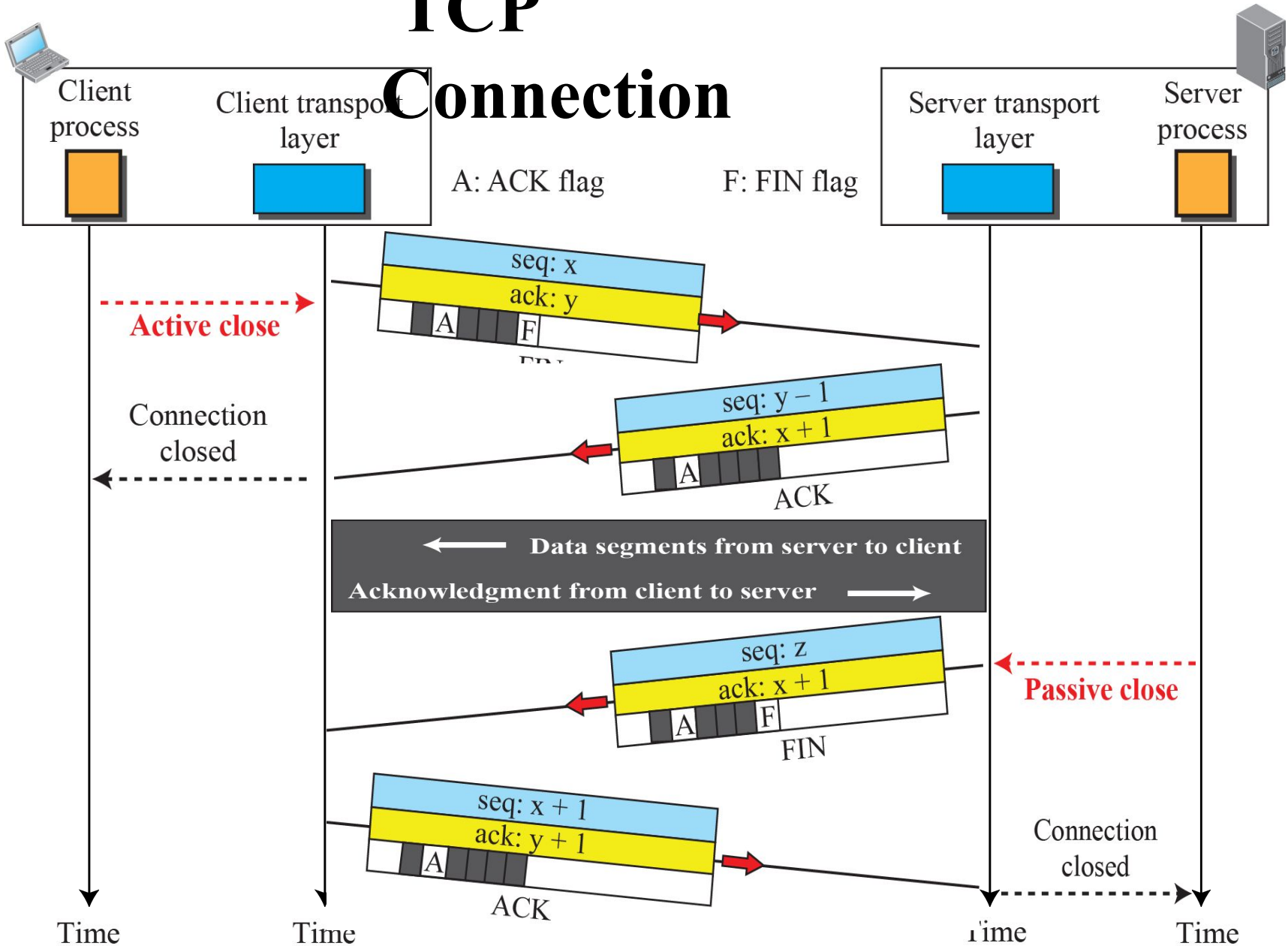
F: FIN flag

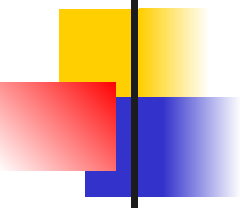
Server transport layer

Server process



TCP Connection





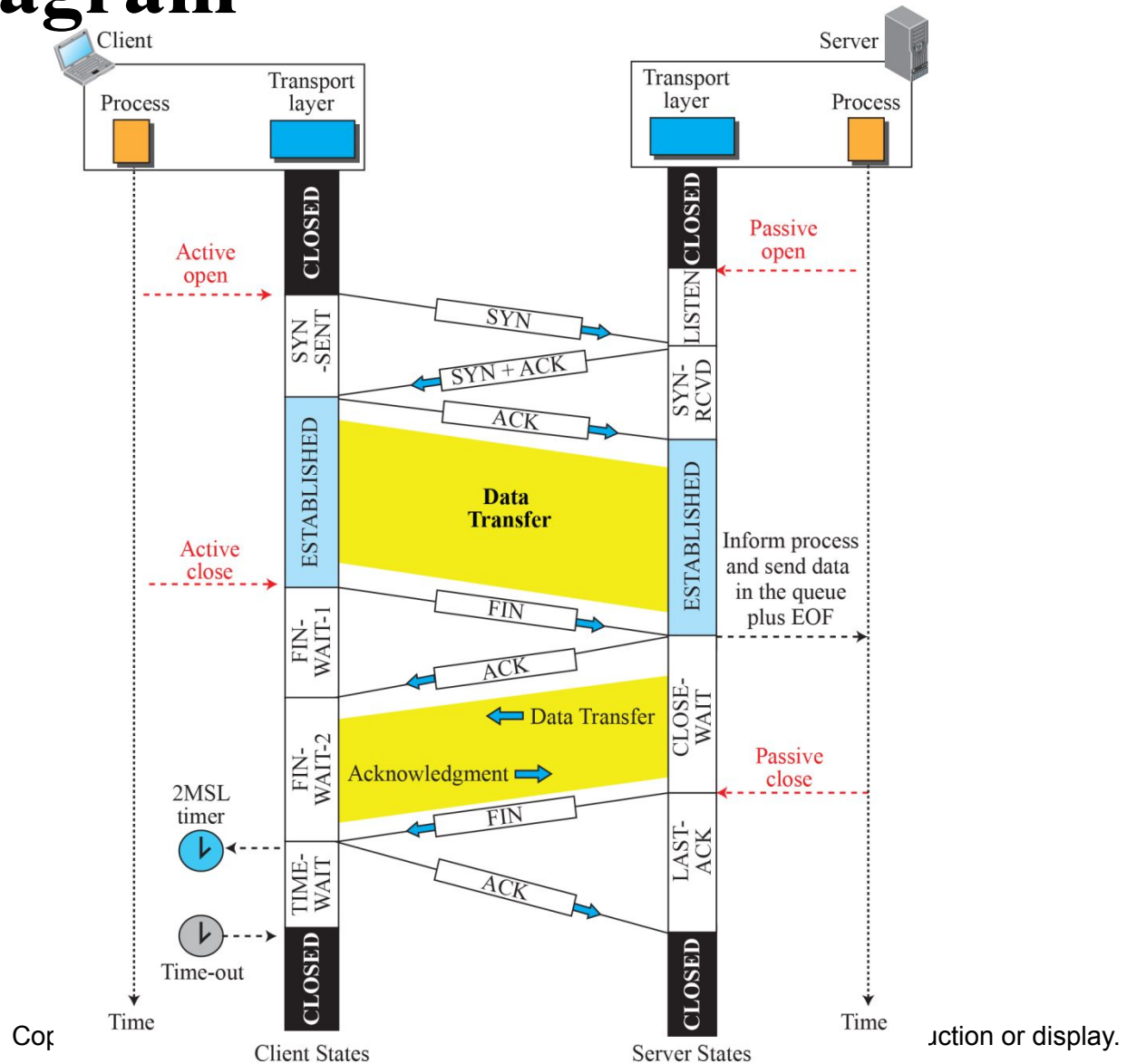
State Transmission Diagram

To keep track of all the different events happening during connection establishment, connection termination, and data transfer, TCP is specified as the finite state machine (FSM) as shown in Figure 3.51.

□ *Scenarios*

◆ *A Half-Close Scenario*

State Transmission Diagram

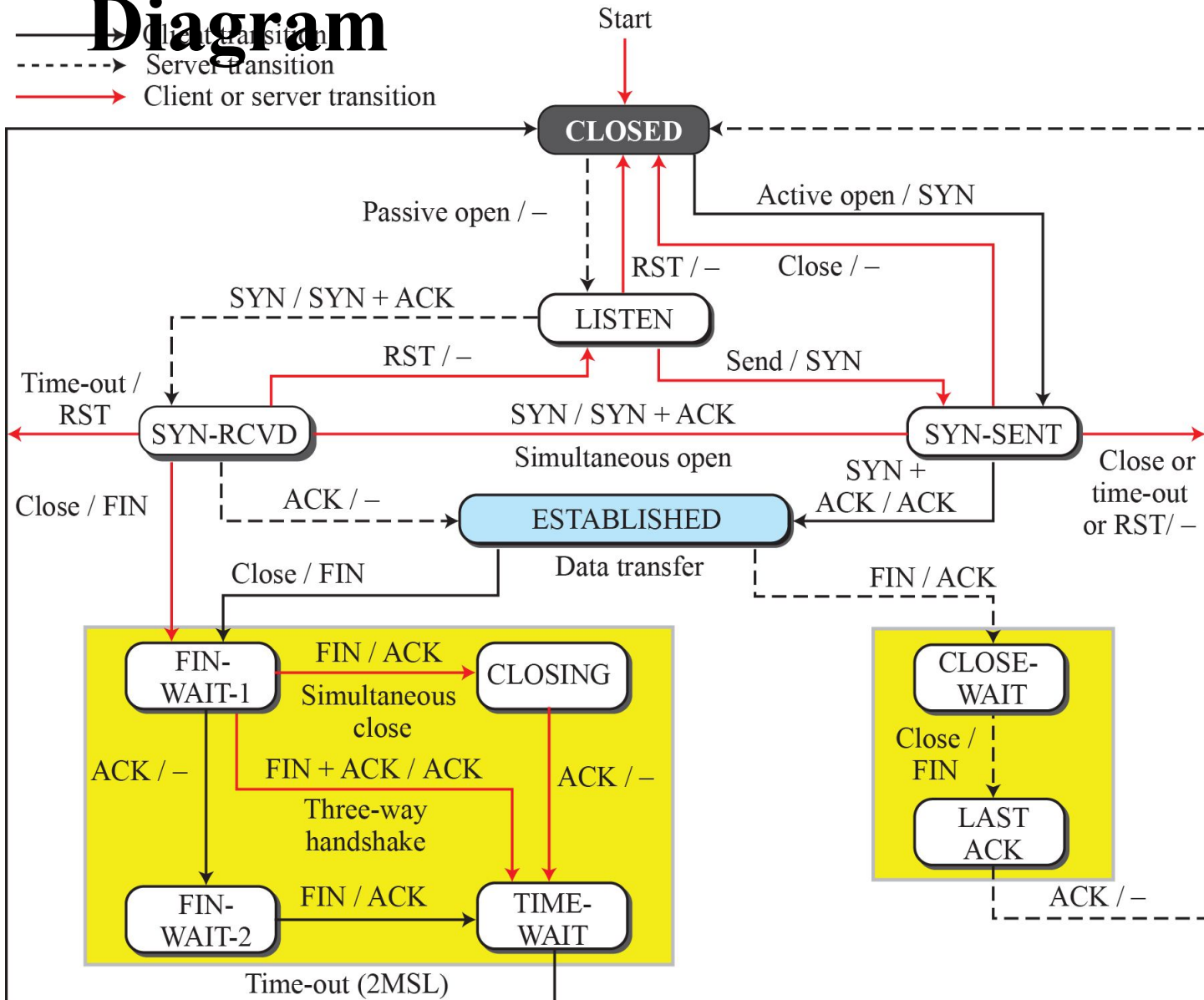


State Transmission Diagram

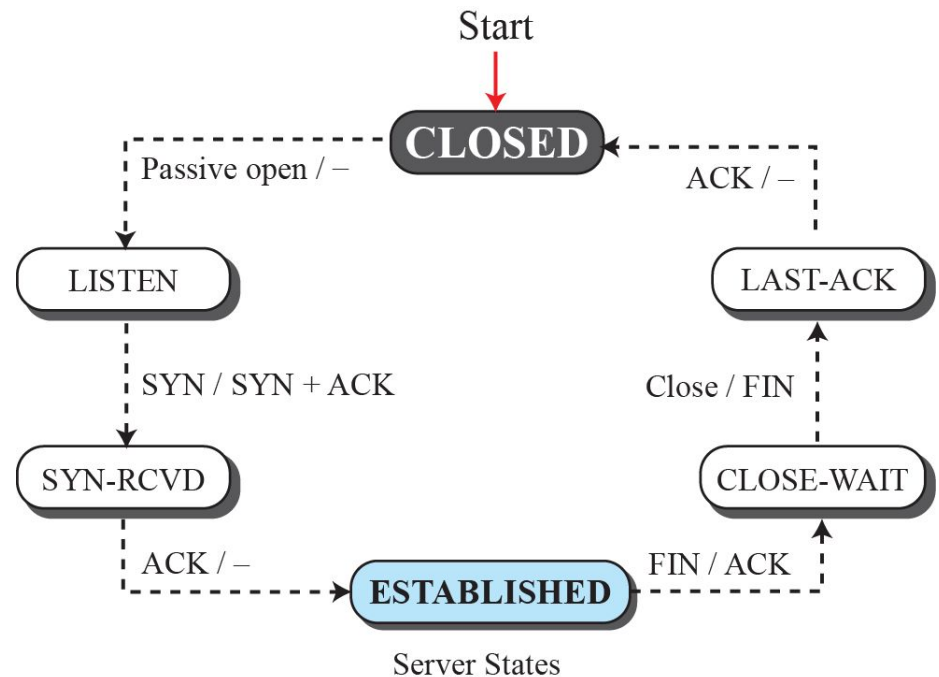
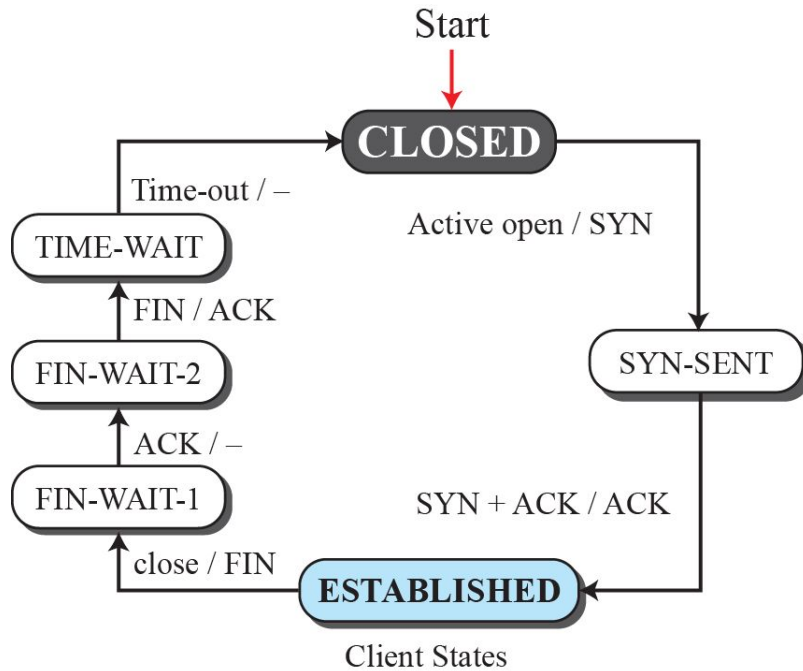
<i>State</i>	<i>Description</i>
CLOSED	No connection exists
LISTEN	Passive open received; waiting for SYN
SYN-SENT	SYN sent; waiting for ACK
SYN-RCVD	SYN+ACK sent; waiting for ACK
ESTABLISHED	Connection established; data transfer in progress
FIN-WAIT-1	First FIN sent; waiting for ACK
FIN-WAIT-2	ACK to first FIN received; waiting for second FIN
CLOSE-WAIT	First FIN received, ACK sent; waiting for application to close
TIME-WAIT	Second FIN received, ACK sent; waiting for 2MSL time-out
LAST-ACK	Second FIN sent; waiting for ACK
CLOSING	Both sides decided to close simultaneously

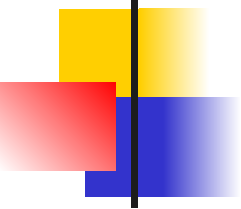
State Transmission

Diagram



State Transmission Diagram





Windows in TCP

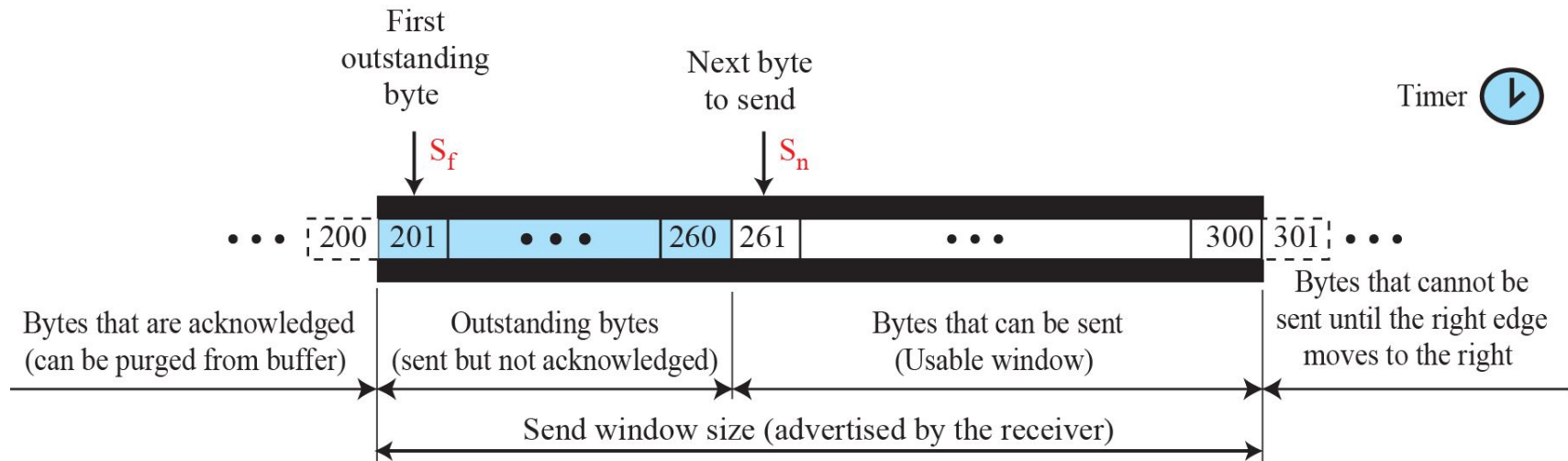
- TCP uses two windows (send window and receive window) for each direction of data transfer, which means four windows for a bidirectional communication.
- To make the discussion simple, we make an unrealistic unidirectional; the bidirectional communication can be inferred using two unidirectional communications with piggybacking.

 *Send Window*

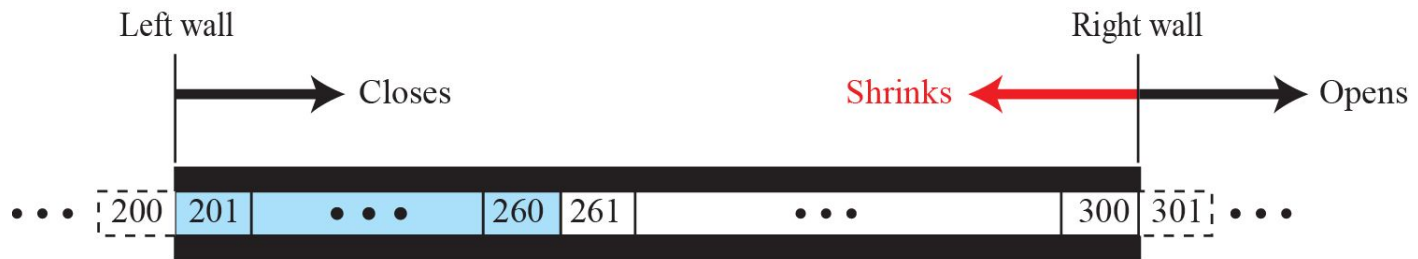
 *Receive Window*

Windows in TCP

Send Windows



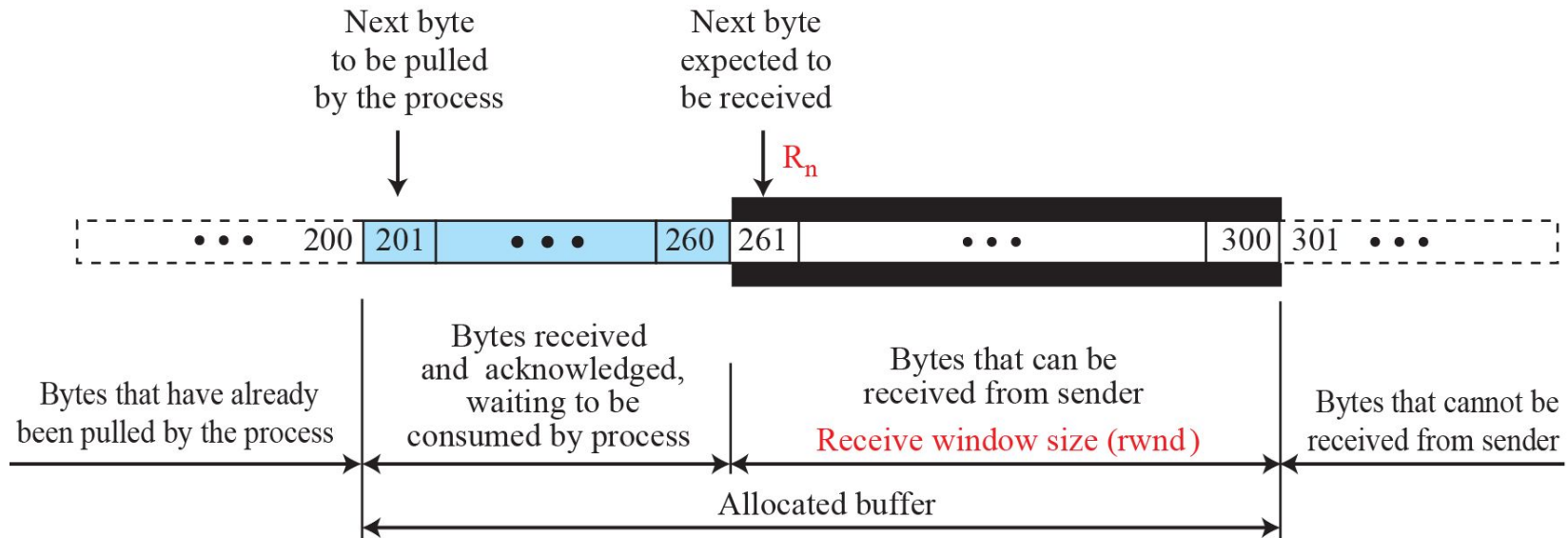
a. Send window



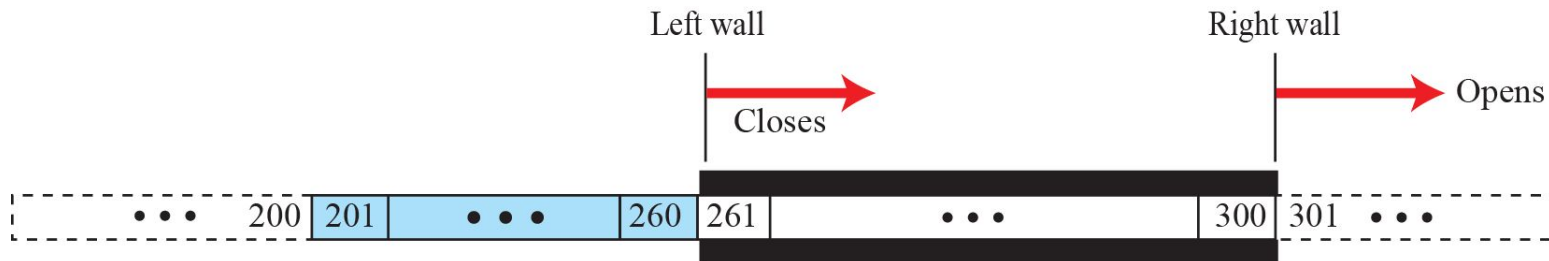
b. Opening, closing, and shrinking send window

Windows in TCP

Receive Window



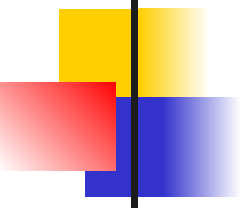
a. Receive window and allocated buffer



b. Opening and closing of receive window

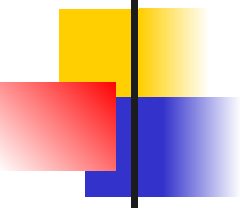
Question

Consider a long-lived *TCP* session with an end-to-end bandwidth of 1 *Gbps* ($= 10^9$ bits-persecond). The session starts with a sequence number of 1234. The minimum time (in seconds, rounded to the closest integer) before this sequence number can be used again is _____.



Flow Control

- As discussed before, flow control balances the rate a producer creates data with the rate a consumer can use the data.
- TCP separates flow control from error control.
- In this section we discuss flow control, ignoring error control.
- We assume that the logical channel between the sending and receiving TCP is error-free.



Flow Control

☐ *Opening and Closing Windows*

◆ *A Scenario*

☐ *Shrinking of Windows*

◆ *Window Shutdown*

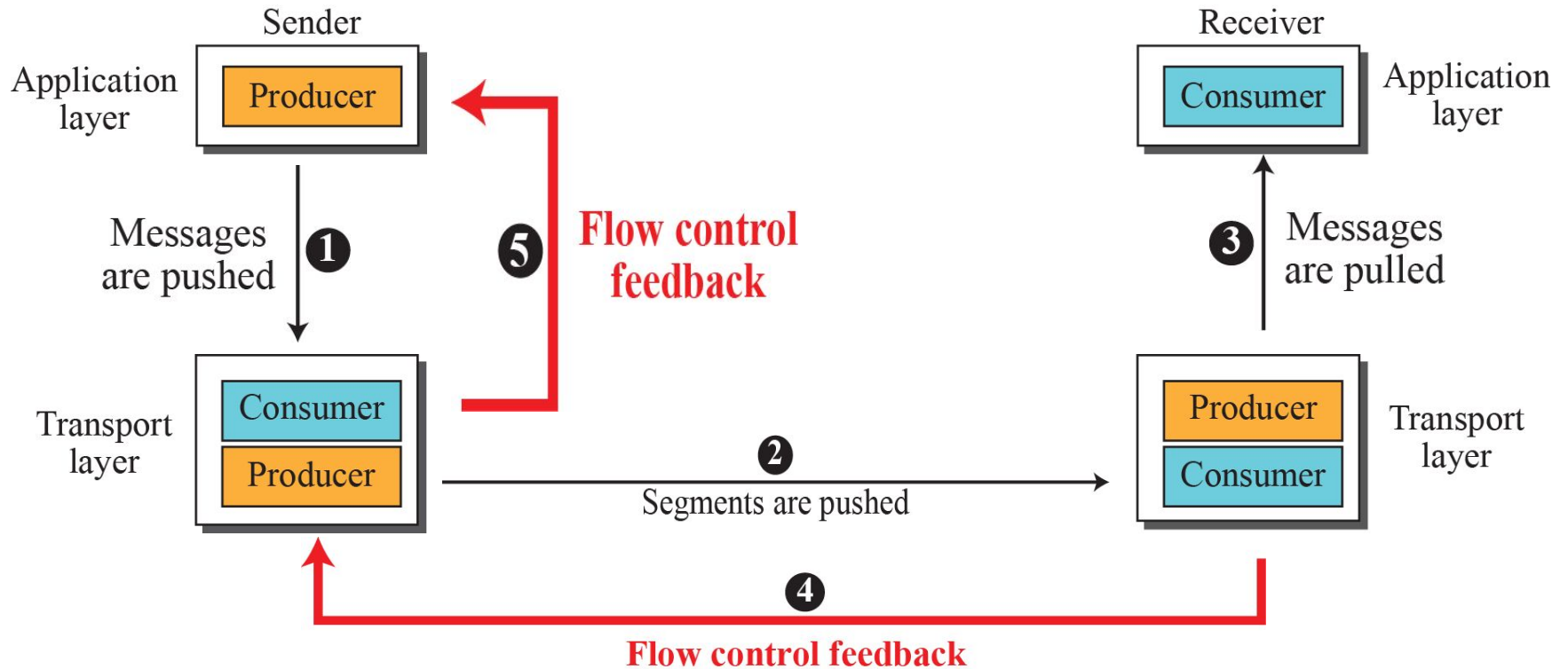
☐ *Silly Window Syndrome*

◆ *Syndrome Created by the Sender*

◆ *Syndrome Created by the Receiver*

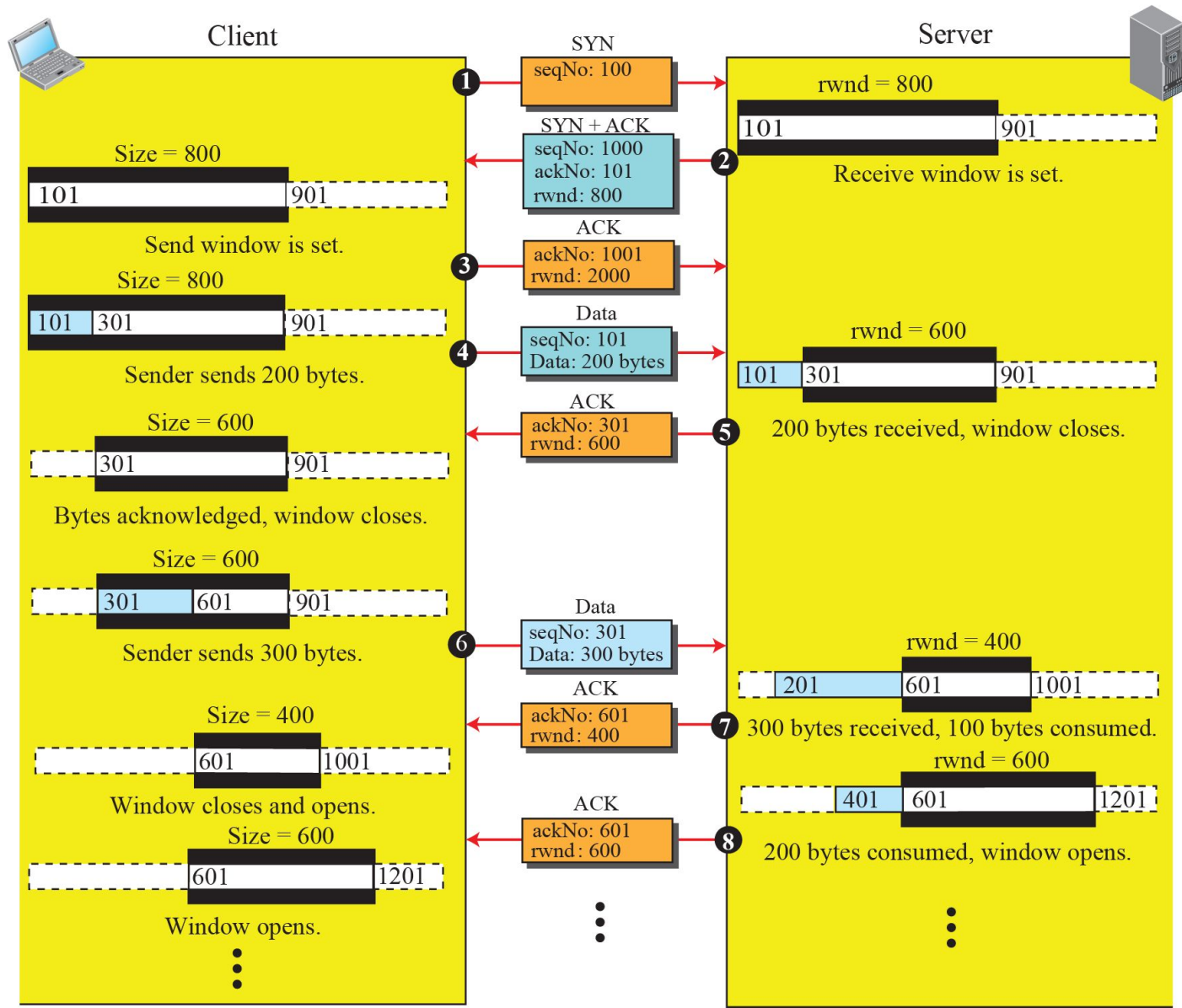
Flow Control

→ Data flow
→ Flow control feedback



Flow Control

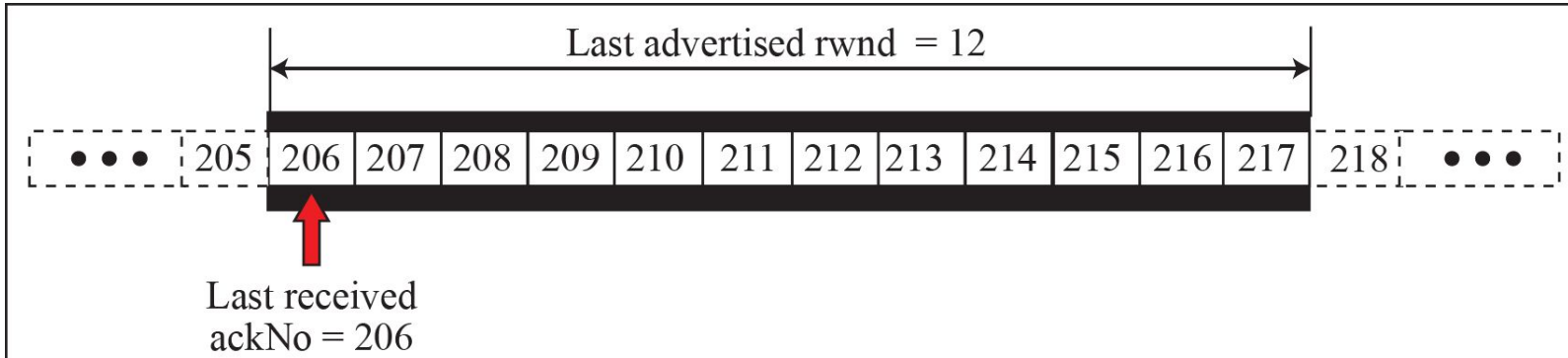
Note: We assume only unidirectional communication from client to server. Therefore, only one window at each side is shown.



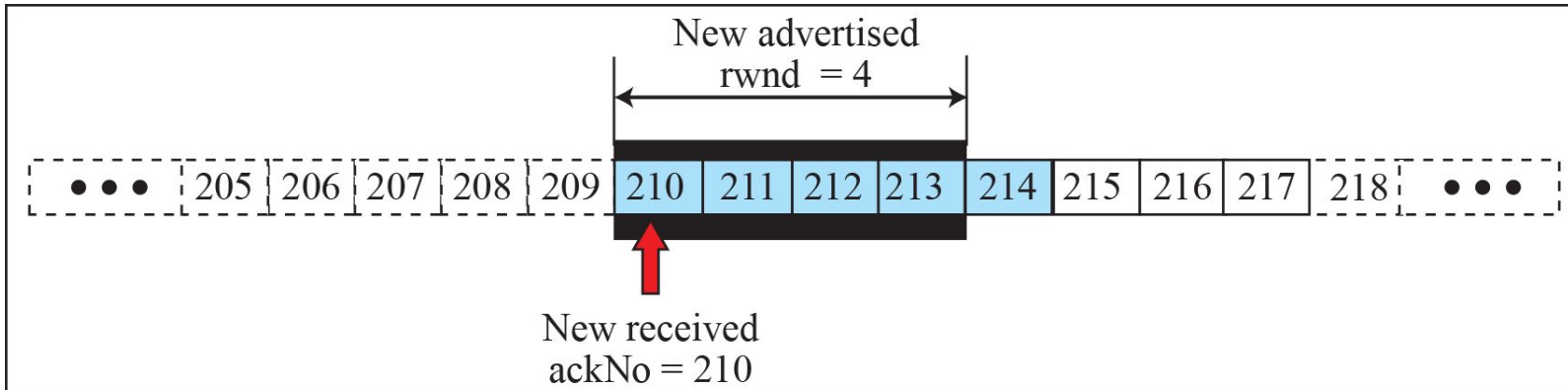
Flow Control

- Figure 3.58 Part a of the figure shows the values of the last acknowledgment and *rwnd*.
- Part b shows the situation in which the sender has sent bytes 206 to 214.
- Bytes 206 to 209 are acknowledged and purged. The new advertisement, however, defines the new value of *rwnd* as 4, in which $210 + 4 < 206 + 12$.
- When the send window shrinks, it creates a problem: byte 214, which has already been sent, is outside the window.
- The relation discussed before forces the receiver to maintain the right-hand wall of the window to be as shown in part a, because the receiver does not know which of the bytes 210 to 217 has already been sent.

Flow Control



a. The window after the last advertisement



b. The window after the new advertisement; window has shrunk

Flow Control

- To prevent shrinking of window

$\text{New ACK} + \text{New Window} \geq \text{Old ACK} + \text{old window}$

- Window Shutdown

- Receiver shutdown window by sending $\text{rwnd} = 0$ when its buffer full
- Sender stop sending data until new window arrive
- Still send 1B data to prevent deadlock called as probing.

Flow Control

- Silly Window Syndrome

41B datagram contain only 1B data which is inefficient

- Syndrome created by sender:

- it is created when application layer creates data slowly

- Solution is Nagle's algo

- ❖ First segment id 1B data

- ❖ Second segment send when a maximum segment size of data accumulated.

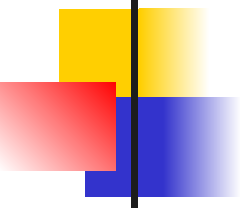
- ❖ Third segment will be send if ACK recieved or have enough data.

- ❖ It depend on speed of application program to create data and network speed to transmit

Flow Control

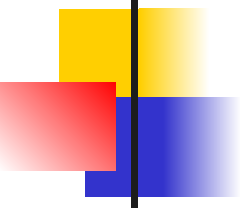
- Syndrome created by receiver:
- it is created when receiver consume data slowly because the buffer will be full => rwnd=0 => sender send 1B Segment
- Solution
- ❖ Clark Solution

Send ACK immediately but keep rwnd=0 until enough space to accommodate a segment
- ❖ Delay ACK
- ✓ Receiver wait till enough space then send ACK
- ✓ Delay should not be >500ms



Error Control

- TCP is a reliable transport-layer protocol.
- This means that an application program that delivers a stream of data to TCP relies on TCP to deliver the entire stream to the application program on the other end in order, without error, and without any part lost or duplicated.



Error Control

- *Checksum*

- *Acknowledgment*

 - ◆ *Cumulative Acknowledgment (ACK)*

 - ◆ *Selective Acknowledgment (SACK)*

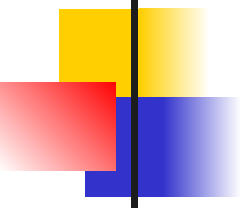
- *Generating Acknowledgments*

- *Retransmission*

 - ◆ *Retransmission after RTO*

 - ◆ *Retransmission after Three Duplicate ACK*

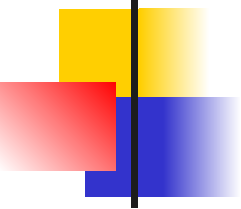
- *Out-of-Order Segments*



Error Control

Some Scenarios

- ❖ *Normal Operation*
- ❖ *Lost Segment*
- ❖ *Fast Retransmission*
- ❖ *Delayed Segment*
- ❖ *Duplicate Segment*
- ❖ *Automatically Corrected Lost ACK*
- ❖ *Correction by Resending a Segment*
- ❖ *Deadlock Created by Lost Acknowledgment*

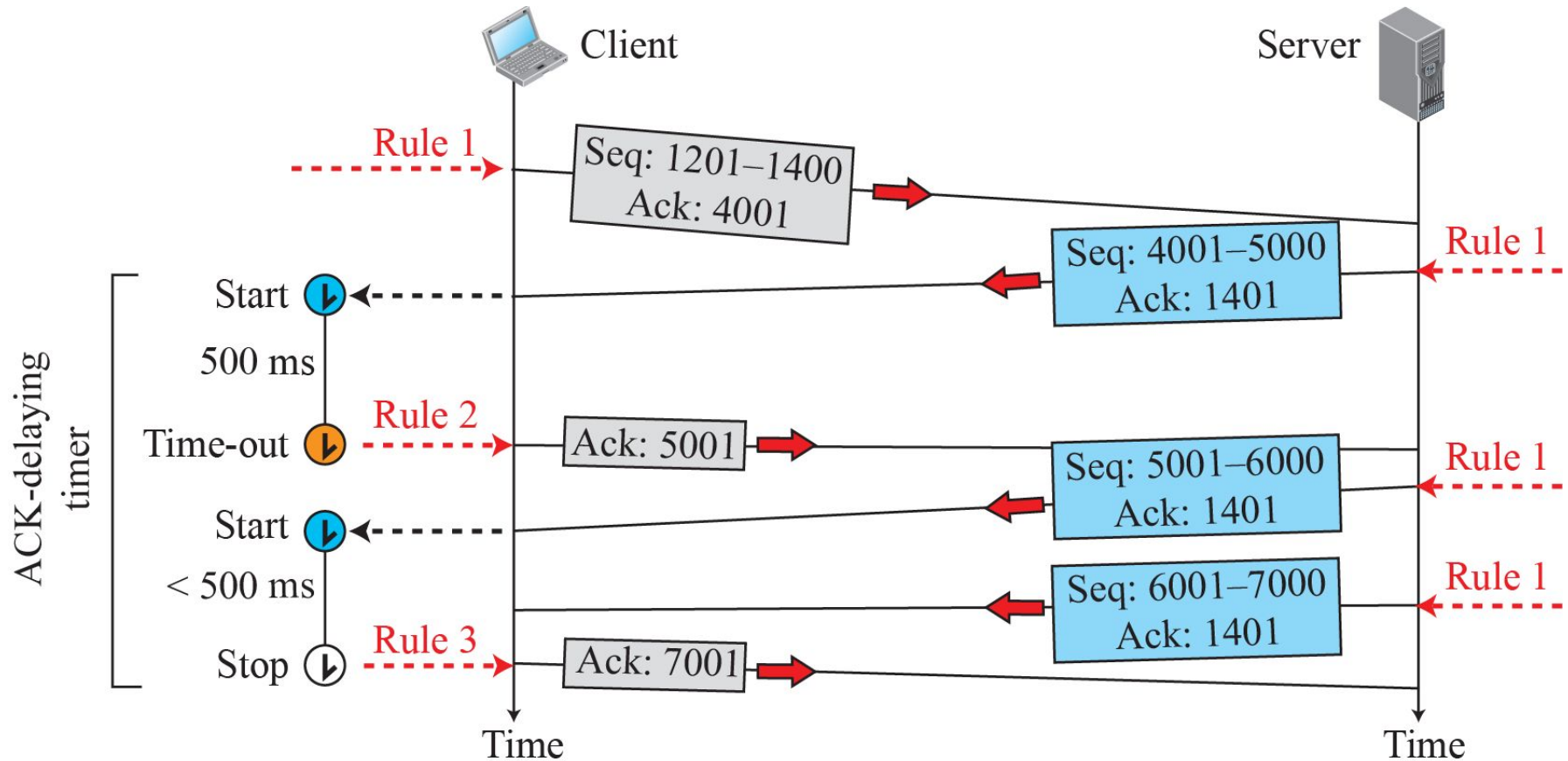


Error Control

- Error control achieved using three ways:
 1. Checksum
 2. ACK
 - Cumulative ACK
 - Selective ACK
 3. Time out
 - Retransmission after time out
 - Retransmission after 3 duplicate ACK

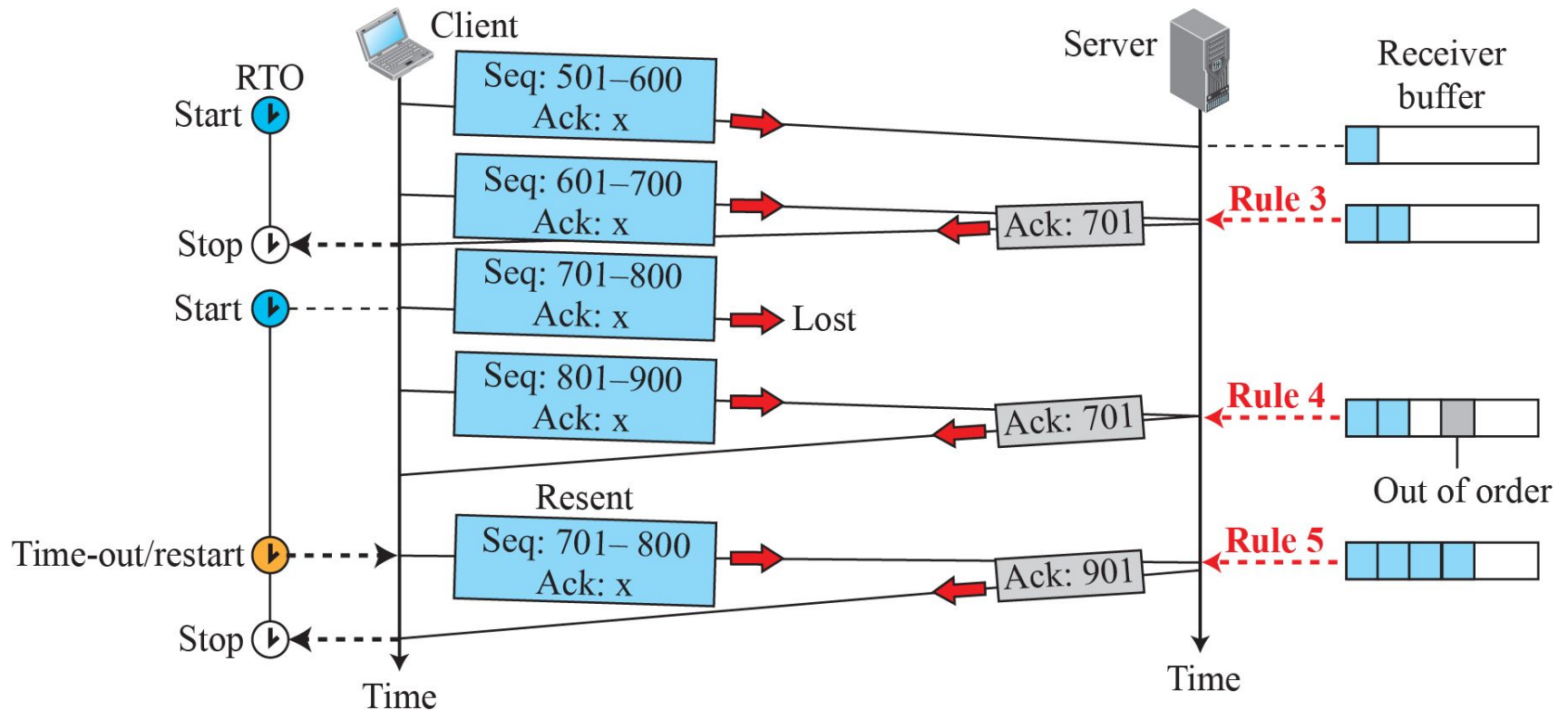
Error Control

Normal Operation



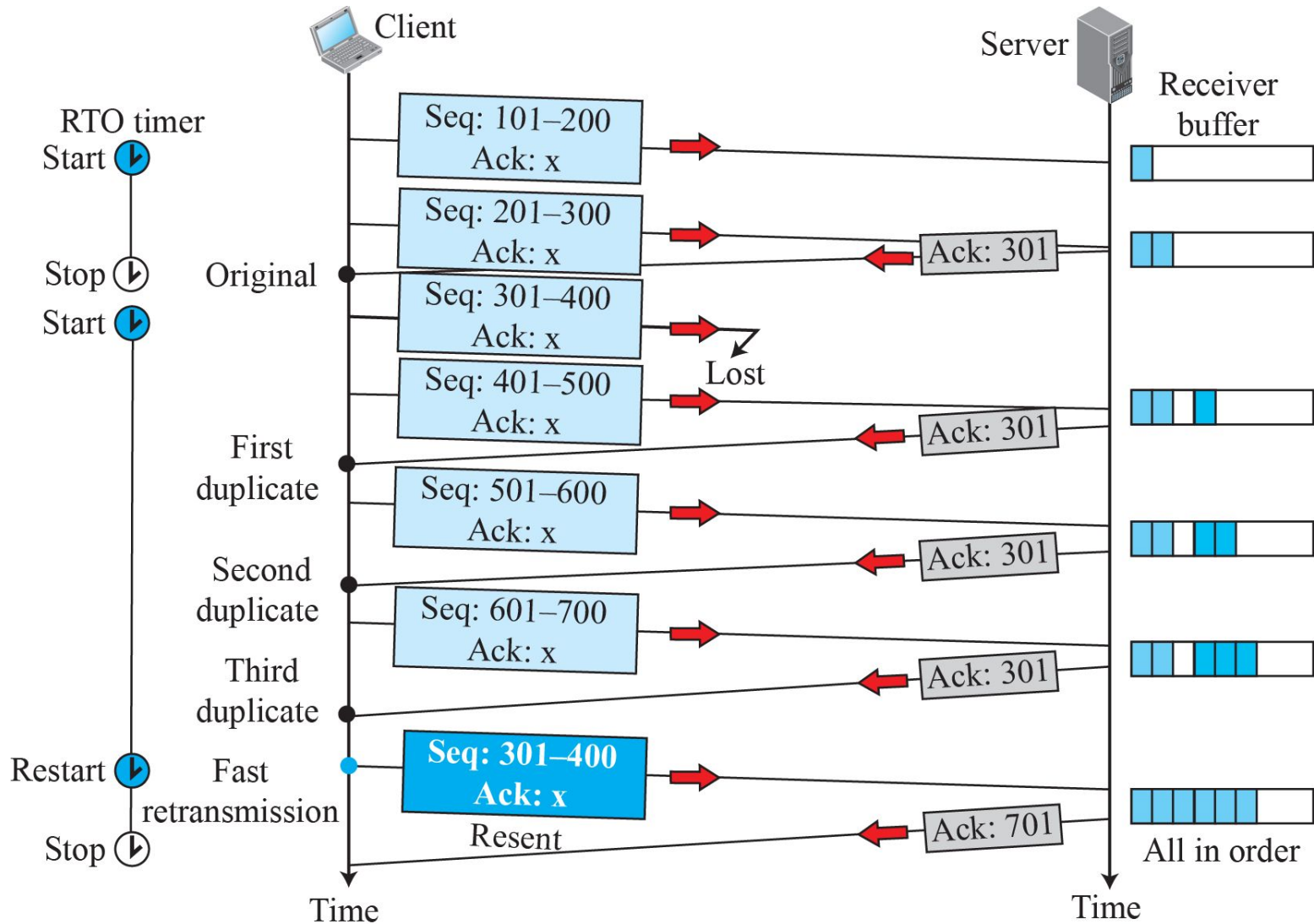
Error Control

Lost Segment



Error Control

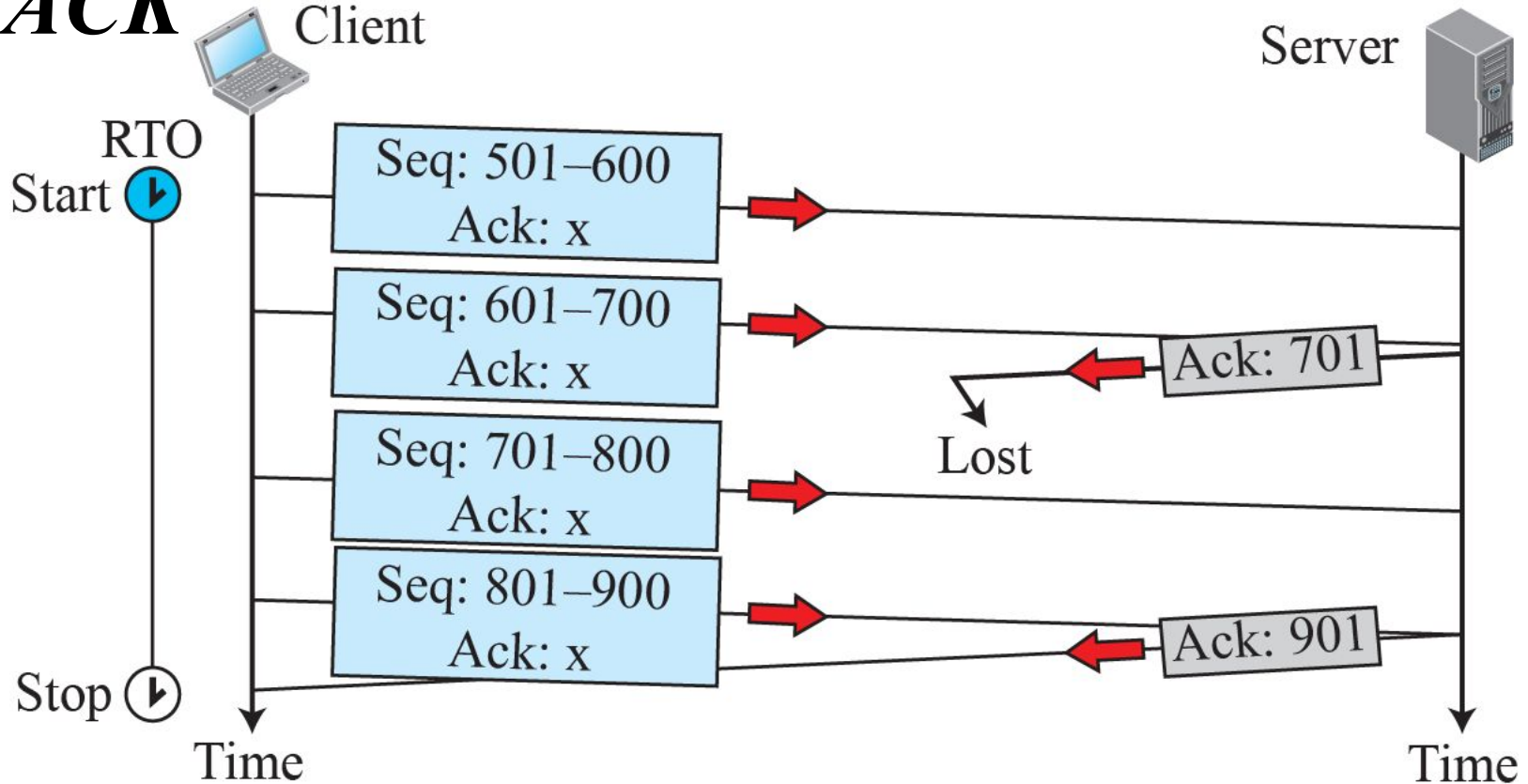
Fast Retransmission



Error Control

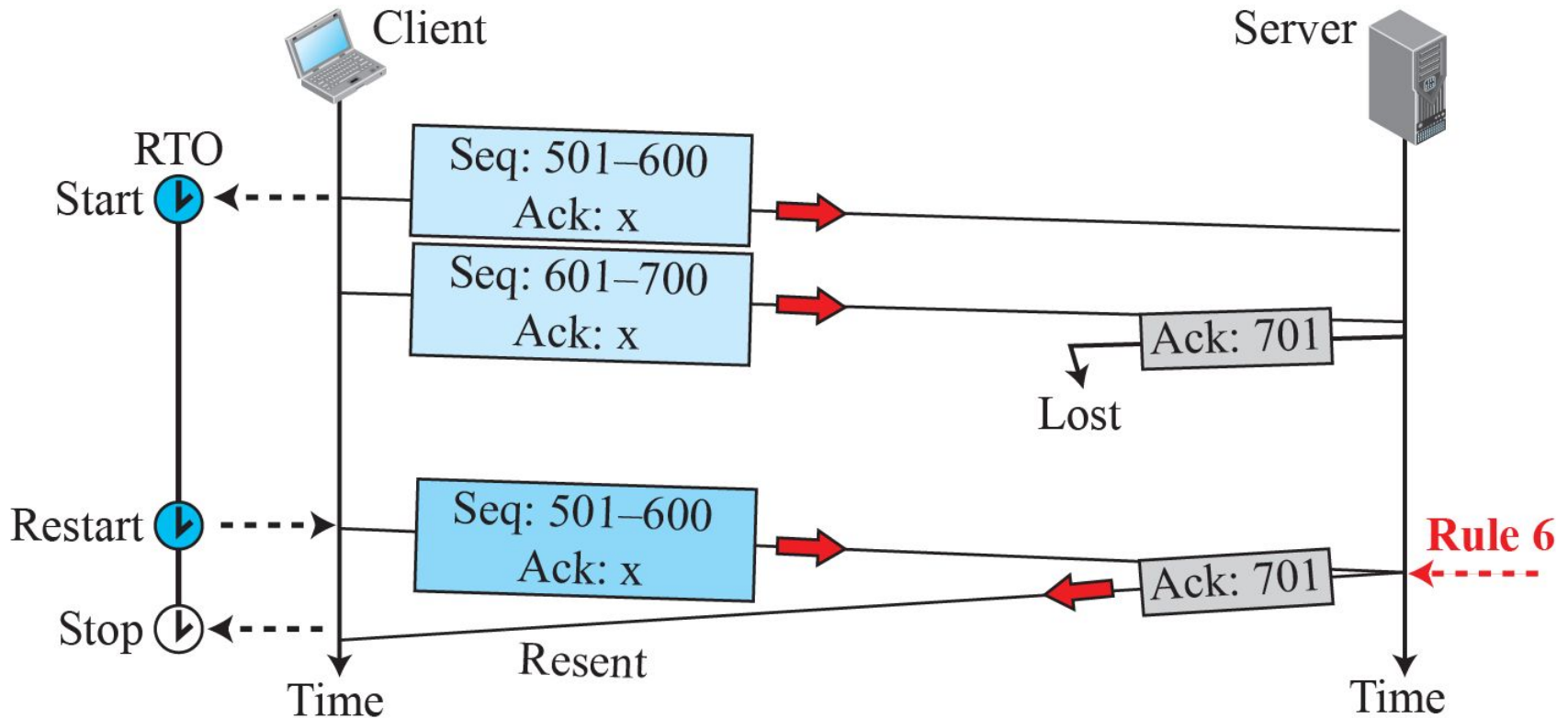
Automatically Corrected Lost

ACK



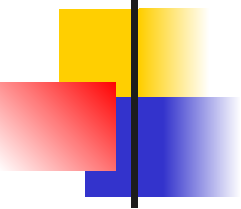
Error Control

Correction by Resending a Segment



Question

- Suppose a TCP client sends 4 segments each of 100bytes with a initial send window size of 500bytes and the sequence number of the first packet is 1. Due to some problem, the third packet is lost in the network. Assume the transport layer delivers the packet immediately to the application layer after receiving it. Draw and explain the time line diagram for the above scenrio containing sequence number, acknowledgment number, and window size in the requisite packets.



TCP Congestion Control

TCP uses different policies to handle the congestion in the network. We describe these policies in this section.

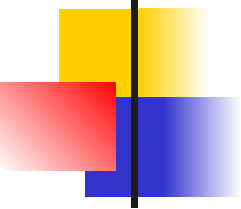
- ☐ *Congestion Window*

- ☐ *Congestion Detection*

- ☐ *Congestion Policies*

- ◆ *Slow Start: Exponential Increase*

- ◆ *Congestion Avoidance: Additive Increase*



TCP Congestion Control

☐ *Policy Transition*

- ◆ *Tahoe TCP*

- ◆ *Reno TCP*

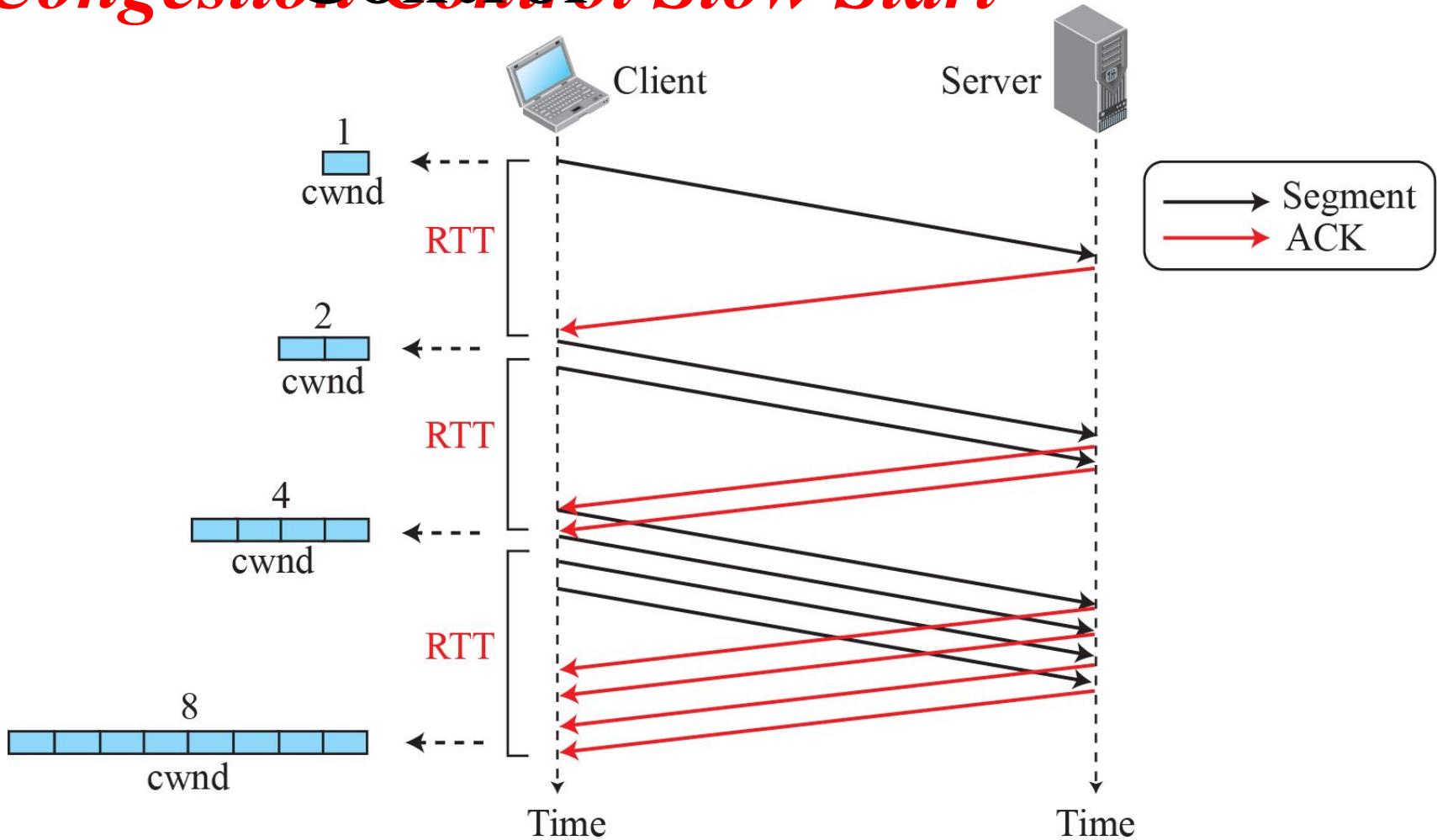
- ◆ *NewReno TCP*

☐ *Additive Increase, Multiplicative Decrease*

☐ *TCP Throughput*

TCP Congestion

Congestion Control Slow Start



TCP Congestion Control

Slow Start

If an ACK arrives, $cwnd = cwnd + 1$.

If we look at the size of the $cwnd$ in terms of round-trip times (RTTs), we find that the growth rate is exponential in terms of each round trip time, which is a very aggressive approach:

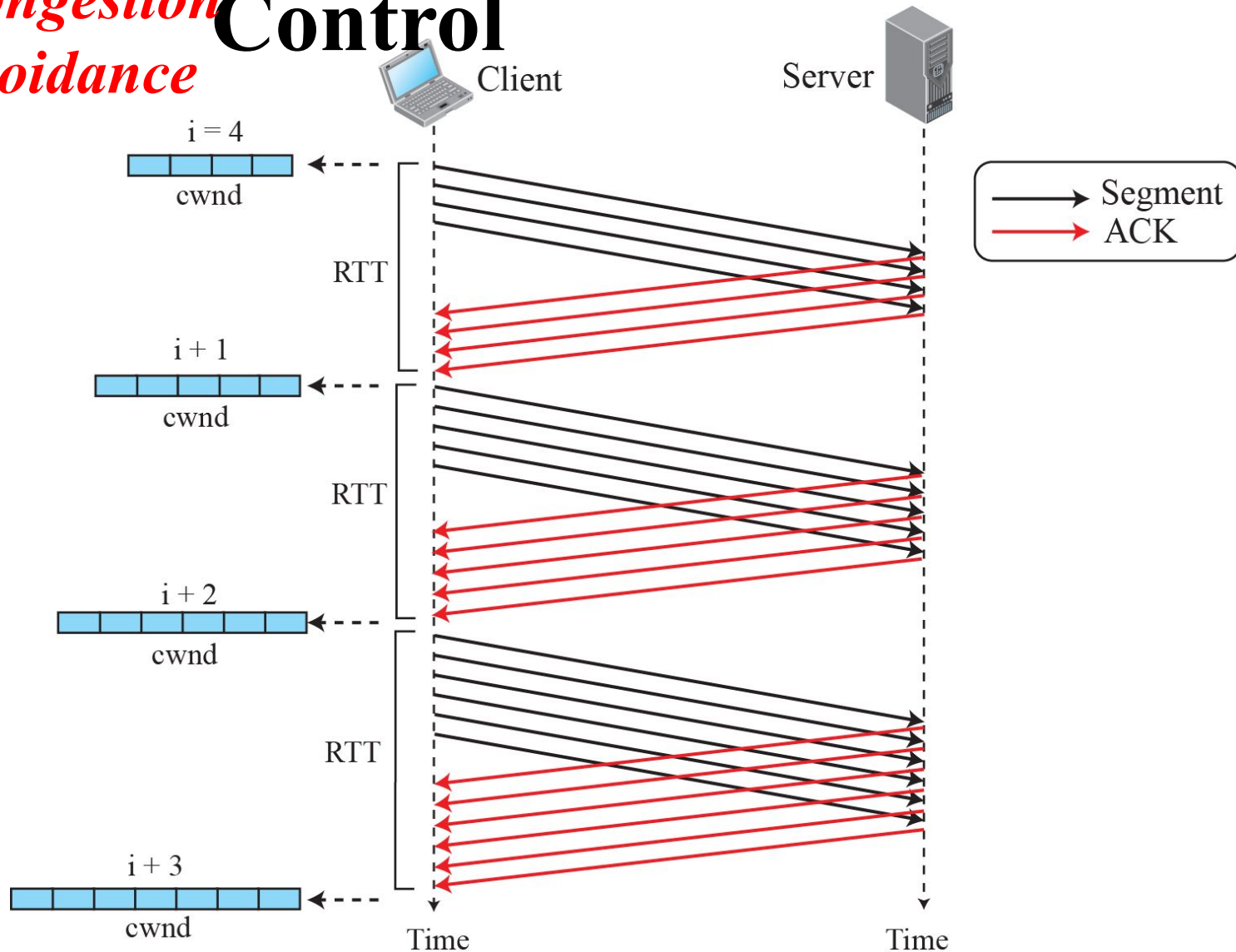
Start	→	$cwnd = 1 \rightarrow 2^0$
After 1 RTT	→	$cwnd = cwnd + 1 = 1 + 1 = 2 \rightarrow 2^1$
After 2 RTT	→	$cwnd = cwnd + 2 = 2 + 2 = 4 \rightarrow 2^2$
After 3 RTT	→	$cwnd = cwnd + 4 = 4 + 4 = 8 \rightarrow 2^3$

A slow start cannot continue indefinitely. There must be a threshold to stop this phase. The sender keeps track of a variable named *ssthresh* (slow-start threshold). When the size of the window in bytes reaches this threshold, slow start stops and the next phase starts.

In the slow-start algorithm, the size of the congestion window increases exponentially until it reaches a threshold.

TCP Congestion

Congestion Control



Copyright © The McGraw-Hill Companies, Inc. Permission required for reproduction or display.

TCP Congestion

Congestion Avoidance Control

If an ACK arrives, $cwnd = cwnd + (1/cwnd)$.

In other words, the size of the window increases only $1/cwnd$ portion of MSS (in bytes). In other words, all segments in the previous window should be acknowledged to increase the window 1 MSS bytes.

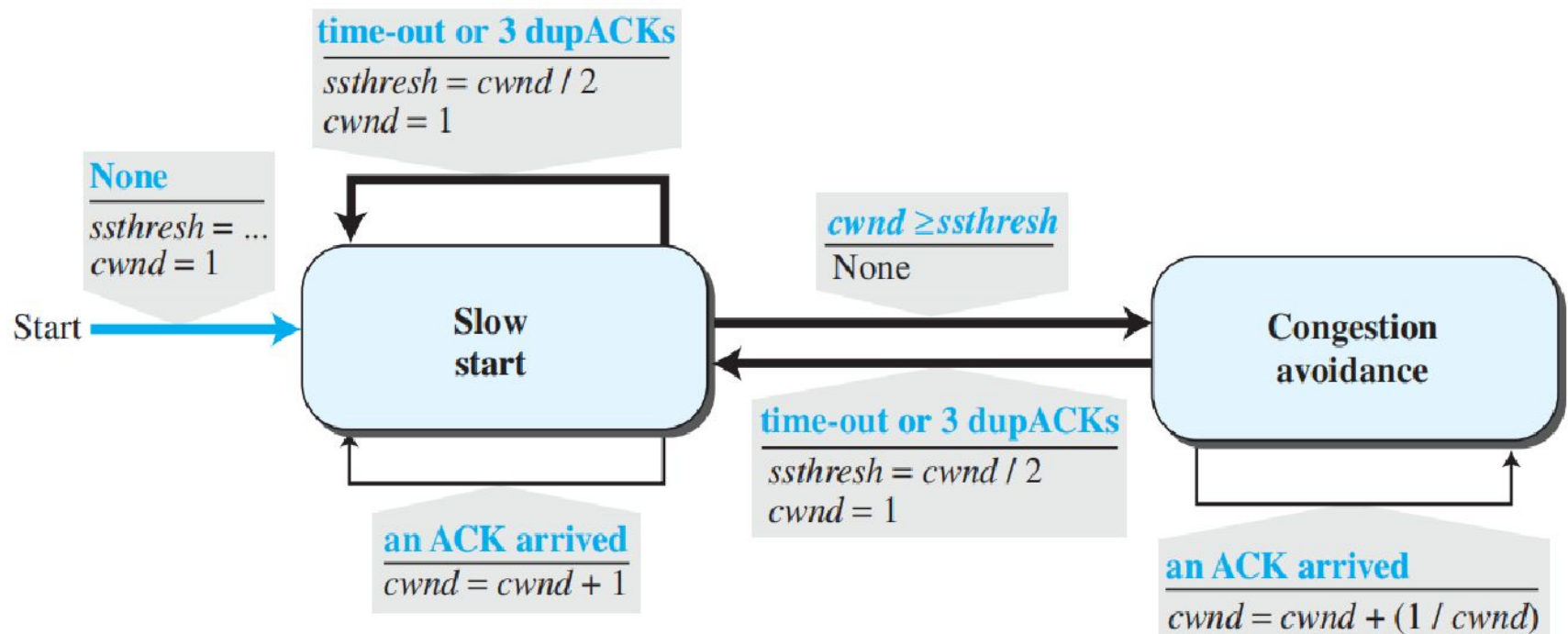
Start	→	$cwnd = i$
After 1 RTT	→	$cwnd = i + 1$
After 2 RTT	→	$cwnd = i + 2$
After 3 RTT	→	$cwnd = i + 3$

In the congestion-avoidance algorithm, the size of the congestion window increases additively until congestion is detected.

TCP Congestion Control

Tahoe TCP Congestion Control

Figure 3.68 FSM for Tahoe TCP



TCP Congestion

Tahoe TCP Congestion Control

Figure 3.69 shows an example of congestion control in a Tahoe TCP.

TCP starts data transfer and sets the *ssthresh* variable to an ambitious value of 16 MSS.

TCP begins at the slow-start (SS) state with the *cwnd* = 1. The congestion window grows exponentially, but a time-out occurs after the third RTT (before reaching the threshold). TCP assumes that there is congestion in the network. It immediately sets the new *ssthresh* = 4 MSS (half of the current *cwnd*, which is 8) and begins a new slow start (SA) state with *cwnd* = 1 MSS.

The congestion grows exponentially until it reaches the newly set threshold.

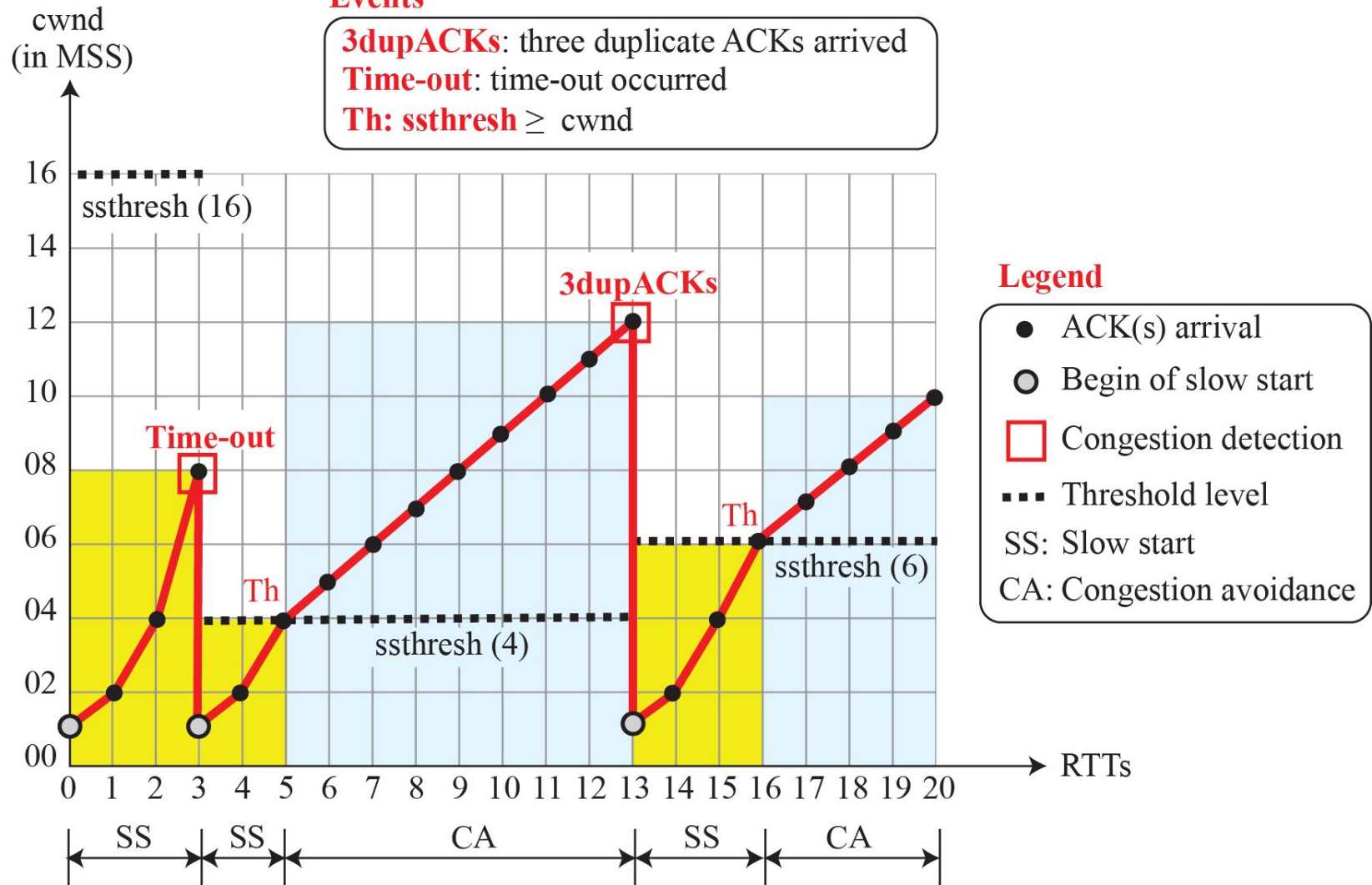
TCP Congestion

Tahoe TCP Congestion Control

- TCP now moves to the congestion avoidance (CA) state and the congestion window grows additively until it reaches $cwnd = 12$ MSS.
- At this moment, three duplicate ACKs arrive, another indication of the congestion in the network.
- TCP again halves the value of $ssthresh$ to 6 MSS and begins a new slow-start (SS) state.
- The exponential growth of the $cwnd$ continues. After RTT 15, the size of $cwnd$ is 4 MSS.
- After sending four segments and receiving only two ACKs, the size of the window reaches the $ssthresh$ (6) and the TCP moves to the congestion avoidance state.
- The data transfer now continues in the congestion avoidance (CA) state until the connection is terminated after RTT 20.

TCP Congestion

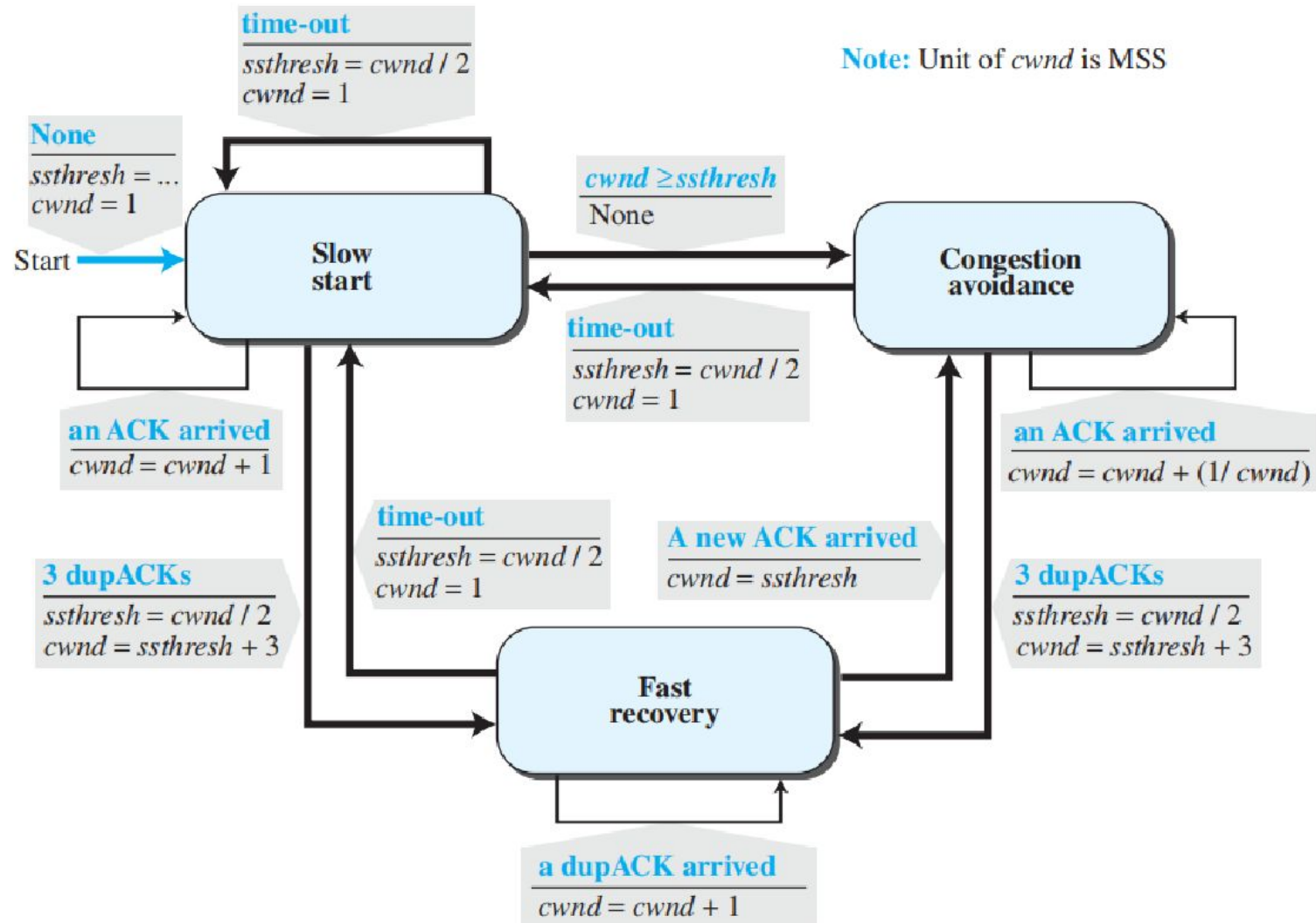
Tahoe TCP Congestion Control



TCP Congestion

Reno TCP Congestion Control

Figure 3.70 FSM for Reno TCP



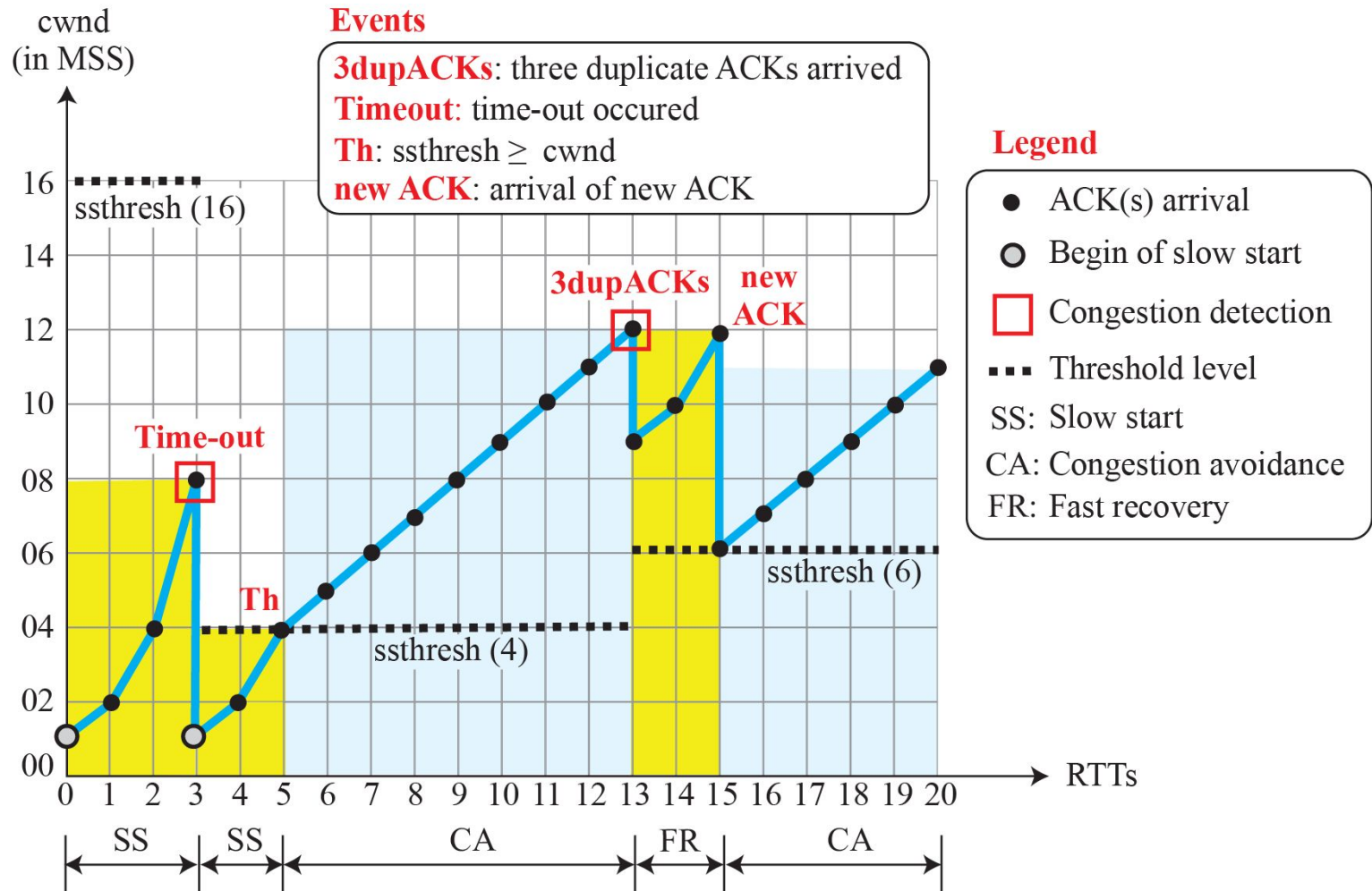
TCP Congestion

Reno TCP Congestion Control

Figure 3.71 shows the same situation as Figure 3.69, but in Reno TCP. The changes in the congestion window are the same until RTT 13 when three duplicate ACKs arrive. At this moment, Reno TCP drops the *ssthresh* to 6 MSS, but it sets the *cwnd* to a much higher value ($ssthresh + 3 = 9$ MSS) instead of 1 MSS. It now moves to the fast recovery state. We assume that two more duplicate ACKs arrive until RTT 15, where *cwnd* grows exponentially. In this moment, a new ACK (not duplicate) arrives that announces the receipt of the lost segment. It now moves to the congestion avoidance state, but first deflates the congestion window to 6 MSS as though ignoring the whole fast-recovery state and moving back to the previous track.

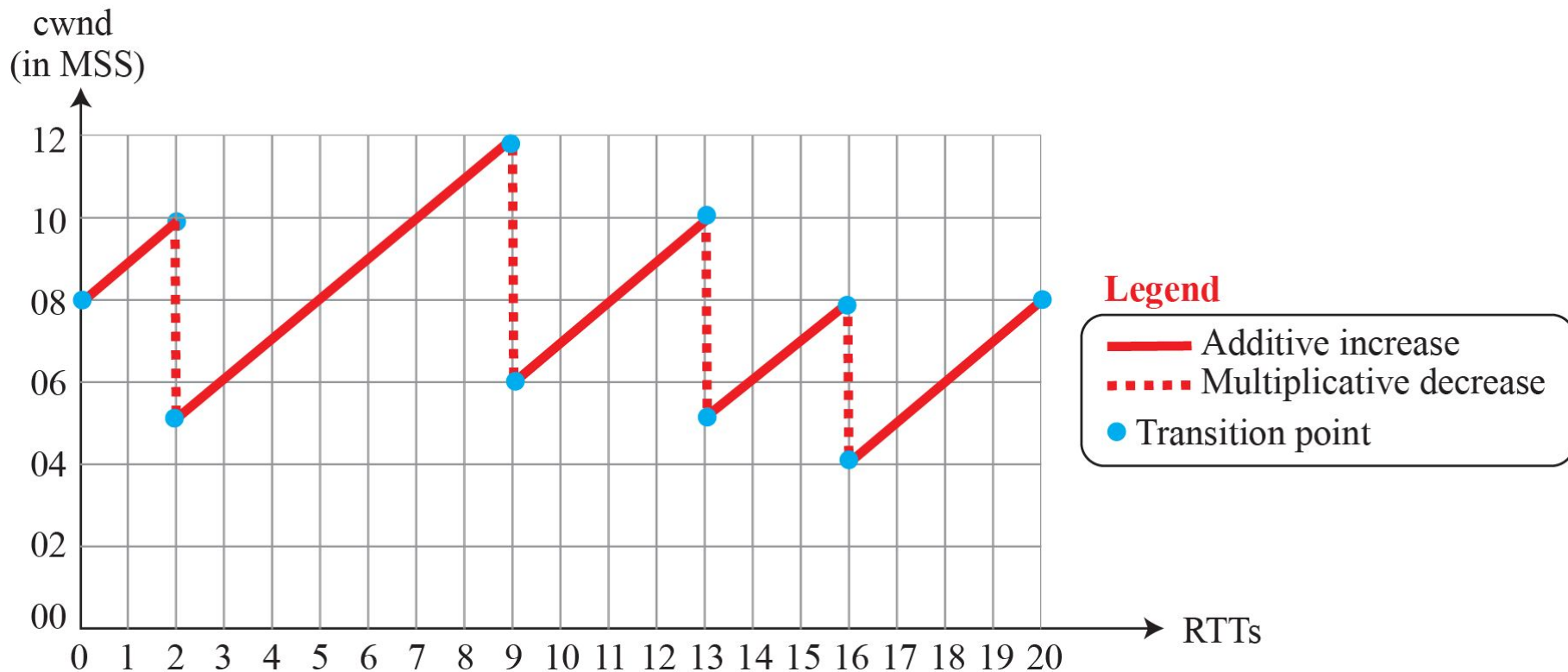
TCP Congestion

Reno TCP Congestion Control



TCP Congestion Control

AIMD Control



TCP Congestion

TCP Congestion Control

If MSS = 10 KB (kilobytes) and RTT = 100 ms in Figure 3.72, we can calculate the throughput as shown below.

$$W_{\max} = (10 + 12 + 10 + 8 + 8) / 5 = 9.6 \text{ MSS}$$

$$\text{Throughput} = (0.75 W_{\max} / \text{RTT}) = 0.75 \times 960 \text{ kbps} / 100 \text{ ms} = 7.2 \text{ Mbps}$$

TCP Congestion

TCP Congestion Control

Q.1. Let the slowstart phase begins with $cwnd=1$ at time $t=0$ and maximum segment size 1500B. What is the RTT value when the congestion window is greater than 25KB?

Q.2. The value of the current congestion window($cwnd$) just before a time out was 32KB. Assume the maximum segment size(MSS) is 2KB. What will be the value of $cwnd$ (in KB) after transmission of the next five packets?

Question

Consider the data transfer using TCP over a 1 Gbps link. Assuming that the maximum segment lifetime (MSL) is set to 60 seconds, the minimum number of bits required for the sequence number field of the TCP header, to prevent the sequence number space from wrapping around during the MSL is _____.

Consider a 100 Mbps link between an earth station (sender) and a satellite (receiver) at an altitude of 2100 km. The signal propagates at a speed of 3×10^8 m/s. The time taken (in milliseconds, rounded off to two decimal places) for the receiver to completely receive a packet of 1000 bytes transmitted by the sender is _____.

Question

Consider the following statements.

I. TCP connections are full duplex

II. TCP has no option for selective acknowledgment

III. TCP connections are message streams

-
- A** Only I is correct
-
- B** Only I and III are correct
-
- C** Only II and III are correct
-
- D** All of I, II and III are correct

Question

Suppose two hosts use a TCP connection to transfer a large file. Which of the following statements is/are FALSE with respect to the TCP connection?

- I. If the sequence number of a segment is m , then the sequence number of the subsequent segment is always $m + 1$.
- II. If the estimated round trip time at any given point of time is t sec, the value of the retransmission timeout is always set to greater than or equal to t sec.
- III. The size of the advertised window never changes during the course of the TCP connection.
- IV. The number of unacknowledged bytes at the sender is always less than or equal to the advertised window

-
- ☒ A III only
-
- ☐ B I and III only
-
- ☐ C I and IV only
-
- ☐ D II and IV only

Question

The transport layer protocols used for real time multimedia, file transfer, DNS and email, respectively are

- A TCP, UDP, UDP and TCP
- B UDP, TCP, TCP and UDP
- C UDP, TCP, UDP and TCP
- D TCP, UDP, TCP and UDP

Question

What is the maximum size of data that the application layer can pass on to the TCP layer below?

- A Any size
- B 2^{16} bytes-size of TCP header
- C 2^{16} bytes
- D 1500 bytes

Question

Packets of the same session may be routed through different paths in

- ☐ A TCP, but not UDP
- ☐ B TCP and UDP
- ☐ C UDP, but not TCP
- ☐ D Neither TCP, nor UDP

Question

Consider a TCP connection between a client and a server with the following specifications: the round trip time is 6 ms, the size of the receiver advertised window is 50 KB, slow start threshold at the client is 32 KB, and the maximum segment size is 2 KB. The connection is established at time $t = 0$. Assume that there are no timeouts and errors during transmission.

Then the size of the congestion window (in KB) at time $t + 60$ ms after all acknowledgements are processed is _____.

Let the size of congestion window of a TCP connection be 32 KB when a timeout occurs. The round trip time of the connection is 100 msec and the maximum segment size used is 2 KB. The time taken (in msec) by the TCP connection to get back to 32 KB congestion window is _____.

Question

Consider the following statements regarding the slow start phase of the *TCP* congestion control algorithm. Note that *cwnd* stands for the *TCP* congestion window and *MSS* denotes the Maximum Segment Size.

- (i) The *cwnd* increases by 2 *MSS* on every successful acknowledgment.
- (ii) The *cwnd* approximately doubles on every successful acknowledgement.
- (iii) The *cwnd* increases by 1 *MSS* every round trip time.
- (iv) The *cwnd* approximately doubles every round trip time.

Which one of the following is correct?

-
- ☐ A Only (ii) and (iii) are true
-
- ☐ B Only (i) and (iii) are true
-
- ☐ C Only (iv) is true
-
- ☐ D Only (i) and (iv) are true

Question

Consider an instance of TCP's Additive Increase Multiplicative Decrease (AIMD) algorithm where the window size at the start of the slow start phase is 2 MSS and the threshold at the start of the first transmission is 8 MSS. Assume that a timeout occurs during the fifth transmission. Find the congestion window size at the end of the tenth transmission.

-
- A 8 MSS
 - B 14 MSS
 - C 7 MSS
 - D 12 MSS
-

Question

Suppose that the maximum transmit window size for a TCP connection is 12000 bytes. Each packet consists of 2000 bytes. At some point of time, the connection is slow start phase with a current transmit window of 4000 bytes. Subsequently, the transmitter receives two acknowledgments. Assume that no packets are lost and there are no time-outs. What is the maximum possible value of the current transmit window?

- ☐ A 4000 bytes
- ☐ B 8000 bytes
- ☐ C 10000 bytes
- ☐ D 12000 bytes

Question

Q - During data communication between a TCP client and server, the receive window size of the client is 900 bytes. At a particular instance, TCP server sends an ACK with an acknowledgment number 101 and receive window size 550. After receiving the ACK, the TCP client sends 200bytes of data. What should be the send window size in bytes at the TCP client and the sequence number of the packet sent?